

Revisiting average case complexity of multilevel syllogistic: From the 1995 Courant Technical Report to Lean 4 Formalization

Lars Warren Ericson¹

¹Catskills Research Company

¹lars.ericson@catskillsresearch.com

June 14, 2026

ORCID: 0000-0001-8299-9361

Primary Category: cs.LO (Logic in Computer Science)

Secondary Category: cs.CC (Computational Complexity)

Lean 4 formalization: https://github.com/catskillsresearch/avg_case_mls

Abstract

We describe a Lean 4 formalization revisiting NYU Courant Technical Report TR1995-711 on the average-case complexity of Multilevel Syllogistic (MLS). The development encodes Reischuk–Schindelhauer average-case classes, an axiomatic MLS/EMLS semantics layer, a partial Ferro–Omodeo–Schwartz decision procedure with proved soundness and partial completeness on a membership-free fragment, serialization and step budgets, and conditional NP-average completeness and non-AvP hardness corollaries modulo explicitly documented structural axioms. Full Lean sources are inlined in the appendix modules.

0.1 1. Introduction: The Vision of AvCom in Program Verification

In the late 1970s and throughout the 1980s, the “Correct Program Technology” (CPT) movement, spearheaded by figures such as Martin Davis and Jacob T. Schwartz, envisioned a software development pipeline where programmers wrote code alongside mathematical specifications [DS77]. A compiler, integrated with an automated theorem prover, would then verify that the program met its specification.

To make this feasible, researchers sought to enrich Floyd-Hoare verification tools with decision procedures for decidable sublanguages of set theory and arithmetic. These logic fragments—such as Multilevel Syllogistic (MLS) and Elementary Multilevel Syllogistic (EMLS)—modeled the set-theoretic operations typical of high-level programming languages like SETL [Sny90a].

However, worst-case complexity analysis posed a major roadblock: the decision problems for MLS and EMLS are NP-complete, and extensions involving Presburger arithmetic exhibit exponential or double-exponential worst-case bounds. To bypass this, researchers pointed to early optimistic results by Goldberg [Gol79], which suggested that the Davis-Putnam procedure and resolution-based SAT solvers could perform exceptionally well on “average” inputs.

In their 1995 Courant Institute Technical Report, “*The average case complexity of multilevel syllogistic*” (TR1995-711), Jim Cox, Lars Ericson, and Bud Mishra analyze the average-case tractability of decidable sublanguages of set theory and arithmetic—such as Multilevel Syllogistic (MLS), Elementary Multilevel Syllogistic (EMLS), and Fractional Programming/Linear Programming (FP/LP). Because these languages are NP-complete in the worst case, the authors

turn to the formal framework of **average-case complexity (AvCom)** to evaluate whether heuristic decision procedures could perform well on average. They marry the mathematical foundations of AvCom with set-theoretic decision procedures to determine whether a typical instance of these verification problems is truly tractable.

This note revisits that report in light of modern proof assistants. Our **proof program** is to establish the AvCom terms and concepts from the literature in Lean 4, formalize MLS and its decision procedures, and then grind out the TR1995-711 theorems—or document honestly where the effort stops. The goal is not to defend the 1995 report; it is to see what survives contact with a proof assistant.

0.1.1 Proof program: outcomes we accept

Outcome	What it means
Proofs check	Definitions match the literature; TR1995-711 theorems are formalized and proved (possibly with explicitly stated hypotheses).
Lean is not expressive enough (yet)	We hit a clear blocker: missing Mathlib infrastructure, noncomputability, or encoding issues. Document the gap precisely.
Paper proofs are wrong	A step in TR1995-711 does not follow from definitions, or a reduction/domination bound fails. Document the counterexample or missing lemma.
Field definitions are not solid	Levin/RS93/distNP/AvP formulations are ambiguous, inconsistent, or not formalizable without arbitrary choices. Document the choice we had to make and what breaks.

We do not treat **sorry** or axioms as success. Axioms are temporary scaffolding with a ticket to remove them.

0.1.2 What TR1995-711 claims (targets)

The report **states proofs**, not conjectures, for:

- **NP-average completeness** of MLS satisfiability (Corollary 5.1), EMLS, FP/LP, and related fragments.
- **Distributional reductions** with domination, from bounded halting (NBH) and other distNP-complete cores.
- **Corollaries** tying average completeness to absence of AvP on simple POL-rankable distributions (conditional on standard collapse hypotheses such as $\text{NEXP} \neq \text{EXP}$).

Our Lean development should eventually either prove these statements from formal definitions or refute a specific step. §9 grades each subphase (Phase 2A is complete; see Results).

0.1.3 Phased plan

Context and Lean infrastructure appear in §§2–4; Phase 1 (AvCom) is §5, Phase 2 (MLS) is §6. Hardness and completeness (Phases 4–5) need both layers. Subphases track progress within each phase; §9 is the report card.

Phase 0 — Infrastructure. Lake project, smoke tests, paper synced to this document.
Status: complete.

Phase 1 — AvCom vocabulary (literature → Lean).

Subphase	Goal	Lean target
1A	Inputs and distributions (ğ5)	Bitstring, len, Distribution, DistributionalProblem, IsPolynomial
1B	Rank and inverse bounds	rank, T_inv (no sorry; finite-support or explicit fork)
1C	Average time and dist-time classes	IsAvTime, DistTime, AvDTime
1D	Classes, reductions, completeness	AvP, InDistNP, DistributionalReduction, IsNPAverageComplete

Exit criterion (Phase 1): all subphase definitions compile without **sorry**; basic lemmas and toy distribution tests; forks documented in DEFINITION_FORKS.md.

Phase 2 — MLS embedding and decision procedure.

Subphase	Goal	Lean / doc
2A	MLS syntax + axiomatic semantics	ğ6 — Term, Relation, Formula, evalTerm, evalFormula
2B	EMLS literals, literalToFormula, conjunctToFormula	ğ6
2C	FOS80 decision procedure for satisfiability	ğ7
2D	Problem encoding and step count	serializeFormula, SatMLS, stepsMLS (remove axioms in ğ8)

Exit criterion (Phase 2): 2A–2D complete; no **sorry** on soundness for the proved decision fragment; completeness scoped honestly.

Phase 3 — Worst-case and coding.

Subphase	Goal
3A	SatMLS ∈ NP (witness / certificate)
3B	Formula encoding size lemmas; polynomial bounds on $\ \varphi\ $

Exit criterion: formal NP membership or a written Mathlib blocker.

Phase 4 — TR1995-711 reductions.

Subphase	Goal
4A	NBH (or report’s distNP-complete core) + POL-rankable μ_0
4B	Reduction f , domination into SatMLS / EMLS / FP/LP

Subphase	Goal
4C	NP-average completeness of SatMLS (Corollary 5.1)

Exit criterion: 4C proved or a specific failed obligation recorded.

Phase 5 — AvP consequences.

Subphase	Goal
5A	Conditional non-AvP from completeness + simple rankable μ
5B	SatMLS_average_hard without sorry ; axioms minimized

0.1.4 Methodology

1. **Definitions before theorems** — no **sorry** in defs we label “final.”
2. **One obligation per issue** — each former **sorry** becomes a named lemma with a one-line statement.
3. **Literature pointer** — every definition cites TR1995-711 section or [RS93], [Lev86], etc.
4. **Fork log** — if we must choose (finite support, encoding, POL-rankable vs P-computable), record in `DEFINITION_FORKS.md`.
5. **CI** — `./run_lean_check.sh` must pass; new sorries require a comment `-- Phase Nx, issue #`.
6. **AI-assisted development** — large language models were used as coding assistants (not co-authors); see **Acknowledgements** for scope, tools, and human verification responsibilities.

We grind on Phases 1→5 in dependency order (subphases may be implemented out of order when independent). §5–6 pair mathematics with Lean encodings; §4 covers Lean strategy; §§7–8 cover decision procedures and hardness theorems; §9 grades each subphase; §10 lists further directions.

0.2 2. Historical Context, Terminology, and Reception of TR1995-711

0.2.1 Application and Findings

Cox, Ericson, and Mishra prove that **EMLS, MLS, and FP/LP are NP-average complete**. This implies there are simple, rankable distributions that will frustrate any decision algorithm for these problems, forcing super-polynomial average-case running times unless deterministic and nondeterministic exponential time are equal ($\text{NEXP} = \text{EXP}$).

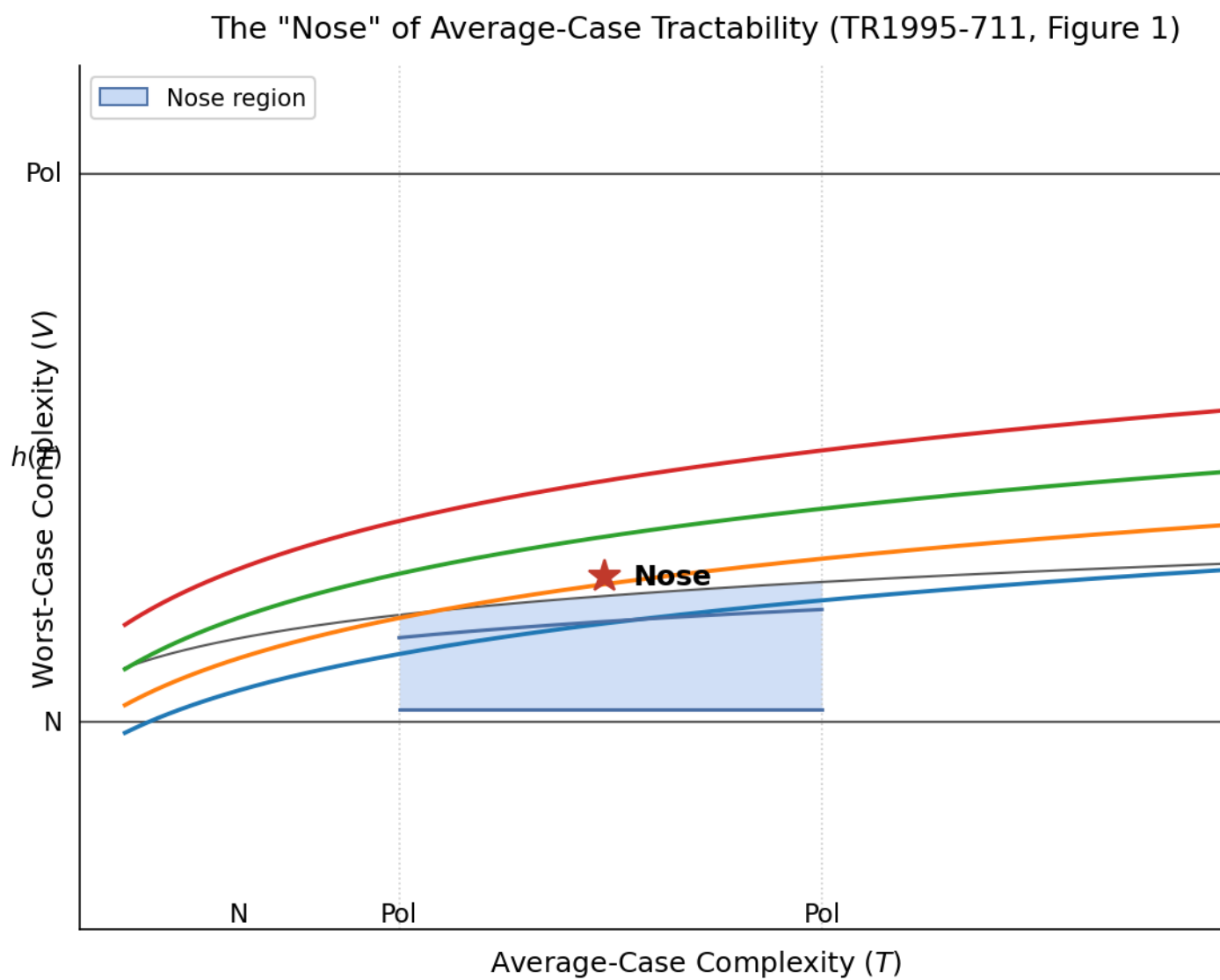


Figure 1: The average-case complexity "nose" diagram (TR1995-711, Figure 1)

0.2.2 The Concept of “The Nose”

The paper features a key visualization of the average-case landscape of NP-complete languages (Figure 1, page 13).

Figure 1 (schematic). Languages L_i are plotted by worst-case complexity V (vertical) and average-case complexity T (horizontal). The shaded **nose** is the tractable region in the polynomial–polynomial corner.

In this diagram, languages L_i are mapped based on their worst-case complexity V (vertical axis) and their average-case complexity T (horizontal axis). * **The Nose** represents the sweet spot of tractability: the shaded region where the worst-case complexity of the ranking function V is bounded by $h(T)$, such that the language still possesses an efficient average-case algorithm.

* Formally, the authors define this boundary as:

$$\text{nose}(L) = \{(T, V) \in (\text{POL}, \text{POL}) : L \in \text{AvDTime}(T, V\text{-rankable})\}$$

- For an NP-average complete problem, the “nose” is trivial or empty under simple distributions, meaning no non-trivial efficient average-case behavior can be guaranteed unless Nondeterministic Exp = Deterministic Exp.

0.2.3 Pre-publication review in 1995

Reviewer Martin Davis asked the authors to give a more pragmatic demonstration of their results before accepting the work into *Communications on Pure and Applied Mathematics* (CPAM). That demonstration never materialized; the concrete heuristics were weak, and the empirical machinery to test these algorithms on large datasets did not yet exist. The paper was never published in CPAM.

That outcome is not the whole story, however. The report’s deeper aim was to supply a **language for describing how hard a typical instance of a verification problem might be**—not to ship a production solver. This historical episode highlights a common turning point in computer science during the mid-1990s: the tension between elegant, highly formal mathematical complexity theory and the messy, empirical reality of practical software engineering.

Three natural questions follow: Were the terms used in the paper invented there, or taken from existing literature? Was the technical report ever referenced? And what became of average-case complexity—and of this particular application to set-theoretic decision procedures—in the intervening thirty years?

0.2.4 Were the Terms Invented in This Paper?

No—the core definitions and terms were not invented in TR1995-711. The authors drew entirely upon the existing complexity literature of the late 1980s and early 1990s:

- **The foundations (AvP, distNP, and the domination condition):** Pioneered by Leonid Levin in his 1986 paper “*Average case complete problems*” [Lev86] and further formalized by Yuri Gurevich [Gur91] and Ben-David, Chor, Goldreich, and Luby [BD-CGL89].
- **The precise formulations (POL, POL-rankable, precise average-case complexity):** Taken directly from Rüdiger Reischuk and Christian Schindelhauer’s 1993 paper, “*Precise average case complexity*” [RS93].

The report’s contribution was not structural novelty in complexity theory itself, but rather its **application**: importing these rigorous, newly developed tools from structural complexity theory and applying them to automated theorem proving and program verification—specifically, showing that set-theoretic fragments such as EMLS and MLS are average-case complete.

0.2.5 Was This Technical Report Ever Referenced?

In terms of direct scientific citations, **TR1995-711** has been almost entirely overlooked. It has virtually zero standard citations in academic journals and did not spawn a direct lineage of follow-up papers in automated theorem proving.

It has nevertheless been kept alive in a specific way: it is cited as a notable applied example on the **Wikipedia page for “Average-case complexity”** (and its translations). Because Wikipedia editors documented it as one of the few explicit applications of Levin’s theory to set-theoretic decision procedures, it remains a known historical reference point in the literature of the field.

0.3 3. Thirty Years of Average-Case Complexity (1995–2026)

Rather than dying, the field of average-case complexity underwent a massive evolution. Its center of gravity migrated away from traditional decision-procedure analysis and became foundational to other, highly successful domains.

0.3.1 Cryptography and Worst-Case-to-Average-Case Reductions

In the late 1990s—beginning with Miklós Ajtai’s landmark 1996 work [Ajt96]—theorists discovered how to prove mathematically that certain average-case problems are hard *assuming only that their worst-case versions are hard*. This paved the way for **lattice-based cryptography** and Oded Regev’s **Learning With Errors (LWE)** framework (2005) [Reg05]. Because cryptography requires that *almost all* generated keys are hard to break (average-case hardness), these frameworks are now the basis for modern post-quantum cryptography standards.

0.3.2 Smoothed Analysis

In 2001, Daniel Spielman and Shang-Hua Teng introduced **smoothed analysis** [ST01]. They argued that analyzing an algorithm under a purely random, mathematically convenient distribution (the “average case”) is often unrealistic and overly pessimistic. Instead, they measured performance under *slight random perturbations of worst-case inputs*. This successfully explained why algorithms like the Simplex method for linear programming run in polynomial time in practice despite worst-case exponential complexity—bridging the gap between theory and practical heuristics in a way that Levin-style rankable distributions alone could not.

0.3.3 Statistical Inference and Machine Learning

In the 2010s and 2020s, average-case complexity found a major new home in high-dimensional statistics and machine learning. Researchers now study the **information–computation gap**—situations where an estimation problem (such as the Planted Clique problem or Tensor PCA) is theoretically solvable given infinite time, but computationally intractable on average for any polynomial-time algorithm.

0.3.4 What Happened to This Specific Use Case (Set-Theoretic Decision Procedures)?

Martin Davis’s skepticism was vindicated by the path the automated theorem proving community took. The attempt to build program verification tools around highly specialized, decidable, average-case-analyzed set-theoretic sublanguages (such as EMLS or MLS) largely became a dead end. The community pivoted instead toward **SMT (Satisfiability Modulo Theories) solvers** (such as Z3 and CVC5) [deM08] and modern **SAT solvers**.

This transition succeeded for several reasons:

- **The structure of real-world code:** Theoretical average-case complexity assumes random inputs under simple mathematical distributions (such as the linear-time rankable distributions in Cox et al.’s paper). Real-world software verification problems, however, are highly structured and logical; they are not random.
- **The triumph of CDCL and heuristics:** Modern SMT/SAT solvers utilize Conflict-Driven Clause Learning (CDCL) and highly engineered heuristics. Empirically, these tools routinely solve industrial-scale verification formulas with millions of variables, bypassing theoretical worst-case or average-case intractability.
- **Empirical benchmarks over proofs:** Rather than proving mathematical average-case tractability, the community created massive, standardized libraries of real-world problem benchmarks (such as SMT-LIB). Solver progress is now measured empirically—a far more pragmatic and successful path than the one Davis requested for CPAM.

The specific marriage of AvCom to MLS decision procedures was largely abandoned for the same reason: proving average-case hardness under mathematically simple, rankable distributions did not reflect the highly structured formulas generated by real-world compilers.

The specific marriage of AvCom to MLS decision procedures was largely abandoned for the same reason: proving average-case hardness under mathematically simple, rankable distributions did not reflect the highly structured formulas generated by real-world compilers.

0.4 4. Lean 4 Formalization Strategy

§§5–6 pair the mathematical definitions with their Lean modules. This section summarizes project infrastructure and design choices. The live code lives in [AvgCaseMls/](#) and is checked by `./run_lean_check.sh` and `./run_lean_tests.sh` (see [INSTALLING_LEAN.md](#)).

0.4.1 Mathlib and the Complexity-Theory Gap

Through 2025–2026, Mathlib4 has begun to host **worst-case** complexity infrastructure (polynomial-time Turing machines, P, NP, and related material under active development). **Average-case complexity—distributional problems, rank functions, $\text{Av}(T)$, DistTime , distributional reductions, and AvP —is not yet a standard Mathlib layer.** TR1995-711 is therefore a natural stress test: it requires both a deep embedding of set-theoretic syntax *and* a bespoke AvCom library built on [RS93].

Our approach mirrors [icon2lean](#): 1. **Definitions first** — encode rank, $\text{Av}(T)$, $\text{DistTime}(T)$, AvP , and distNP alongside the AvCom definitions in §5. 2. **Deep embedding of ML-S/EMLS** — inductive syntax + semantic evaluation in §6. 3. **Decision procedure skeleton** — a computable `decideMLS` with stated soundness/completeness for **satisfiability**, plus a future **step-counting** function to relate the model-graph algorithm to $\text{Av}(T)$ (§7). 4. **Hardness statements** — structural theorems such as `SatMLS_average_hard` with explicit `sorry` placeholders until reductions from TR1995-711 are formalized (§8).

0.4.2 Design Choices for Executable vs. Proof Layer

Concept	Lean representation	Rationale
Inputs	<code>Bitstring := List Bool, len</code>	Matches $\Sigma = \{0, 1\}$ encodings in the report
Distributions	<code>structure Distribution</code> with explicit finite support, off-support zero, <code>support.sum prob ≤ 1</code>	Avoids infinite sums; rank and testing are well-defined (see DEFINITION_FORKS.md)
Rank	<code>noncomputable def rank</code>	Cardinality over all strings is not computable
Set semantics	Axiomatic <code>MLS.ZFSet</code> + <code>noncomputable evalTerm</code>	Supports nested sets without committing to full ZF in Mathlib; <code>Mathlib.Data.ZFC.Basic</code> is an alternative for a future refactor
EMLS	<code>Literal, literalToFormula, conjunctToFormula, Literal.holds</code> (ğ6)	FOS80 ğ3 normal form for ğ7 decision procedure
Tests	<code>#eval</code> + <code>native_decide</code> on decidable fragments	Same regression pattern as <code>Icon2lean/Tests.lean</code>

0.5 5. Average-Case Complexity (AvCom): Theory, Classes, and Lean Encoding

The formal definitions in this section follow TR1995-711 ğ3.2 ([TR1995-711.pdf](#)). In the mid-1990s, structural average-case complexity was a young, highly mathematical field. Each mathematical definition below is paired with its Lean counterpart in [AvgCaseMls/AvCom.lean](#) where it exists today.

0.5.1 Why Naive Averaging Fails

Prior to Leonid Levin’s 1986 breakthrough [Lev86], researchers measured average running time naively:

$$\text{Time}_M^\mu(n) = \sum_{|x|=n} \mu_n(x) \text{time}_M(x)$$

As noted by Ben-David et al. [BDCGL89] and Gurevich, this formulation is deeply flawed: * **Model-dependent and encoding-dependent:** Slight changes in the binary representation of inputs radically alter the average complexity. * **Not closed under functional composition:** An algorithm that runs in average $O(n)$ time can yield an average-case exponential runtime when composed with a polynomial-time pre-processing step.

Simply taking the expected running time of an algorithm weighted over all inputs of size n is therefore inadequate as a robust complexity measure.

0.5.2 Levin’s Robust Formulation

Levin solved these issues with a robust notion of “polynomial time on average.” Under this framework, a running time $T(x)$ is average-polynomial under a distribution μ if there is a

constant $c > 0$ such that the expected value of $T(x)^{1/c}/|x|$ is finite:

$$\sum_x \mu(x) \frac{T(x)^{1/c}}{|x|} < \infty$$

Rather than analyzing a language in isolation, average-case complexity pairs a language L (a decision problem) with a probability distribution μ on its instances, denoted as the distributional problem (L, μ) .

0.5.3 Reischuk-Schindelhauer’s Precise Classes

In 1993, Reischuk and Schindelhauer [RS93] streamlined Levin’s theory by introducing **ranking functions** to capture the distribution profile. Cox, Ericson, and Mishra rely primarily on the average-case analogues of P and NP under both Levin’s traditional definitions and this precise average-case framework. TR1995-711 §3.2 notes explicitly that terminology in this area had **not yet been standardized** in the literature even in 1995; the report follows [RS93] and cites [Lev86, BDCGL89, Gur91, VR92, SY92].

The subsections below collect the definitions as used in the report, with Lean encodings interleaved.

0.5.4 Inputs, Distributions, and Rank

Fix a finite alphabet Σ (in practice $\Sigma = \{0, 1\}$). An **input** is a string $x \in \Sigma^*$, with **length** $|x|$ (in Lean we use `Bitstring := List Bool` and `len s := s.length`).

A **probability distribution** on Σ^* is a function $\mu : \Sigma^* \rightarrow [0, 1]$ such that $\sum_x \mu(x) \leq 1$ and $\mu(x) \geq 0$ for all x . In the Lean sketch we axiomatize this with finite `Finset` sums rather than infinite series, which is adequate for the rank-based definitions that follow.

A **distributional problem** is a pair (L, μ) where $L \subseteq \Sigma^*$ is a decision problem (language) and μ is a distribution on its instances.

The **rank** of x under μ counts how many inputs are at least as probable as x :

$$\text{rank}_\mu(x) = |\{z \in \Sigma^* : \mu(z) \geq \mu(x)\}|$$

When $\mu(x) = 0$, the rank is taken to be 0 (inputs of measure zero carry no average-case weight). In Lean, **rank** is **noncomputable** (real comparisons are classical) and counts only over the finite `support`; see `DEFINITION_FORKS.md`.

0.5.5 Complexity Bounds, POL, and Rankable Distributions

Following [RS93], let **POL** denote the class of **polynomial complexity bounds**—functions $T : \mathbb{N} \rightarrow \mathbb{N}$ such that $T(n) \leq cn^k + c$ for some constants c, k (formalized in Lean as `IsPolynomial`).

A distribution μ is **T -rankable** if $\text{rank}_\mu(x) \leq T(|x|)$ for all x .

A distribution μ is **POL-rankable** if it is T -rankable for some $T \in \text{POL}$ **and** the rank function $\text{rank}_\mu(x)$ is computable in deterministic polynomial time (in binary). TR1995-711 uses POL-rankable distributions throughout its hardness constructions.

A real-valued function $m : [0, 1] \rightarrow [0, 1]$ is **monotone** if $x < y$ implies $m(x) < m(y)$. A **monotone transformation** of a distribution μ is a reweighting $m \circ \mu$ obtained from such an m with $\sum_x m(\mu(x)) < 1$. Levin’s original average-time definition quantifies over all monotone transformations of μ ; [RS93] shows this is equivalent to a simpler rank-sum condition (below).

0.5.6 Levin’s μ -Average Time and the RS93 Alternative

Let $f : \Sigma^* \rightarrow \mathbb{N}$ be a running-time function and $T : \mathbb{N} \rightarrow \mathbb{N}$ a monotone complexity bound with **generalized inverse** $T^{-1}(m) = \min\{n : T(n) \geq m\}$.

Levin’s formulation (conceptual): the pair (f, μ) lies in $\text{Av}(T)$ if, for every monotone transformation m of μ , a certain T^{-1} -weighted expectation remains bounded. This formulation is robust but references all monotone reweightings of μ and is awkward to formalize directly.

Reischuk–Schindelhauer alternative (used in TR1995-711): $(f, \mu) \in \text{Av}(T)$ if for all integers $\ell \geq 1$,

$$\sum_{\text{rank}_\mu(x) \leq \ell} \frac{T^{-1}(f(x))}{|x|} \leq \ell$$

This is the definition implemented structurally in [AvgCaseMls/AvCom.lean](#) as `IsAvTime`. Intuitively, high-rank (low-probability) inputs may take large time $f(x)$, but the inverse-bound mass $T^{-1}(f(x))$ per bit of input cannot accumulate faster than the rank budget ℓ .

0.5.7 Average Complexity Classes

Let M be a deterministic Turing machine with running time $f_M(x)$ on input x .

- **DistTime(T):** the class of distributional problems (L, μ) for which there exists a deterministic algorithm M deciding L such that $(f_M, \mu) \in \text{Av}(T)$.
- **AvDTime(T, C):** as above, but restricting μ to be C -**rankable** distributions (for a complexity class C of rank bounds). This class drives the “**nose**” diagram: languages tractable on average when the ranking function of the distribution is itself bounded by $V \in \text{POL}$.
- **AvP (Average Polynomial Time):** $\text{DistTime}(\text{POL}, \text{POL-rankable})$ —distributional problems efficiently solvable on average over POL-rankable μ . Equivalently: $(L, \mu) \in \text{AvP}$ if L is decidable in average polynomial time under a POL-rankable distribution.
- **distNP (also written NP^{dist} in the report):** $\{(L, \mu) : L \in \text{NP}, \mu \in \text{POL-rankable}\}$. Membership in NP means witnesses are verifiable in polynomial time on a nondeterministic Turing machine (NTM).

Under this framework, Cox, Ericson, and Mishra utilize several precise complexity classes:

- **POL-rankable Distributions:** As defined above—polynomial rank bound plus polynomial-time rank computation.
- **Av(T) (Average Time T):** Pairs (f, μ) satisfying the RS93 rank-sum inequality; for a machine deciding L , require $(f_M, \mu) \in \text{Av}(T)$.
- **AvP (Average Polynomial Time):** $\text{DistTime}(\text{POL}, \text{POL-rankable})$ —distributional problems (L, μ) efficiently solvable on average over POL-rankable distributions.
- **distNP (Distributional NP):** $\{(L, \mu) : L \in \text{NP}, \mu \in \text{POL-rankable}\}$ (the report also discusses P-computable and P-samplable distributions in the broader literature).

0.5.8 Distributional Reductions and NP-Average Completeness

To transfer hardness results between average-case problems, TR1995-711 §3.2 defines **distributional reductions**. A reduction from (L_1, μ_1) to (L_2, μ_2) is a polynomial-time computable function $f : \Sigma^* \rightarrow \Sigma^*$ such that:

1. **Correctness:** $x \in L_1 \iff f(x) \in L_2$ for all x .
2. **Domination:** letting $p_i(x) = \text{rank}_{\mu_i}(x)$, there exist constants $c_0, c_1 > 0$ such that

$$p_2(f(x)) \leq c_0 |x|^{c_1} p_1(x)$$

for all x .

The domination condition ensures that if (L_2, μ_2) is solvable in average polynomial time, tractability is preserved for (L_1, μ_1) : f cannot map many low-rank inputs of μ_1 into disproportionately high-rank images under μ_2 .

A distributional problem (L, μ) is **NP-average complete** (NP-distributional complete) if: $(L, \mu) \in \text{distNP}$, and $\ast$ every $(L', \mu') \in \text{distNP}$ is distributionally reducible to (L, μ) .

TR1995-711 Corollary 5.1 (page 12) states that **MLS satisfiability** is NP-average complete; related corollaries cover EMLS, FP/LP, and further set-theoretic fragments. The proofs combine distributional reductions from bounded halting for NTMs with the rankable distributions constructed in the report.

0.5.9 Lean encoding (Phases 1A–1D)

Here we translate TR1995-711 §3.2 into Lean 4 using the RS93 rank-sum definition of $\text{Av}(T)$. The module [AvgCaseMls/AvCom.lean](#) currently defines:

- `Bitstring`, `len`, `lenBot` — inputs $x \in \{0, 1\}^*$, length $|x|$, and $\max 1 \mid x|$ for RS93 denominators (see [DEFINITION_FORKS.md](#));
- `Distribution` — finite `support`, non-negative `prob`, mass zero off support, `support.sum prob ≤ 1`; constructors `pointMass`, `uniformOn`;
- `DistributionalProblem`, `IsPolynomial` (+ basic lemmas) — Phase **1A** complete in [AvgCaseMls/AvCom.lean](#);
- `rank` — $\text{rank}_\mu(x)$ as a support filter cardinality; rank 0 when $\mu.\text{prob } x = 0$ (Phase **1B**);
- `T_inv` — partial search for $\min\{n \mid T(n) \geq m\}$ from `n = 0` (Phase **1B**);
- `IsAvTime`, `IsAv`, `rankLe` — RS93 rank-sum average time (Phase **1C**);
- `IsTRankable`, `IsPolRankable`, `DistTime`, `AvDTime` — dist-time classes (Phase **1C**);
- `AvP`, `InDistNP`, `DistributionalReduction`, `IsNPAAverageComplete` — average classes and reductions (Phase **1D**).

All Phase **1** AvCom scaffolding is in [AvgCaseMls/AvCom.lean](#). Later phases connect MLS (§6–§8) and hardness (§8).

Mapping the Nose diagram to Lean types Section 2’s **Nose** diagram (Figure 1) plots a language L by two polynomial bounds:

1. **Worst-case complexity (V):** in Lean, a function $V : \text{Nat} \rightarrow \text{Nat}$ with `IsPolynomial V`, used in `IsTRankable V  $\mu$` — the worst-case cost of computing $\text{rank}_\mu(x)$ on inputs of length $|x|$.
2. **Average-case complexity (T):** in Lean, a function $T : \text{Nat} \rightarrow \text{Nat}$ with `IsPolynomial T`, used in `DistTime T  $\langle L, \mu \rangle$` — average running time under the RS93 rank-sum bound for distribution μ .

The tractable **nose** boundary $\text{nose}(L) = \{(T, V) \in (\text{POL}, \text{POL}) : L \in \text{AvDTime}(T, V\text{-rankable})\}$ is therefore realized directly as pairs of polynomial bounds satisfying `AvDTime T V  $\langle L, \mu \rangle$` , i.e. `IsTRankable V  $\mu$   $\wedge$  DistTime T  $\langle L, \mu \rangle$` . NP-average complete targets such as MLS satisfiability have empty or trivial noses under simple POL-rankable distributions unless $\text{NEXP} = \text{EXP}$ — the conditional hardness corollaries in §8.

0.5.10 AvgCaseMls/AvCom.lean

AvgCaseMls/AvCom.lean

Lean 4 Certificate

```

/-
Copyright (c) 2026 Catskills Research Company. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Lars Warren Ericson, Catskills Research Company
-/

import Mathlib.Data.Real.Basic
import Mathlib.Algebra.BigOperators.Group.Finset.Basic
import Mathlib.Tactic.FieldSimp
import Mathlib.Tactic.Ring

/-!
Average-case complexity definitions (ReischukSchindelbauer framework).

Extracted from ['arxiv.md'](..../arxiv.md) §5.

**Phase 1A:** 'Bitstring', 'len', 'Distribution', 'DistributionalProblem', '
  IsPolynomial'.

**Phase 1B:** 'rank', 'T_inv'.

**Phase 1C:** 'IsAvTime', 'DistTime', 'AvDTime', rankability predicates.

**Phase 1D:** 'AvP', 'InDistNP', 'DistributionalReduction', 'IsNPAverageComplete
  '.

**Phase 2+:** MLS hardness proofs open in later modules.
-/

open Finset

namespace AvCom

```

```

/-! ## Phase 1A  inputs, distributions, POL -/

/-- Binary inputs  $x \in \{0,1\}^*$  (TR1995-711 §3.2). -/
abbrev Bitstring := List Bool

/-- Length  $|x|$  as 'List.length'. -/
def len (s : Bitstring) : Nat := s.length

@[simp] theorem len_eq (s : Bitstring) : len s = s.length := rfl

/--
Length used in RS93 denominators: 'max 1 |x|' so the empty string is guarded.
See 'DEFINITION_FORKS.md'.
-/
def lenBot (s : Bitstring) : Nat := max 1 s.length

theorem lenBot_empty : lenBot ([]) : Bitstring) = 1 := rfl

theorem lenBot_ne_zero (s : Bitstring) : 0 < lenBot s := by
  have : 1 ≤ lenBot s := le_max_left 1 s.length
  omega

/--
A finite-support probability distribution on bitstrings.

Literature:  $\mu : \Sigma^* \rightarrow [0,1]$  with  $\sum_x \mu(x) \leq 1$ .
We restrict to an explicit finite 'support' so rank and testing are well-defined
(Phase 1B).
-/
structure Distribution where
  support : Finset Bitstring
  prob : Bitstring → Real
  prob_nonneg : ∀ s, 0 ≤ prob s
  prob_zero_outside : ∀ s, s ∉ support → prob s = 0
  prob_sum_le_one : support.sum prob ≤ 1

namespace Distribution

/-- Total mass on the declared support. -/
noncomputable def mass ( $\mu$  : Distribution) : Real :=
   $\mu$ .support.sum  $\mu$ .prob

theorem mass_le_one ( $\mu$  : Distribution) :  $\mu$ .mass ≤ 1 :=
   $\mu$ .prob_sum_le_one

end Distribution

noncomputable def pointMassProb (x : Bitstring) (p : Real) (s : Bitstring) :
  Real :=
  if s = x then p else 0

noncomputable def uniformProb (S : Finset Bitstring) (s : Bitstring) : Real :=
  if s ∈ S then 1 / (S.card : Real) else 0

/-- Point mass  $p$  on a single string (requires  $0 \leq p \leq 1$ ). -/
noncomputable def pointMass (x : Bitstring) (p : Real)

```

```

    (hp0 : 0 ≤ p) (hp1 : p ≤ 1) : Distribution where
support := {x}
prob := pointMassProb x p
prob_nonneg s := by
  unfold pointMassProb
  split_ifs with h
  û exact hp0
  û exact le_rfl
prob_zero_outside s hs := by
  unfold pointMassProb
  by_cases h : s = x
  û exfalso
  exact hs (mem_singleton.mpr h)
  û simp [h]
prob_sum_le_one := by
  rw [sum_singleton]
  simp only [pointMassProb]
  exact hp1

/-- Uniform distribution on a nonempty finite support. -/
noncomputable def uniformOn (S : Finset Bitstring) (h : S.Nonempty) :
  Distribution where
support := S
prob := uniformProb S
prob_nonneg s := by
  unfold uniformProb
  split_ifs with hs
  û exact div_nonneg zero_le_one (Nat.cast_nonneg S.card)
  û exact le_rfl
prob_zero_outside s hs := by
  unfold uniformProb
  by_cases hmem : s ∈ S
  û exfalso
  exact hs hmem
  û simp [hmem]
prob_sum_le_one := by
  have hcard0 : (0 : Real) < S.card :=
    Nat.cast_pos.mpr (card_pos.mpr h)
  have hsum : (S.sum (uniformProb S) : Real) = S.sum fun _ => 1 / (S.card :
    Real) :=
    sum_congr rfl fun s hs => by simp [uniformProb, hs]
  calc
    (S.sum (uniformProb S) : Real) = S.sum fun _ => 1 / (S.card : Real) :=
    hsum
    _ = (S.card : Real) * (1 / (S.card : Real)) := by rw [sum_const,
    nsmul_eq_mul]
    _ = 1 := by
      have hne : (S.card : Real) ≠ 0 := Nat.cast_ne_zero.mpr (card_pos.mpr h)
      .ne'
      field_simp [hne]
      _ ≤ 1 := le_rfl

theorem uniformOn_mass (S : Finset Bitstring) (h : S.Nonempty) : (uniformOn S h)
  .mass = 1 := by
  unfold Distribution.mass uniformOn

```

```

have hsum : (S.sum (uniformProb S) : Real) = S.sum fun _ => 1 / (S.card : Real)
:=
  sum_congr rfl fun s hs => by simp [uniformProb, hs]
calc
  (S.sum (uniformProb S) : Real) = S.sum fun _ => 1 / (S.card : Real) := hsum
  _ = (S.card : Real) * (1 / (S.card : Real)) := by rw [sum_const,
nsmul_eq_mul]
  _ = 1 := by
    have hne : (S.card : Real) ≠ 0 := Nat.cast_ne_zero.mpr (card_pos.mpr h).
ne'
    field_simp [hne]

/--
A distributional problem  $(L, \mu)$ : a language over bitstrings paired with
a distribution.
-/
structure DistributionalProblem where
  L : Set Bitstring
   $\mu$  : Distribution

/--
POL (polynomial complexity bounds):  $T(n) \leq c n^k + c$  for some constants
 $c, k$ .
-/
def IsPolynomial (T : Nat → Nat) : Prop :=
   $\exists c k : \text{Nat}, \forall n, T n \leq c * n^k + c$ 

namespace IsPolynomial

theorem id : IsPolynomial id := ⟨1, 1, fun n => by simp⟩

theorem const (c : Nat) : IsPolynomial (fun _ => c) := ⟨c, 0, fun n => by simp⟩

theorem add_one (T : Nat → Nat) (h : IsPolynomial T) : IsPolynomial fun n => T
  n + 1 := by
  obtain ⟨c, k, hT⟩ := h
  refine ⟨c + 1, k, fun n => ?_⟩
  calc
    T n + 1 ≤ c * n^k + c + 1 := Nat.add_le_add_right (hT n) 1
    _ ≤ (c + 1) * n^k + (c + 1) := by
      rw [Nat.add_mul, Nat.one_mul]
      omega

theorem add_const (T : Nat → Nat) (d : Nat) (h : IsPolynomial T) :
  IsPolynomial fun n => T n + d := by
  induction d with
  | zero => simp using h
  | succ d ih =>
    simp [Nat.add_assoc] using add_one (T := fun n => T n + d) ih

end IsPolynomial

/-! ## Phase 1B rank and inverse bounds -/

/--
Rank of 'x' under ' $\mu$ ': count of support strings at least as probable as 'x'.

```



```

When ' $\mu$ .prob x = 0', rank is '0' (RS93/TR1995-711 §3.2).

Counts only over ' $\mu$ .support'; see ['DEFINITION_FORKS.md'](..../DEFINITION_FORKS.md
).
-/
noncomputable def rank ( $\mu$  : Distribution) (x : Bitstring) : Nat :=
  if  $\mu$ .prob x = 0 then 0
  else
    open Classical in
    ( $\mu$ .support.filter (fun z =>  $\mu$ .prob x  $\leq$   $\mu$ .prob z)).card

namespace rank

theorem zero ( $\mu$  : Distribution) (x : Bitstring) (h :  $\mu$ .prob x = 0) :
  rank  $\mu$  x = 0 := by
  simp [rank, h]

theorem le_support_card ( $\mu$  : Distribution) (x : Bitstring) :
  rank  $\mu$  x  $\leq$   $\mu$ .support.card := by
  unfold rank
  split_ifs with h
   $\omega$  omega
   $\omega$  exact card_filter_le _ _

end rank

/--
Generalized inverse: minimum 'n' with 'T n  $\geq$  m', found by search from '0'.

Partial: diverges if no such 'n' exists (e.g. 'T := fun _ => 0', 'm > 0').
See ['DEFINITION_FORKS.md'](..../DEFINITION_FORKS.md).
-/
partial def T_invAux (T : Nat  $\rightarrow$  Nat) (m n : Nat) : Nat :=
  if T n  $\geq$  m then n else T_invAux T m (n + 1)

def T_inv (T : Nat  $\rightarrow$  Nat) (m : Nat) : Nat :=
  if m = 0 then 0 else T_invAux T m 0

namespace T_inv

theorem zero (T : Nat  $\rightarrow$  Nat) : T_inv T 0 = 0 := rfl

end T_inv

/-! ## Phase 1C average time and dist-time classes -/

/-- Inputs whose rank under ' $\mu$ ' is at most 'l'. -/
noncomputable def rankLe ( $\mu$  : Distribution) (l : Nat) : Finset Bitstring :=
   $\mu$ .support.filter (fun x => rank  $\mu$  x  $\leq$  l)

theorem rankLe_mem { $\mu$  : Distribution} {l : Nat} {x : Bitstring} :
  x  $\in$  rankLe  $\mu$  l  $\iff$  x  $\in$   $\mu$ .support  $\wedge$  rank  $\mu$  x  $\leq$  l := by
  simp [rankLe, mem_filter]

/--
RS93 average-time condition (TR1995-711 §3.2): for all 'l  $\geq$  1',

```

```

'_{rank_μ(x) ≤ l} Tz(f(x)) / lenBot(x) ≤ 1'.
-/
def IsAvTime (T : Nat → Nat) (f : Bitstring → Nat) (μ : Distribution) : Prop
:=
  ∀ l : Nat, l ≥ 1 →
    (rankLe μ l).sum (fun x => (T_inv T (f x) : Real) / (lenBot x : Real)) ≤ (1
      : Real)

/-- Notation matching the literature: '(f, μ) ∈ Av(T)'. -/
abbrev IsAv (T : Nat → Nat) (f : Bitstring → Nat) (μ : Distribution) : Prop :=
  IsAvTime T f μ

namespace IsAvTime

theorem zero (T : Nat → Nat) (μ : Distribution) : IsAvTime T (fun _ => 0) μ :=
  by
    intro l hl
    have hterm : ∀ x ∈ rankLe μ l, (T_inv T (0) : Real) / (lenBot x : Real) = 0 :=
      by
        intro x hx
        rw [T_inv.zero]
        norm_cast
        exact zero_div _
    rw [sum_eq_zero hterm]
    norm_cast
    omega

end IsAvTime

/-- 'μ' is 'V'-rankable: 'rank_μ(x) ≤ V(|x|)' for all 'x'. -/
def IsTRankable (V : Nat → Nat) (μ : Distribution) : Prop :=
  ∀ x, rank μ x ≤ V (len x)

/-- POL-rankable: bounded by some 'V ∈ POL' (polynomial-time rank computation
deferred). -/
def IsPolRankable (μ : Distribution) : Prop :=
  ∃ V : Nat → Nat, IsPolynomial V ∧ IsTRankable V μ

namespace IsTRankable

theorem of_support (V : Nat → Nat) (μ : Distribution)
  (h : ∀ x ∈ μ.support, rank μ x ≤ V (len x)) :
  IsTRankable V μ := by
    intro x
    by_cases hx : x ∈ μ.support
    û exact h x hx
    û have hr : rank μ x = 0 := rank.zero μ x (μ.prob_zero_outside x hx)
      rw [hr]
      exact Nat.zero_le _

end IsTRankable

/--
'DistTime T': some running-time function witnesses 'IsAvTime T f μ'.

```

```

We do not yet tie 'f' to a decider for 'L'; see ['DEFINITION_FORKS.md'](../
DEFINITION_FORKS.md).
-/
def DistTime (T : Nat → Nat) (prob : DistributionalProblem) : Prop :=
  ∃ f : Bitstring → Nat, IsAvTime T f prob.μ

namespace IsPolRankable

theorem uniformOn_polRankable (S : Finset Bitstring) (h : S.Nonempty) :
  IsPolRankable (uniformOn S h) :=
  ⟨fun _ => S.card, IsPolynomial.const S.card,
   IsTRankable.of_support _ _ fun x _ =>
    rank.le_support_card (uniformOn S h) x⟩

end IsPolRankable

namespace DistTime

theorem of_avTime {T : Nat → Nat} {prob : DistributionalProblem} {f : Bitstring
  → Nat}
  (h : IsAvTime T f prob.μ) : DistTime T prob :=
  ⟨f, h⟩

theorem zero (T : Nat → Nat) (prob : DistributionalProblem) : DistTime T prob
  :=
  of_avTime (IsAvTime.zero T prob.μ)

/-- Average-time witnesses depend only on the distribution, not the language
label. -/
theorem same_μ {L L' : Set Bitstring} {μ : Distribution} {T : Nat → Nat} :
  DistTime T ⟨L, μ⟩ DistTime T ⟨L', μ⟩ :=
  Iff.rfl

end DistTime

/--
'AvDTime T V': 'DistTime T' on problems whose distribution is 'V'-rankable.
Matches the report's 'AvDTime(T, C)' with 'C' instantiated as a rank bound.
-/
def AvDTime (T V : Nat → Nat) (prob : DistributionalProblem) : Prop :=
  IsTRankable V prob.μ ∧ DistTime T prob

namespace AvDTime

theorem of_distTime {T V : Nat → Nat} {prob : DistributionalProblem}
  (hV : IsTRankable V prob.μ) (hT : DistTime T prob) : AvDTime T V prob :=
  ⟨hV, hT⟩

end AvDTime

/-! ## Phase 1D AvP, distNP, reductions, completeness -/

/--
Certificate-based NP membership (Phase **3A** fork): poly-sized witnesses plus a
'Bool' verifier. See ['DEFINITION_FORKS.md'](../DEFINITION_FORKS.md).
-/

```

```

def InNP (L : Set Bitstring) : Prop :=
  ∃ (verify : Bitstring → Bitstring → Bool) (bound : Nat → Nat),
    IsPolynomial bound ∧
    (∀ x, x ∈ L ∃ cert, len cert ≤ bound (len x) ∧ verify x cert = true)

namespace InNP

theorem intro {L : Set Bitstring} {verify : Bitstring → Bitstring → Bool} {
  bound : Nat → Nat}
  (hbound : IsPolynomial bound)
  (h : ∀ x, x ∈ L ∃ cert, len cert ≤ bound (len x) ∧ verify x cert = true) :
  InNP L :=
  ⟨verify, bound, hbound, h⟩

theorem empty : InNP (∅ : Set Bitstring) :=
  intro (verify := fun _ _ => false) (bound := fun _ => 0) (IsPolynomial.const
    0) fun x => by
    simp

end InNP

/-- 'distNP': NP language plus POL-rankable distribution (TR1995-711 §3.2). -/
def InDistNP (prob : DistributionalProblem) : Prop :=
  InNP prob.L ∧ IsPolRankable prob.μ

namespace InDistNP

theorem intro {prob : DistributionalProblem} (hNP : InNP prob.L) (hμ :
  IsPolRankable prob.μ) :
  InDistNP prob :=
  ⟨hNP, hμ⟩

theorem uniformOn (L : Set Bitstring) (S : Finset Bitstring) (h : S.Nonempty)
  (hNP : InNP L) :
  InDistNP ⟨L, uniformOn S h⟩ :=
  intro hNP (IsPolRankable.uniformOn_polRankable S h)

end InDistNP

/--
Distributional reduction (TR1995-711 §3.2): map 'f' with correctness 'x ∈ L ⇒ f(x)
  ∈ L',
polynomial length growth 'lenBot (f x) ≤ k + lenBot(x)^(k)', and domination
'rank_{μ}(f(x)) ≤ c + lenBot(x)^c + rank_{μ}(x)'.
-/
def DistributionalReduction (source target : DistributionalProblem) : Prop :=
  ∃ f : Bitstring → Bitstring,
    (∀ x, x ∈ source.L ⇒ f x ∈ target.L) ∧
    (∃ k0 k1 : Nat, ∀ x, lenBot (f x) ≤ k0 + (lenBot x) ^ k1) ∧
    (∃ c0 c1 : Nat, 0 < c0 ∧ 0 < c1 ∧
      ∀ x, rank target.μ (f x) ≤ c0 + (lenBot x) ^ c1 + rank source.μ x)

namespace DistributionalReduction

theorem refl (p : DistributionalProblem) : DistributionalReduction p p := by

```

```

refine ⟨id, ?_, ⟨1, 1, fun x => ?_⟩, ⟨1, 1, one_pos, one_pos, fun x => ?_⟩⟩
û intro x; simp
û simp [lenBot, id_eq]
û simp only [id_eq, pow_one, one_mul]
  rcases Nat.eq_zero_or_pos (rank p.μ x) with hr | hr
  û omega
  û exact Nat.le_mul_of_pos_left (rank p.μ x) (lenBot_ne_zero x)

private theorem compose_lenBound {f g : Bitstring → Bitstring} {k0 k1 k0' k1' :
  Nat}
  (hf : ∀ x, lenBot (f x) ≤ k0 * (lenBot x) ^ k1)
  (hg : ∀ x, lenBot (g x) ≤ k0' * (lenBot x) ^ k1') :
  ∀ x, lenBot (g (f x)) ≤ k0' * k0 ^ k1' * (lenBot x) ^ (k1 * k1') := by
  intro x
  have hpow : (lenBot (f x)) ^ k1' ≤ (k0 * (lenBot x) ^ k1) ^ k1' := by
    cases k1' with
    | zero => simp
    | succ k => exact Nat.pow_le_pow_left (hf x) (k + 1)
  calc
    lenBot (g (f x)) ≤ k0' * (lenBot (f x)) ^ k1' := hg (f x)
    _ ≤ k0' * (k0 * (lenBot x) ^ k1) ^ k1' := Nat.mul_le_mul_left _ hpow
    _ = k0' * k0 ^ k1' * (lenBot x ^ k1) ^ k1' := by rw [Nat.mul_pow, ← Nat.
mul_assoc]
    _ = k0' * k0 ^ k1' * lenBot x ^ (k1 * k1') := by rw [Nat.pow_mul]

private theorem compose_rankBound {p1 p2 p3 : DistributionalProblem} {f g :
  Bitstring → Bitstring}
  {k0 k1 c0 c1 d0 d1 : Nat}
  (hf : ∀ x, lenBot (f x) ≤ k0 * (lenBot x) ^ k1)
  (hdom12 : ∀ x, rank p2.μ (f x) ≤ c0 * (lenBot x) ^ c1 * rank p1.μ x)
  (hdom23 : ∀ x, rank p3.μ (g x) ≤ d0 * (lenBot x) ^ d1 * rank p2.μ x) :
  ∀ x,
    rank p3.μ (g (f x)) ≤
      d0 * c0 * k0 ^ d1 * (lenBot x) ^ (k1 * d1 + c1) * rank p1.μ x := by
  intro x
  have hpow : (lenBot (f x)) ^ d1 ≤ (k0 * (lenBot x) ^ k1) ^ d1 := by
    cases d1 with
    | zero => simp
    | succ d => exact Nat.pow_le_pow_left (hf x) (d + 1)
  calc
    rank p3.μ (g (f x)) ≤ d0 * (lenBot (f x)) ^ d1 * rank p2.μ (f x) := hdom23
    (f x)
    _ ≤ d0 * (k0 * (lenBot x) ^ k1) ^ d1 * rank p2.μ (f x) :=
      Nat.mul_le_mul_right _ (Nat.mul_le_mul_left d0 hpow)
    _ ≤ d0 * (k0 * (lenBot x) ^ k1) ^ d1 * (c0 * (lenBot x) ^ c1 * rank p1.μ x)
    :=
      Nat.mul_le_mul_left _ (hdom12 x)
    _ = d0 * c0 * k0 ^ d1 * (lenBot x) ^ (k1 * d1 + c1) * rank p1.μ x := by
      ring_nf

/--
Compose distributional reductions (TR1995-711 §3.2 transitivity).
-/
theorem trans {p1 p2 p3 : DistributionalProblem}
  (h12 : DistributionalReduction p1 p2) (h23 : DistributionalReduction p2 p3)
  :

```

```

    DistributionalReduction p1 p3 := by
    obtain ⟨f, hf, hlenSpec, hdomSpec⟩ := h12
    obtain ⟨k0, k1, hlen12⟩ := hlenSpec
    obtain ⟨c0, c1, hc0, hc1, hdom12⟩ := hdomSpec
    obtain ⟨g, hg, hlenSpec', hdomSpec'⟩ := h23
    obtain ⟨k0', k1', hlen23⟩ := hlenSpec'
    obtain ⟨d0, d1, hd0, hd1, hdom23⟩ := hdomSpec'
    refine
      ⟨fun x => g (f x), ?_, ⟨k0' * k0 ^ k1', k1 * k1', ?_⟩,
        ⟨d0 * c0 * k0 ^ d1, k1 * d1 + c1, ?_, ?_, ?_⟩⟩
    û intro x; exact (hf x).trans (hg (f x))
    û intro x; exact compose_lenBound hlen12 hlen23 x
    û have hk0 : 0 < k0 := by
      by_contra hle
      simp only [not_lt] at hle
      have := hlen12 ([] : Bitstring)
      simp [lenBot] at this
      omega
    exact Nat.mul_pos (Nat.mul_pos hd0 hc0) (Nat.pow_pos hk0)
    û exact Nat.lt_of_lt_of_le hc1 (Nat.le_add_left _ _)
    û intro x; exact compose_rankBound hlen12 hdom12 hdom23 x

end DistributionalReduction

/--
Average polynomial time: POL-rankable distribution plus 'DistTime T' for some 'T
∈ POL'.
Matches 'DistTime(POL, POL-rankable)' in TR1995-711 §3.2.
-/
def AvP (prob : DistributionalProblem) : Prop :=
  IsPolRankable prob.μ ∧ ∃ T : Nat → Nat, IsPolynomial T ∧ DistTime T prob

namespace AvP

theorem of_distTime {prob : DistributionalProblem} (hμ : IsPolRankable prob.μ)
  {T : Nat → Nat} (hT : IsPolynomial T) (h : DistTime T prob) : AvP prob :=
  ⟨hμ, T, hT, h⟩

theorem zero {prob : DistributionalProblem} (hμ : IsPolRankable prob.μ) {T : Nat
  → Nat}
  (hT : IsPolynomial T) : AvP prob :=
  of_distTime hμ hT (DistTime.zero T prob)

/-- ['AvP'] depends on the distribution and time bounds, not the language label
(see ['DistTime.same_μ']). -/
theorem same_μ {L L' : Set Bitstring} {μ : Distribution} :
  AvP ⟨L, μ⟩ AvP ⟨L', μ⟩ := by
  constructor <=> intro ⟨hμ, T, hT, hDT⟩ <=> exact ⟨hμ, T, hT, by simp [
    DistTime] using hDT⟩

end AvP

/-- NP-average (distNP) completeness: in 'distNP' and hard for all of 'distNP'.
-/
def IsNPAverageComplete (target : DistributionalProblem) : Prop :=

```

```

InDistNP target ∧ ∀ source, InDistNP source → DistributionalReduction source
target

namespace IsNPAverageComplete

theorem intro {target : DistributionalProblem} (h : InDistNP target)
  (hred : ∀ source, InDistNP source → DistributionalReduction source target)
  :
  IsNPAverageComplete target :=
  ⟨h, hred⟩

/--
If 'mid' is NP-average complete and 'mid' reduces to 'target', then 'target' is
complete.
Corollary 5.1 pipeline: distNP-complete NBH core → MLS target.
-/
theorem of_reductor {mid target : DistributionalProblem}
  (hTarget : InDistNP target) (hMid : IsNPAverageComplete mid)
  (hRed : DistributionalReduction mid target) :
  IsNPAverageComplete target :=
  intro hTarget fun source hsource =>
  DistributionalReduction.trans (hMid.2 source hsource) hRed

end IsNPAverageComplete

end AvCom

```

0.6 6. Multilevel Syllogistic (MLS): Grammar and Lean Encoding

0.6.1 Syntax of MLS and EMLS

Multilevel Syllogistic (MLS) is a decidable fragment of Zermelo-Fraenkel set theory. Its syntax allows set variables, the empty set ($\emptyset$), binary set operators (union $\cup$, intersection $\cap$, set difference $\setminus$), binary set relations (membership $\in$, non-membership $\notin$, equality $=$, inequality $\neq$), and standard propositional connectives.

The grammar is formally defined as: * **Terms:** $T \rightarrow v_i \mid \emptyset \mid T \cup T \mid T \cap T \mid T \setminus T$ * **Literals:** $L \rightarrow T \in T \mid T \notin T \mid T = T \mid T \neq T$ * **Formulas:** $\Phi \rightarrow L \mid \neg \Phi \mid \Phi \wedge \Phi \mid \Phi \vee \Phi \mid \Phi \Rightarrow \Phi \mid \Phi \equiv \Phi$

Elementary Multilevel Syllogistic (EMLS) simplifies the terms by restricting conjuncts to “flat” elementary literals:

$$v_i = \emptyset, \quad v_i = v_j \cup v_k, \quad v_i = v_j \setminus v_k, \quad v_i = v_j \cap v_k, \quad v_i \in v_j, \quad v_i \notin v_j, \quad v_i \neq v_j$$

0.6.2 Lean encoding (Phase 2A)

Scope. [AvgCaseMls/MLS.lean](#) (Phase **2A**) and [AvgCaseMls/EMLS.lean](#) (Phase **2B**) compile with no `sorry`. Phase **2C** (decision procedure, §7) and **2D** (serialization and step counting, §8) are separate obligations.

Set variables are identified with natural-number indices ($\text{Nat} \rightarrow \text{ZFSet}$ environments), matching the report’s v_i notation. MLS formulas talk about membership chains $v_i \in v_j \in v_k \in \dots$. We use a custom axiomatized `ZFSet` sort so the development is self-contained and `evalTerm/evalFormula` are explicitly `noncomputable` (axioms are not compiled). A Mathlib-backed refactor would replace `axiom ZFSet` with imports from `Mathlib.Data.ZFC.Basic`.

The listing below matches [AvgCaseMls/MLS.lean](#).

0.6.3 AvgCaseMls/MLS.lean

AvgCaseMls/MLS.lean

Lean 4 Certificate

```
/-
Copyright (c) 2026 Catskills Research Company. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Lars Warren Ericson, Catskills Research Company
-/

/-!
Deep embedding of Multilevel Syllogistic (MLS) syntax and set-theoretic
semantics.

Extracted from ['arxiv.md'](../arxiv.md) §6.
-/

namespace MLS

/-! ### Syntactic terms -/

inductive Term : Type
| var      : Nat → Term
| empty    : Term
| union    : Term → Term → Term
| inter    : Term → Term → Term
| diff     : Term → Term → Term
deriving DecidableEq, Repr

/-! ### Set-theoretic relations -/

inductive Relation : Type
| mem      : Term → Term → Relation
| not_mem  : Term → Term → Relation
| eq       : Term → Term → Relation
| neq      : Term → Term → Relation
deriving DecidableEq, Repr

/-! ### Propositional formulas -/

inductive Formula : Type
| rel      : Relation → Formula
| not      : Formula → Formula
| and      : Formula → Formula → Formula
| or       : Formula → Formula → Formula
| imp      : Formula → Formula → Formula
| iff      : Formula → Formula → Formula
deriving DecidableEq, Repr

/-! ### Axiomatic semantics -/

axiom ZFSet : Type
```



```

axiom ZFSet.empty : ZFSet
axiom ZFSet.union : ZFSet → ZFSet → ZFSet
axiom ZFSet.inter : ZFSet → ZFSet → ZFSet
axiom ZFSet.diff  : ZFSet → ZFSet → ZFSet
axiom ZFSet.mem   : ZFSet → ZFSet → Prop

/-- Standard ZF Axiom of Foundation (Regularity): no set is a member of itself.
-/
axiom ZFSet.regularity : ∀ x, ¬ ZFSet.mem x x

/-- Distinct nonempty tags for Step 3 witness environments (Phase 2C). -/
axiom ZFSet.tag : Nat → ZFSet

axiom ZFSet.tag_ne_empty (n : Nat) : ZFSet.tag n ≠ ZFSet.empty

axiom ZFSet.tag_injective : Function.Injective ZFSet.tag

def Env : Type := Nat → ZFSet

noncomputable def evalTerm (env : Env) : Term → ZFSet
| Term.var n      => env n
| Term.empty      => ZFSet.empty
| Term.union t1 t2 => ZFSet.union (evalTerm env t1) (evalTerm env t2)
| Term.inter t1 t2 => ZFSet.inter (evalTerm env t1) (evalTerm env t2)
| Term.diff t1 t2  => ZFSet.diff (evalTerm env t1) (evalTerm env t2)

noncomputable def evalFormula (env : Env) : Formula → Prop
| Formula.rel (Relation.mem t1 t2)      => ZFSet.mem (evalTerm env t1) (
  evalTerm env t2)
| Formula.rel (Relation.not_mem t1 t2) => ¬ ZFSet.mem (evalTerm env t1) (
  evalTerm env t2)
| Formula.rel (Relation.eq t1 t2)       => evalTerm env t1 = evalTerm env t2
| Formula.rel (Relation.neq t1 t2)      => evalTerm env t1 ≠ evalTerm env t2
| Formula.not f                          => ¬ evalFormula env f
| Formula.and f1 f2                     => evalFormula env f1 ∧ evalFormula env
  f2
| Formula.or f1 f2                      => evalFormula env f1 ∨ evalFormula env
  f2
| Formula.imp f1 f2                     => evalFormula env f1 → evalFormula
  env f2
| Formula.iff f1 f2                     => evalFormula env f1 ↔ evalFormula env
  f2

end MLS

```

EMLS (Phase 2B). Elementary literals and translation into MLS live in [AvgCaseMls/EMLS.lean](#), following Ferro–Omodeo–Schwartz [FOS80] §3 ([3-540-10009-1_8.pdf](#)):

0.6.4 AvgCaseMls/EMLS.lean

AvgCaseMls/EMLS.lean

Lean 4 Certificate

```

/-
Copyright (c) 2026 Catskills Research Company. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Lars Warren Ericson, Catskills Research Company
-/

import AvgCaseMls.MLS

/-!
**Phase 2B complete:** 'Literal', 'literalToFormula', 'conjunctToFormula', '
  Literal.holds', translation lemmas; 'relationToLiteral?' for §7
  normalization.

Literals follow FerroOmodeoSchwartz [FOS80] §3 ('3-540-10009-1_8.pdf' in this
  repo):
conjunctions of (* )  $x = y \diamond z$ , ( $x \in y$ ), ( $x \notin y$ ), ( $x \neq y$ ),
  ( $x \neq y$ ),
with  $\diamond \in \{\cup, \cap, \setminus\}$  and  $x, y, z$  set variables.
-/

namespace MLS.EMLS

/-! ### Elementary literals (FOS80 §3) -/

inductive BinOp
| union
| inter
| diff
deriving DecidableEq, Repr

inductive Literal
| eqOp : Nat → Nat → Nat → BinOp → Literal -- (* )  $x = y \ z$ 
| eqEmpty : Nat → Literal --  $x = \emptyset$ 
| mem : Nat → Nat → Literal -- ( $\in$ )  $x \in y$ 
| notMem : Nat → Nat → Literal -- ( $\notin$ )  $x \notin y$ 
| neq : Nat → Nat → Literal -- ( $\neq$ )  $x \neq y$ 
deriving DecidableEq, Repr

abbrev Conjunct := List Literal

def binOpToTerm (op : BinOp) (y z : Nat) : Term :=
  match op with
  | .union => Term.union (Term.var y) (Term.var z)
  | .inter => Term.inter (Term.var y) (Term.var z)
  | .diff => Term.diff (Term.var y) (Term.var z)

def literalToFormula : Literal → Formula
| .eqOp x y z op => Formula.rel (Relation.eq (Term.var x) (binOpToTerm op y z))
| .eqEmpty x => Formula.rel (Relation.eq (Term.var x) Term.empty)
| .mem x y => Formula.rel (Relation.mem (Term.var x) (Term.var y))
| .notMem x y => Formula.rel (Relation.not_mem (Term.var x) (Term.var y))
| .neq x y => Formula.rel (Relation.neq (Term.var x) (Term.var y))

def conjunctToFormula : Conjunct → Option Formula

```

```

| [] => none
| [l] => some (literalToFormula l)
| l :: ls@(_ :: _) =>
  match conjunctToFormula ls with
  | none => none
  | some f => some (Formula.and (literalToFormula l) f)

/-! ### Semantics and translation lemmas (Phase 2B) -/

/-- Semantic satisfaction of an elementary literal under 'env'. -/
noncomputable def Literal.holds (env : Env) : Literal → Prop
| .eqOp x y z op =>
  evalTerm env (Term.var x) = evalTerm env (binOpToTerm op y z)
| .eqEmpty x =>
  evalTerm env (Term.var x) = evalTerm env Term.empty
| .mem x y =>
  ZFSet.mem (evalTerm env (Term.var x)) (evalTerm env (Term.var y))
| .notMem x y =>
  ¬ ZFSet.mem (evalTerm env (Term.var x)) (evalTerm env (Term.var y))
| .neq x y =>
  evalTerm env (Term.var x) ≠ evalTerm env (Term.var y)

/-! ### Step 4 semantic grounding (ZF regularity) -/

/--
A self-membership literal 'x ∈ x' is unsatisfiable under the Axiom of Foundation
.
Grounds syntactic Step 4 cycle detection for single-node loops.
-/
theorem step4_self_loop_unsat (env : Env) (x : Nat) :
  ¬ Literal.holds env (Literal.mem x x) := by
  intro hmem
  simp only [Literal.holds, evalTerm] at hmem
  exact ZFSet.regularity (env x) hmem

theorem literalToFormula_eval (env : Env) (lit : Literal) :
  evalFormula env (literalToFormula lit) Literal.holds env lit := by
  cases lit <;> simp [Literal.holds, literalToFormula, evalFormula, evalTerm,
    binOpToTerm]

private theorem conjunctToFormula_some_of_ne_nil {c : Conjunct} (h : c ≠ []) :
  ∃ f, conjunctToFormula c = some f := by
  cases c with
  | nil => contradiction
  | cons l ls =>
    cases ls with
    | nil => exact ⟨literalToFormula l, by simp [conjunctToFormula]⟩
    | cons l' ls' =>
      obtain ⟨f, hf⟩ := @conjunctToFormula_some_of_ne_nil (l' :: ls') (List.
        cons_ne_nil l' ls')
      exact ⟨Formula.and (literalToFormula l) f, by simp [conjunctToFormula, hf]⟩
    }

theorem conjunctToFormula_none_iff (c : Conjunct) :
  conjunctToFormula c = none ↔ c = [] := by
  constructor

```

```

    û intro h
      cases c with
      | nil => rfl
      | cons l ls =>
        obtain ⟨f, hf⟩ := @conjunctToFormula_some_of_ne_nil (l :: ls) (List.
        cons_ne_nil l ls)
        rw [hf] at h
        cases h
    û intro h; subst h; simp [conjunctToFormula]

theorem conjunctToFormula_singleton (lit : Literal) :
  conjunctToFormula [lit] = some (literalToFormula lit) := by
  simp [conjunctToFormula]

theorem conjunctToFormula_eval (env : Env) (c : Conjunct) (f : Formula)
  (h : conjunctToFormula c = some f) :
  evalFormula env f ∀ lit ∈ c, Literal.holds env lit := by
  induction c generalizing f with
  | nil => simp [conjunctToFormula] at h
  | cons l ls ih =>
    cases ls with
    | nil =>
      simp [conjunctToFormula] at h
      subst h
      simp [literalToFormula_eval]
    | cons l' ls' =>
      simp [conjunctToFormula] at h
      cases h' : conjunctToFormula (l' :: ls') with
      | none =>
        exfalso
        have := @conjunctToFormula_some_of_ne_nil (l' :: ls') (List.
        cons_ne_nil l' ls')
        simp [h'] at this
      | some f' =>
        rw [h'] at h
        injection h with hf_eq
        subst hf_eq
        have ih' := ih f' h'
        simp only [literalToFormula_eval, List.mem_cons, evalFormula, ih']
        constructor
        û intro ⟨h0, h1⟩ lit hl
          cases hl with
          | inl hl => subst hl; exact h0
          | inr hl => exact h1 lit hl
        û intro hall
          exact ⟨hall l (by simp), fun lit hl => hall lit (Or.inr hl)⟩

/-! ### MLS relation literal (FOS80 §3 patterns) -/

def varTerm? : Term → Option Nat
| Term.var n => some n
| _ => none

theorem varTerm?_eq (t : Term) (n : Nat) (h : varTerm? t = some n) : t = Term.
  var n := by
  cases t <;> simp [varTerm?] at h <;> cases h <;> rfl

```

```

def binaryOpTerm? : Term → Option (Nat × Nat × BinOp)
| Term.union (Term.var y) (Term.var z) => some (y, z, .union)
| Term.inter (Term.var y) (Term.var z) => some (y, z, .inter)
| Term.diff (Term.var y) (Term.var z) => some (y, z, .diff)
| _ => none

def relationToLiteral? : Relation → Option Literal
| Relation.mem t1 t2 => do
  let x ← varTerm? t1
  let y ← varTerm? t2
  return .mem x y
| Relation.not_mem t1 t2 => do
  let x ← varTerm? t1
  let y ← varTerm? t2
  return .notMem x y
| Relation.eq t1 Term.empty => do
  let x ← varTerm? t1
  return .eqEmpty x
| Relation.eq (Term.var x) t2 => do
  let (y, z, op) ← binaryOpTerm? t2
  return .eqOp x y z op
| Relation.neq t1 t2 => do
  let x ← varTerm? t1
  let y ← varTerm? t2
  return .neq x y
| _ => none

theorem relationToLiteral?_eval (env : Env) (r : Relation) (lit : Literal)
  (h : relationToLiteral? r = some lit) :
  evalFormula env (Formula.rel r) Literal.holds env lit := by
  revert lit
  cases r with
  | mem t1 t2 =>
    intro lit h
    cases ht1 : varTerm? t1 with
    | none => simp [relationToLiteral?, ht1] at h
    | some x =>
      cases ht2 : varTerm? t2 with
      | none => simp [relationToLiteral?, ht2] at h
      | some y =>
        simp [relationToLiteral?, ht1, ht2, Literal.holds, evalFormula, evalTerm]
        ] at h
        cases h
        rw [varTerm?_eq t1 x ht1, varTerm?_eq t2 y ht2]
        simp [evalTerm, Literal.holds]
  | not_mem t1 t2 =>
    intro lit h
    cases ht1 : varTerm? t1 with
    | none => simp [relationToLiteral?, ht1] at h
    | some x =>
      cases ht2 : varTerm? t2 with
      | none => simp [relationToLiteral?, ht2] at h
      | some y =>
        simp [relationToLiteral?, ht1, ht2, Literal.holds, evalFormula, evalTerm]
        ] at h

```

```

      cases h
      rw [varTerm?_eq t1 x ht1, varTerm?_eq t2 y ht2]
      simp [evalTerm, Literal.holds]
| eq t1 t2 =>
  intro lit h
  by_cases ht2 : t2 = Term.empty
  û subst ht2
  cases ht1 : varTerm? t1 with
  | none => simp [relationToLiteral?, ht1] at h
  | some x =>
    simp [relationToLiteral?, ht1, Literal.holds, evalFormula, evalTerm] at
h
  cases h
  rw [varTerm?_eq t1 x ht1]
  simp [evalTerm, Literal.holds]
û by_cases hunion :  $\exists x y z, t1 = \text{Term.var } x \wedge t2 = \text{Term.union } (\text{Term.var } y)$ 
  (Term.var z)
  û obtain ⟨x, y, z, ht1, ht2⟩ := hunion
  subst ht1 ht2
  simp [relationToLiteral?, Literal.holds, evalFormula, evalTerm,
binOpToTerm] at h
  cases h; simp [evalTerm, Literal.holds]
  û by_cases hinter :  $\exists x y z, t1 = \text{Term.var } x \wedge t2 = \text{Term.inter } (\text{Term.var } y)$ 
  (Term.var z)
  û obtain ⟨x, y, z, ht1, ht2⟩ := hinter
  subst ht1 ht2
  simp [relationToLiteral?, Literal.holds, evalFormula, evalTerm,
binOpToTerm] at h
  cases h; simp [evalTerm, Literal.holds]
  û by_cases hdiff :  $\exists x y z, t1 = \text{Term.var } x \wedge t2 = \text{Term.diff } (\text{Term.var } y)$ 
  (Term.var z)
  û obtain ⟨x, y, z, ht1, ht2⟩ := hdiff
  subst ht1 ht2
  simp [relationToLiteral?, Literal.holds, evalFormula, evalTerm,
binOpToTerm] at h
  cases h; simp [evalTerm, Literal.holds]
  û exfalse
  have hnone :
    relationToLiteral? (Relation.eq t1 t2) = none := by
  cases t1 with
  | empty =>
    cases t2 <;> simp [relationToLiteral?, varTerm?, binaryOpTerm?]
  | union u v =>
    cases t2 <;> simp [relationToLiteral?, varTerm?, binaryOpTerm?]
  | inter u v =>
    cases t2 <;> simp [relationToLiteral?, varTerm?, binaryOpTerm?]
  | diff u v =>
    cases t2 <;> simp [relationToLiteral?, varTerm?, binaryOpTerm?]
  | var x =>
    cases t2 with
    | empty => exact absurd rfl ht2
    | var n => simp [relationToLiteral?, varTerm?, binaryOpTerm?]
    | union u v =>
      cases u with
      | empty => simp [relationToLiteral?, varTerm?, binaryOpTerm?]

```

```

| union _ _ => simp [relationToLiteral?, varTerm?,
binaryOpTerm?]
| inter _ _ => simp [relationToLiteral?, varTerm?,
binaryOpTerm?]
| diff _ _ => simp [relationToLiteral?, varTerm?, binaryOpTerm
?]
| var y =>
cases v with
| empty => simp [relationToLiteral?, varTerm?, binaryOpTerm
?]
| union _ _ => simp [relationToLiteral?, varTerm?,
binaryOpTerm?]
| inter _ _ => simp [relationToLiteral?, varTerm?,
binaryOpTerm?]
| diff _ _ => simp [relationToLiteral?, varTerm?,
binaryOpTerm?]
| var z =>
exfalse
apply hunion
exact ⟨x, y, z, rfl, rfl⟩
| inter u v =>
cases u with
| empty => simp [relationToLiteral?, varTerm?, binaryOpTerm?]
| union _ _ => simp [relationToLiteral?, varTerm?,
binaryOpTerm?]
| inter _ _ => simp [relationToLiteral?, varTerm?,
binaryOpTerm?]
| diff _ _ => simp [relationToLiteral?, varTerm?, binaryOpTerm
?]
| var y =>
cases v with
| empty => simp [relationToLiteral?, varTerm?, binaryOpTerm
?]
| union _ _ => simp [relationToLiteral?, varTerm?,
binaryOpTerm?]
| inter _ _ => simp [relationToLiteral?, varTerm?,
binaryOpTerm?]
| diff _ _ => simp [relationToLiteral?, varTerm?,
binaryOpTerm?]
| var z =>
exfalse
apply hinter
exact ⟨x, y, z, rfl, rfl⟩
| diff u v =>
cases u with
| empty => simp [relationToLiteral?, varTerm?, binaryOpTerm?]
| union _ _ => simp [relationToLiteral?, varTerm?,
binaryOpTerm?]
| inter _ _ => simp [relationToLiteral?, varTerm?,
binaryOpTerm?]
| diff _ _ => simp [relationToLiteral?, varTerm?, binaryOpTerm
?]
| var y =>
cases v with
| empty => simp [relationToLiteral?, varTerm?, binaryOpTerm
?]

```

```

        | union _ _ => simp [relationToLiteral?, varTerm?,
binaryOpTerm?]
        | inter _ _ => simp [relationToLiteral?, varTerm?,
binaryOpTerm?]
        | diff _ _ => simp [relationToLiteral?, varTerm?,
binaryOpTerm?]
        | var z =>
            exfalse
            apply hdiff
            exact ⟨x, y, z, rfl, rfl⟩
        rw [hnone] at h
        cases h
    | neq t1 t2 =>
        intro lit h
        cases ht1 : varTerm? t1 with
        | none => simp [relationToLiteral?, ht1] at h
        | some x =>
            cases ht2 : varTerm? t2 with
            | none => simp [relationToLiteral?, ht2] at h
            | some y =>
                simp [relationToLiteral?, ht1, ht2, Literal.holds, evalFormula, evalTerm]
            ] at h
            cases h
            rw [varTerm?_eq t1 x ht1, varTerm?_eq t2 y ht2]
            simp [evalTerm, Literal.holds]

def memLiterals (c : Conjunct) : List (Nat × Nat) :=
  c.filterMap fun
    | .mem x y => some (x, y)
    | _ => none

def notMemLiterals (c : Conjunct) : List (Nat × Nat) :=
  c.filterMap fun
    | .notMem x y => some (x, y)
    | _ => none

def neqLiterals (c : Conjunct) : List (Nat × Nat) :=
  c.filterMap fun
    | .neq x y => some (x, y)
    | _ => none

def hasEqEmpty (c : Conjunct) (x : Nat) : Bool :=
  c.any fun | .eqEmpty y => decide (y = x) | _ => false

def hasEqOpLiteral (c : Conjunct) : Bool :=
  c.any fun | .eqOp _ _ _ => true | _ => false

end MLS.EMLS

```

Normalization from general MLS formulas to EMLS conjuncts remains partial (`formulaToConjunct?` in §7); the model-graph decision procedure is §7 only.

0.7 7. Decision Procedures for MLS and EMLS in Lean 4

Phase **2C**. This section merges the decision-procedure mathematics with the Lean encoding. The primary reference is Ferro, Omodeo, and Schwartz [FOS80]—*Decision procedures for some fragments of set theory*—as reproduced in [3-540-10009-1_8.pdf](#) (§3: multilevel syllogistic without singleton or cardinality). TR1995-711 and §6 cite the same **model-graph / elementary-literal** pipeline.

0.7.1 Problem shape (FOS80 §3)

Validity of an MLS formula reduces to satisfiability of a **conjunction** q of elementary literals over set variables $x, y, z, \dots$:

Form	Literal
$(*)$	$x = y \diamond z$ with $\diamond \in \{\cup, \cap, \backslash\}$
$(\in)$	$x \in y$
$(\notin)$	$x \notin y$
$(\neq)$	$x \neq y$

Let q^* be the sub-conjunction of $(*)$ literals, $V_\in$ the variables appearing on the left of $(\in)$ / $(\notin)$ literals, and L_x the literals whose left-hand side is x .

0.7.2 Algorithm outline ([FOS80] §3)

1. **Step 1 (substitution)**: For each $x \in V_\in$, expand L_x using $(\neq)$ literals and merge equivalence classes from q^* ; replace membership literals $x \in y$, $x \notin y$ accordingly.
2. **Step 2**: If any literal $x \neq x$ appears, return **UNSATISFIABLE**.
3. **Step 3**: If $V_\in$ is empty, return **SATISFIABLE**.
4. **Step 4**: Search for a **singleton model** (every variable interpreted as a subset of $\{\emptyset\}$) by assigning an ordering $x < y$ when $M(x) \in M(y)$; detect cycles (e.g. $x \in y \wedge y \in z \wedge z \in x$) as unsatisfiable. In the worst case TR1995-711 cites 2^{4n^3} candidate models for n variables.

[FOS80] §4 extends the language with singletons, cardinality, and arithmetic; we do **not** encode that extension yet.

0.7.3 Lean modules

File	Role
AvgCaseMls/EMLS.lean	Literal, Conjunct, literalToFormula, conjunctToFormula
AvgCaseMls/DecideMLS.lean	formulaToConjunct?, decideConjunct, decideMLSSat, soundness/completeness

Implemented today: partial normalization (formulaToConjunct? on conjunctions of flat literals), FOS80 **Step 2** contradictions ($x \neq x$, $x \in y \wedge x \notin y$), **Step 3** (no membership literals $\Rightarrow$ SAT), and a **partial Step 4** (membership-order cycle detection). **Open**: full Step 1 substitution, complete singleton-model search, and proofs.

Satisfiability (not validity):

- **Soundness:** if `decideMLSSat  $\varphi$  = true`, then $\exists env, \text{evalFormula } env \varphi$.
- **Completeness:** if φ is satisfiable, then `decideMLSSat  $\varphi$  = true` (on the implemented fragment).

0.7.4 AvgCaseMls/DecideMLS.lean

AvgCaseMls/DecideMLS.lean

Lean 4 Certificate

```

/-
Copyright (c) 2026 Catskills Research Company. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Lars Warren Ericson, Catskills Research Company
-/

import AvgCaseMls.EMLS

/-!
FOS80 §3 decision procedure for MLS / EMLS **satisfiability** (Phase **2C**).

Steps 24 are implemented; Step 1 substitution remains open. **Soundness** and **
partial completeness**
on 'InDecideSoundFragment' / 'InDecideSoundFormula' are proved; global
completeness remains open
see ['DEFINITION_FORKS.md'](../DEFINITION_FORKS.md).
-/

namespace MLS

open EMLS

theorem List_any_iff {} (l : List ) (p : → Bool) :
  l.any p = true   $\exists x, x \in l \wedge p\ x = \text{true}$  := by
  induction l with
  | nil => simp
  | cons y ys ih =>
    rw [List.any, Bool.or_eq_true, ih]
    constructor
     $\mathfrak{u}$  intro h
    cases h with
    | inl hp => exact ⟨y, by simp, hp⟩
    | inr h => obtain ⟨x, hx, hpx⟩ := h; exact ⟨x, by simp [hx], hpx⟩
     $\mathfrak{u}$  intro ⟨x, hx, hpx⟩
    rw [List.mem_cons] at hx
    cases hx with
    | inl hx => subst hx; exact Or.inl hpx
    | inr hx => exact Or.inr ⟨x, hx, hpx⟩

/-! ### Formula → EMLS conjunct (Step 0 / partial Step 1) -/

def formulaToConjunct? : Formula → Option Conjunct
| Formula.rel r =>
  EMLS.relationToLiteral? r |>.map List.singleton
| Formula.and f1 f2 => do
  let c1 ← formulaToConjunct? f1

```

```

    let c2 ← formulaToConjunct? f2
    return c1 ++ c2
| _ => none

theorem formulaToConjunct?_and (f1 f2 : Formula) (c1 c2 : Conjunct)
  (h1 : formulaToConjunct? f1 = some c1) (h2 : formulaToConjunct? f2 = some c2)
  ) :
  formulaToConjunct? (Formula.and f1 f2) = some (c1 ++ c2) := by
  simp [formulaToConjunct?, h1, h2]

theorem formulaToConjunct?_satisfies (f : Formula) (c : Conjunct)
  (hc : formulaToConjunct? f = some c) (env : Env)
  (h : ∀ lit ∈ c, Literal.holds env lit) : evalFormula env f := by
  match f with
  | Formula.rel r =>
    cases hl : relationToLiteral? r with
    | none => simp [formulaToConjunct?, hl] at hc
    | some lit =>
      simp [formulaToConjunct?, hl] at hc
      subst hc
      exact (relationToLiteral?_eval env r lit hl).mpr (h lit (List.
        mem_singleton_self lit))
  | Formula.and f1 f2 =>
    cases hc1 : formulaToConjunct? f1 with
    | none => simp [formulaToConjunct?, hc1] at hc
    | some c1 =>
      cases hc2 : formulaToConjunct? f2 with
      | none => simp [formulaToConjunct?, hc2] at hc
      | some c2 =>
        simp [formulaToConjunct?, hc1, hc2] at hc
        subst hc
        exact And.intro
          (formulaToConjunct?_satisfies f1 c1 hc1 env fun lit hl =>
            h lit (List.mem_append.mpr (Or.inl hl)))
          (formulaToConjunct?_satisfies f2 c2 hc2 env fun lit hl =>
            h lit (List.mem_append.mpr (Or.inr hl)))
  | Formula.not _ =>
    simp [formulaToConjunct?] at hc
  | Formula.or _ _ =>
    simp [formulaToConjunct?] at hc
  | Formula.imp _ _ =>
    simp [formulaToConjunct?] at hc
  | Formula.iff _ _ =>
    simp [formulaToConjunct?] at hc

/-! ### FOS80 Step 2 -/

def hasStep2Contradiction (c : Conjunct) : Bool :=
  c.any fun lit1 =>
    c.any fun lit2 =>
      match lit1, lit2 with
      | .neq x y, _ => decide (x = y)
      | .mem x y, .notMem x' y' => decide (x = x' && y = y')
      | _, _ => false

/-! ### FOS80 Step 3 -/

```

```

def hasMembershipLiteral (c : Conjunct) : Bool :=
  c.any fun
    | .mem _ _ | .notMem _ _ => true
    | _ => false

def hasStep3Obstruction (c : Conjunct) : Bool :=
  (neqLiterals c).any fun (x, y) =>
    decide (x ≠ y && hasEqEmpty c x && hasEqEmpty c y)

noncomputable def witnessEnv (c : Conjunct) : Env :=
  fun n => if hasEqEmpty c n then ZFSet.empty else ZFSet.tag n

/-! ### Literal / conjunct helpers -/

theorem hasEqEmpty_of_mem (c : Conjunct) (x : Nat) (hx : .eqEmpty x ∈ c) :
  hasEqEmpty c x = true := by
  rw [hasEqEmpty, List_any_iff]
  exact ⟨.eqEmpty x, hx, by simp⟩

theorem hasEqOpLiteral_of_eqOp (c : Conjunct) (x y z : Nat) (op : BinOp)
  (hx : .eqOp x y z op ∈ c) : hasEqOpLiteral c = true := by
  rw [hasEqOpLiteral, List_any_iff]
  exact ⟨.eqOp x y z op, hx, by simp⟩

theorem hasMembershipLiteral_of_mem (c : Conjunct) (x y : Nat) (hx : .mem x y ∈
  c) :
  hasMembershipLiteral c = true := by
  rw [hasMembershipLiteral, List_any_iff]
  exact ⟨.mem x y, hx, by simp⟩

theorem hasMembershipLiteral_of_notMem (c : Conjunct) (x y : Nat) (hx : .notMem
  x y ∈ c) :
  hasMembershipLiteral c = true := by
  rw [hasMembershipLiteral, List_any_iff]
  exact ⟨.notMem x y, hx, by simp⟩

theorem hasStep2_of_neq_refl (c : Conjunct) (x : Nat) (hx : .neq x x ∈ c) :
  hasStep2Contradiction c = true := by
  unfold hasStep2Contradiction
  rw [List_any_iff]
  refine ⟨.neq x x, hx, ?_⟩
  rw [List_any_iff]
  exact ⟨.neq x x, hx, by simp⟩

theorem hasStep2_of_mem_notMem (c : Conjunct) (x y : Nat) (hmem : .mem x y ∈ c)
  (hnot : .notMem x y ∈ c) : hasStep2Contradiction c = true := by
  unfold hasStep2Contradiction
  rw [List_any_iff]
  refine ⟨.mem x y, hmem, ?_⟩
  rw [List_any_iff]
  exact ⟨.notMem x y, hnot, by simp⟩

theorem hasStep3Obstruction_of (c : Conjunct) (x y : Nat) (hxy : x ≠ y)
  (hx : hasEqEmpty c x = true) (hy : hasEqEmpty c y = true) (hne : .neq x y ∈
  c) :

```

```

    hasStep3Obstruction c = true := by
    unfold hasStep3Obstruction
    rw [List_any_iff]
    refine ⟨(x, y), ?_, ?_⟩
    û simp [neqLiterals, List.mem_filterMap]
    exact ⟨.neq x y, hne, rfl⟩
    û simp [hxy, hx, hy]

namespace Step2

theorem neq_refl_unsat (env : Env) (x : Nat) : ¬ Literal.holds env (.neq x x) :=
  by
  simp [Literal.holds, evalTerm]

theorem mem_notMem_unsat (env : Env) (x y : Nat) :
  ¬ (Literal.holds env (.mem x y) ∧ Literal.holds env (.notMem x y)) := by
  intro ⟨hmem, hnot⟩
  exact hnot hmem

theorem unsat (c : Conjunct) (h : hasStep2Contradiction c = true) :
  ¬ ∃ env, ∀ lit ∈ c, Literal.holds env lit := by
  intro ⟨env, hall⟩
  unfold hasStep2Contradiction at h
  rw [List_any_iff] at h
  obtain ⟨lit1, hl1, h1⟩ := h
  rw [List_any_iff] at h1
  obtain ⟨lit2, hl2, h2⟩ := h1
  cases lit1 with
  | neq x y =>
    have hxy : x = y := by simp using h2
    subst hxy
    exact neq_refl_unsat env x (hall _ hl1)
  | mem x y =>
    cases lit2 with
    | notMem x' y' =>
      have hpair : x = x' ∧ y = y' := by
        simp [Bool.and_eq_true, decide_eq_true_iff] at h2
        exact h2
      rcases hpair with ⟨rfl, rfl⟩
      exact mem_notMem_unsat env x y ⟨hall _ hl1, hall _ hl2⟩
    | _ => simp at h2
  | _ => simp at h2

end Step2

namespace Step3

theorem witness_eqEmpty (c : Conjunct) (x : Nat) (hx : .eqEmpty x ∈ c) :
  Literal.holds (witnessEnv c) (.eqEmpty x) := by
  have he := hasEqEmpty_of_mem c x hx
  simp [Literal.holds, witnessEnv, he, evalTerm]

theorem witness_neq (c : Conjunct) (x y : Nat) (hxy : x ≠ y) (_hne : .neq x y ∈
  c)
  (hob : hasStep3Obstruction c = false) :
  Literal.holds (witnessEnv c) (.neq x y) := by

```

```

simp only [Literal.holds, witnessEnv, evalTerm]
by_cases hx : hasEqEmpty c x = true
û by_cases hy : hasEqEmpty c y = true
  û exact absurd (hasStep3Obstruction_of c x y hxy hx hy _hne) (by simp using
    hob)
  û simp [hx, hy]
    exact Ne.symm (ZFSet.tag_ne_empty y)
û by_cases hy : hasEqEmpty c y = true
  û simp [hx, hy]
    exact ZFSet.tag_ne_empty x
  û simp [hx, hy, ZFSet.tag_injective.ne hxy]

theorem witness (c : Conjunct) (h2 : hasStep2Contradiction c = false)
  (hmem : hasMembershipLiteral c = false) (hop : hasEqOpLiteral c = false)
  (h3 : hasStep3Obstruction c = false) :
  ∃ env, ∀ lit ∈ c, Literal.holds env lit := by
  refine ⟨witnessEnv c, fun lit hl => ?_⟩
  cases lit with
  | eqOp x y z op =>
    exact absurd (hasEqOpLiteral_of_eqOp c x y z op hl) (by simp using hop)
  | eqEmpty x =>
    exact witness_eqEmpty c x hl
  | mem x y =>
    exact absurd (hasMembershipLiteral_of_mem c x y hl) (by simp using hmem)
  | notMem x y =>
    exact absurd (hasMembershipLiteral_of_notMem c x y hl) (by simp using hmem)
  | neq x y =>
    by_cases hxy : x = y
    û exact absurd (hasStep2_of_neq_refl c x (by simp [hxy] using hl)) (by
      simp using h2)
    û exact witness_neq c x y hxy hl h3

end Step3

/-! ### FOS80 Step 4 (partial) -/

/-- Syntactic membership-cycle check; self-loops refuted semantically by ['EMLS.
step4_self_loop_unsat']. -/

def varsInConjunct (c : Conjunct) : List Nat :=
  (c.flatMap fun
    | .eqOp x y z _ => [x, y, z]
    | .eqEmpty x => [x]
    | .mem x y | .notMem x y | .neq x y => [x, y]).eraseDups

def hasMemCycle (edges : List (Nat × Nat)) (nodes : List Nat) : Bool :=
  let fuel := nodes.length + 1
  let rec dfs (remaining : Nat) (path : List Nat) (u : Nat) : Bool :=
    if remaining = 0 then
      false
    else if path.contains u then
      true
    else
      let path' := u :: path
      (edges.filterMap fun (a, b) => if a = u then some b else none).any (dfs (
        remaining - 1) path')

```

```

nodes.any fun start => dfs fuel [] start

def hasStep4Obstruction (c : Conjunct) : Bool :=
  let mem := memLiterals c
  let notMem := notMemLiterals c
  let nodes := varsInConjunct c
  hasMemCycle mem nodes ||
    (notMem.any fun (x, y) => decide (x = y && mem.contains (x, y)))

/-! ### Decision -/

def decideConjunct (c : Conjunct) : Bool :=
  if hasStep2Contradiction c then false
  else if hasStep3Obstruction c then false
  else if !hasMembershipLiteral c then true
  else if hasStep4Obstruction c then false
  else true

theorem decideConjunct_true_of_step3 (c : Conjunct)
  (h2 : hasStep2Contradiction c = false) (h3o : hasStep3Obstruction c = false)
  (hmem : hasMembershipLiteral c = false) :
  decideConjunct c = true := by
  simp [decideConjunct, h2, h3o, hmem]

def decideEMLSSat (c : Conjunct) : Bool :=
  decideConjunct c

def decideMLSSat (f : Formula) : Bool :=
  match formulaToConjunct? f with
  | some c => decideConjunct c
  | none =>
    match f with
    | Formula.rel (Relation.eq Term.empty Term.empty) => true
    | Formula.rel (Relation.neq Term.empty Term.empty) => false
    | _ => false

def decideEMLSSat? (c : Conjunct) : Option Bool :=
  conjunctToFormula c |>.map decideMLSSat

abbrev decideMLS := decideMLSSat

/-! ### Sound fragment -/

def InDecideSoundFragment (c : Conjunct) : Prop :=
  hasStep2Contradiction c = false ∧
  hasStep3Obstruction c = false ∧
  hasMembershipLiteral c = false ∧
  hasEqOpLiteral c = false

theorem decideConjunct_sound (c : Conjunct) (_h : decideConjunct c = true)
  (hfrag : InDecideSoundFragment c) :
  ∃ env, ∀ lit ∈ c, Literal.holds env lit :=
  Step3.witness c hfrag.1 hfrag.2.2.1 hfrag.2.2.2 hfrag.2.1

theorem decideConjunct_unsat_step2 (c : Conjunct) (h2 : hasStep2Contradiction c
  = true) :

```

```

    decideConjunct c = false := by simp [decideConjunct, h2]

theorem decideConjunct_refutes_step2 (c : Conjunct) (h2 : hasStep2Contradiction
  c = true) :
  ¬ ∃ env, ∀ lit ∈ c, Literal.holds env lit :=
  Step2.unsat c h2

def InDecideSoundFormula (f : Formula) : Prop :=
  ∃ c, formulaToConjunct? f = some c ∧ InDecideSoundFragment c

theorem decideMLSSat_sound (f : Formula) (h : decideMLSSat f = true)
  (hfrag : InDecideSoundFormula f) :
  ∃ env, evalFormula env f := by
  obtain ⟨c, hc, hfragc⟩ := hfrag
  simp [decideMLSSat, hc] at h
  obtain ⟨env, henv⟩ := decideConjunct_sound c h hfragc
  exact ⟨env, formulaToConjunct?_satisfies f c hc env henv⟩

theorem decideMLS_sound (f : Formula) (h : decideMLS f = true)
  (hfrag : InDecideSoundFormula f) :
  ∃ env, evalFormula env f :=
  decideMLSSat_sound f h hfrag

/-! ### Partial completeness (sound, membership-free fragment) -/

/--
On the sound fragment, ['decideConjunct'] never returns 'false': Step 2/3 do not
fire and
membership literals are absent, so the procedure accepts after the membership
guard.
-/
theorem decideConjunct_complete_sound_fragment (c : Conjunct) (hfrag :
  InDecideSoundFragment c) :
  decideConjunct c = true := by
  unfold decideConjunct
  have h2 := hfrag.1
  have h3 := hfrag.2.1
  have hmem := hfrag.2.2.1
  simp [h2, h3, hmem]

/--
Formula-level partial completeness: on ['InDecideSoundFormula'], ['decideMLSSat
'] returns 'true'.
-/
theorem decideMLSSat_complete_sound_fragment (f : Formula) (hfrag :
  InDecideSoundFormula f) :
  decideMLSSat f = true := by
  obtain ⟨c, hc, hfragc⟩ := hfrag
  simp [decideMLSSat, hc]
  exact decideConjunct_complete_sound_fragment c hfragc

theorem decideMLS_complete_sound_fragment (f : Formula) (hfrag :
  InDecideSoundFormula f) :
  decideMLS f = true :=
  decideMLSSat_complete_sound_fragment f hfrag

```



```

/-! ### Global completeness not proved (Step 1 / full Step 4 open) -/

/--
Full FOS80 completeness: requires Step 1 substitution and complete Step 4 model
search.
See ['decideMLSSat_complete_sound_fragment'] for the verified membership-free
sound fragment.
-/
theorem decideMLSSat_complete (f : Formula) (_h : ∃ env, evalFormula env f) :
  decideMLSSat f = true := by
  sorry

theorem decideMLS_complete (f : Formula) (h : ∃ env, evalFormula env f) :
  decideMLS f = true :=
  decideMLSSat_complete f h

end MLS

```

The live implementation is [AvgCaseMls/DecideMLS.lean](#). A **step-counting** function `stepsMLS` (Phase 2D) will relate the procedure to $\text{Av}(T)$ in §5.

0.8 8. Lean 4 Verification: Proving Average-Case Hardness Properties

The `AvCom` classes are defined in §5; `MLS` syntax is in §6; decision procedures are in §7.

0.8.1 Phase 3 — Encoding and NP membership

[AvgCaseMls/Serialization.lean](#) defines `serializeFormula`, `SatMLS`, and encoding-size bounds. [AvgCaseMls/NPMembership.lean](#) proves checker-based NP membership (`SatMLSChecker_in_NP`).

0.8.2 AvgCaseMls/Serialization.lean

[AvgCaseMls/Serialization.lean](#)

Lean 4 Certificate

```

/-
Copyright (c) 2026 Catskills Research Company. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Lars Warren Ericson, Catskills Research Company
-/

import AvgCaseMls.AvCom
import AvgCaseMls.DecideMLS
import Mathlib.Tactic

/-!
Phase **2D:** binary encoding of MLS formulas and a syntactic step budget for ['
decideMLS'].

See ['DEFINITION_FORKS.md'](../DEFINITION_FORKS.md) for encoding choices.
Polynomial

```

```

size bounds (Phase **3B**) can cite ['formulaSize'] and ['len_serializeFormula
'].
-/

namespace MLS

open AvCom EMLS

/-! ### Nat and syntax tags -/

/-- Unary Nat encoding: 'n' copies of 'true' followed by 'false'. -/
def encodeNat : Nat → Bitstring
| 0 => [false]
| n + 1 => true :: encodeNat n

def termTag : Term → Bitstring
| .var _ => [false, false, false]
| .empty => [false, false, true]
| .union _ _ => [false, true, false]
| .inter _ _ => [false, true, true]
| .diff _ _ => [true, false, false]

def relationTag : Relation → Bitstring
| .mem _ _ => [false, false]
| .not_mem _ _ => [false, true]
| .eq _ _ => [true, false]
| .neq _ _ => [true, true]

def formulaTag : Formula → Bitstring
| .rel _ => [false, false, false]
| .not _ => [false, false, true]
| .and _ _ => [false, true, false]
| .or _ _ => [false, true, true]
| .imp _ _ => [true, false, false]
| .iff _ _ => [true, false, true]

@[simp] theorem len_encodeNat (n : Nat) : len (encodeNat n) = n + 1 := by
  induction n with
  | zero => simp [encodeNat, len]
  | succ n ih =>
    simp only [encodeNat, len, List.length_cons]
    rw [← len_eq, ih]

@[simp] theorem length_encodeNat (n : Nat) : (encodeNat n).length = n + 1 := by
  simpa [len_eq] using len_encodeNat n

@[simp] theorem len_termTag (t : Term) : len (termTag t) = 3 := by
  cases t <;> simp [termTag, len]

@[simp] theorem len_relationTag (r : Relation) : len (relationTag r) = 2 := by
  cases r <;> simp [relationTag, len]

@[simp] theorem len_formulaTag (f : Formula) : len (formulaTag f) = 3 := by
  cases f <;> simp [formulaTag, len]

/-! ### Wire sizes (exact serialized length) -/

```

```

def wireSizeNat (n : Nat) : Nat := n + 1

def wireSizeTerm : Term → Nat
| .var n => 3 + wireSizeNat n
| .empty => 3
| .union t1 t2 => 3 + wireSizeTerm t1 + wireSizeTerm t2
| .inter t1 t2 => 3 + wireSizeTerm t1 + wireSizeTerm t2
| .diff t1 t2 => 3 + wireSizeTerm t1 + wireSizeTerm t2

def wireSizeRelation : Relation → Nat
| .mem t1 t2 => 2 + wireSizeTerm t1 + wireSizeTerm t2
| .not_mem t1 t2 => 2 + wireSizeTerm t1 + wireSizeTerm t2
| .eq t1 t2 => 2 + wireSizeTerm t1 + wireSizeTerm t2
| .neq t1 t2 => 2 + wireSizeTerm t1 + wireSizeTerm t2

def wireSizeFormula : Formula → Nat
| .rel r => 3 + wireSizeRelation r
| .not f => 3 + wireSizeFormula f
| .and f1 f2 => 3 + wireSizeFormula f1 + wireSizeFormula f2
| .or f1 f2 => 3 + wireSizeFormula f1 + wireSizeFormula f2
| .imp f1 f2 => 3 + wireSizeFormula f1 + wireSizeFormula f2
| .iff f1 f2 => 3 + wireSizeFormula f1 + wireSizeFormula f2

abbrev formulaSize := wireSizeFormula

/-' ### Serialization -'/

def serializeTerm : Term → Bitstring
| .var n => termTag (.var n) ++ encodeNat n
| .empty => termTag Term.empty
| .union t1 t2 => termTag (.union t1 t2) ++ serializeTerm t1 ++ serializeTerm t2
| .inter t1 t2 => termTag (.inter t1 t2) ++ serializeTerm t1 ++ serializeTerm t2
| .diff t1 t2 => termTag (.diff t1 t2) ++ serializeTerm t1 ++ serializeTerm t2

def serializeRelation : Relation → Bitstring
| .mem t1 t2 => relationTag (.mem t1 t2) ++ serializeTerm t1 ++ serializeTerm t2
| .not_mem t1 t2 => relationTag (.not_mem t1 t2) ++ serializeTerm t1 ++
serializeTerm t2
| .eq t1 t2 => relationTag (.eq t1 t2) ++ serializeTerm t1 ++ serializeTerm t2
| .neq t1 t2 => relationTag (.neq t1 t2) ++ serializeTerm t1 ++ serializeTerm t2

def serializeFormula : Formula → Bitstring
| .rel r => formulaTag (.rel r) ++ serializeRelation r
| .not f => formulaTag (.not f) ++ serializeFormula f
| .and f1 f2 => formulaTag (.and f1 f2) ++ serializeFormula f1 ++
serializeFormula f2
| .or f1 f2 => formulaTag (.or f1 f2) ++ serializeFormula f1 ++
serializeFormula f2
| .imp f1 f2 => formulaTag (.imp f1 f2) ++ serializeFormula f1 ++
serializeFormula f2

```

```

| .iff f1 f2 => formulaTag (.iff f1 f2) ++ serializeFormula f1 ++
  serializeFormula f2

theorem length_serializeTerm (t : Term) : (serializeTerm t).length =
  wireSizeTerm t := by
  induction t with
  | var n =>
    simp [serializeTerm, wireSizeTerm, wireSizeNat, termTag, List.length_append,
      length_encodeNat]
    omega
  | empty => simp [serializeTerm, wireSizeTerm, termTag]
  | union t1 t2 ih1 ih2 =>
    simp [serializeTerm, wireSizeTerm, termTag, List.length_append, ih1, ih2]
    omega
  | inter t1 t2 ih1 ih2 =>
    simp [serializeTerm, wireSizeTerm, termTag, List.length_append, ih1, ih2]
    omega
  | diff t1 t2 ih1 ih2 =>
    simp [serializeTerm, wireSizeTerm, termTag, List.length_append, ih1, ih2]
    omega

theorem len_serializeTerm (t : Term) : len (serializeTerm t) = wireSizeTerm t :=
  by
  simp [len_eq] using length_serializeTerm t

theorem length_serializeRelation (r : Relation) : (serializeRelation r).length =
  wireSizeRelation r := by
  cases r with
  | mem t1 t2 =>
    simp [serializeRelation, wireSizeRelation, relationTag, List.length_append,
      length_serializeTerm t1, length_serializeTerm t2]
    omega
  | not_mem t1 t2 =>
    simp [serializeRelation, wireSizeRelation, relationTag, List.length_append,
      length_serializeTerm t1, length_serializeTerm t2]
    omega
  | eq t1 t2 =>
    simp [serializeRelation, wireSizeRelation, relationTag, List.length_append,
      length_serializeTerm t1, length_serializeTerm t2]
    omega
  | neq t1 t2 =>
    simp [serializeRelation, wireSizeRelation, relationTag, List.length_append,
      length_serializeTerm t1, length_serializeTerm t2]
    omega

theorem len_serializeRelation (r : Relation) : len (serializeRelation r) =
  wireSizeRelation r := by
  simp [len_eq] using length_serializeRelation r

theorem length_serializeFormula (f : Formula) : (serializeFormula f).length =
  wireSizeFormula f := by
  induction f with
  | rel r =>
    simp [serializeFormula, wireSizeFormula, formulaTag, List.length_append,
      length_serializeRelation r]
    omega

```

```

| not f ih =>
  simp [serializeFormula, wireSizeFormula, formulaTag, List.length_append, ih]
  omega
| and f1 f2 ih1 ih2 =>
  simp [serializeFormula, wireSizeFormula, formulaTag, List.length_append, ih1
    , ih2]
  omega
| or f1 f2 ih1 ih2 =>
  simp [serializeFormula, wireSizeFormula, formulaTag, List.length_append, ih1
    , ih2]
  omega
| imp f1 f2 ih1 ih2 =>
  simp [serializeFormula, wireSizeFormula, formulaTag, List.length_append, ih1
    , ih2]
  omega
| iff f1 f2 ih1 ih2 =>
  simp [serializeFormula, wireSizeFormula, formulaTag, List.length_append, ih1
    , ih2]
  omega

theorem len_serializeFormula (f : Formula) : len (serializeFormula f) =
  wireSizeFormula f := by
  simpa [len_eq] using length_serializeFormula f

theorem wireSizeTerm_pos (t : Term) : 0 < wireSizeTerm t := by
  cases t <;> simp [wireSizeTerm, wireSizeNat]

theorem formulaSize_pos (f : Formula) : 0 < wireSizeFormula f := by
  cases f with
  | rel r =>
    cases r <;> simp [wireSizeFormula, wireSizeRelation, wireSizeTerm,
      wireSizeNat] <;> omega
  | not f =>
    simp only [wireSizeFormula]
    have h := formulaSize_pos f
    omega
  | and f1 f2 =>
    simp only [wireSizeFormula]
    have h1 := formulaSize_pos f1
    have h2 := formulaSize_pos f2
    omega
  | or f1 f2 =>
    simp only [wireSizeFormula]
    have h1 := formulaSize_pos f1
    have h2 := formulaSize_pos f2
    omega
  | imp f1 f2 =>
    simp only [wireSizeFormula]
    have h1 := formulaSize_pos f1
    have h2 := formulaSize_pos f2
    omega
  | iff f1 f2 =>
    simp only [wireSizeFormula]
    have h1 := formulaSize_pos f1
    have h2 := formulaSize_pos f2
    omega

```

```

/-! ### Step budget for ['decideMLS'] -/

/-- Syntactic step budget for FOS80 Steps 24 on conjunct 'c'. -/
def stepsConjunct (c : Conjunct) : Nat :=
  let n := c.length
  n * n + (neqLiterals c).length + (varsInConjunct c).length + 1

/--
Step budget for ['decideMLSSat']: AST size plus conjunct decision work when
['formulaToConjunct?'] succeeds.
-/
def stepsMLS (f : Formula) : Nat :=
  wireSizeFormula f +
  match formulaToConjunct? f with
  | none => 0
  | some c => stepsConjunct c

theorem stepsMLS_pos (f : Formula) : 0 < stepsMLS f := by
  simp only [stepsMLS]
  exact Nat.add_pos_left (formulaSize_pos f) _

/-! ### Phase 3B syntax mass and polynomial encoding bounds -/

def termNodes : Term → Nat
| .var _ => 1
| .empty => 1
| .union t1 t2 => 1 + termNodes t1 + termNodes t2
| .inter t1 t2 => 1 + termNodes t1 + termNodes t2
| .diff t1 t2 => 1 + termNodes t1 + termNodes t2

def maxVarTerm : Term → Nat
| .var n => n
| .empty => 0
| .union t1 t2 => max (maxVarTerm t1) (maxVarTerm t2)
| .inter t1 t2 => max (maxVarTerm t1) (maxVarTerm t2)
| .diff t1 t2 => max (maxVarTerm t1) (maxVarTerm t2)

def relationNodes : Relation → Nat
| .mem t1 t2 => 1 + termNodes t1 + termNodes t2
| .not_mem t1 t2 => 1 + termNodes t1 + termNodes t2
| .eq t1 t2 => 1 + termNodes t1 + termNodes t2
| .neq t1 t2 => 1 + termNodes t1 + termNodes t2

def maxVarRelation : Relation → Nat
| .mem t1 t2 => max (maxVarTerm t1) (maxVarTerm t2)
| .not_mem t1 t2 => max (maxVarTerm t1) (maxVarTerm t2)
| .eq t1 t2 => max (maxVarTerm t1) (maxVarTerm t2)
| .neq t1 t2 => max (maxVarTerm t1) (maxVarTerm t2)

def formulaNodes : Formula → Nat
| .rel r => 1 + relationNodes r
| .not f => 1 + formulaNodes f
| .and f1 f2 => 1 + formulaNodes f1 + formulaNodes f2
| .or f1 f2 => 1 + formulaNodes f1 + formulaNodes f2
| .imp f1 f2 => 1 + formulaNodes f1 + formulaNodes f2

```

```

| .iff f1 f2 => 1 + formulaNodes f1 + formulaNodes f2

def maxVarFormula : Formula → Nat
| .rel r => maxVarRelation r
| .not f => maxVarFormula f
| .and f1 f2 => max (maxVarFormula f1) (maxVarFormula f2)
| .or f1 f2 => max (maxVarFormula f1) (maxVarFormula f2)
| .imp f1 f2 => max (maxVarFormula f1) (maxVarFormula f2)
| .iff f1 f2 => max (maxVarFormula f1) (maxVarFormula f2)

/-- Single-parameter syntax mass: AST node count plus largest variable index. -/
def formulaAstMass (f : Formula) : Nat :=
  formulaNodes f + maxVarFormula f

theorem termNodes_pos (t : Term) : 0 < termNodes t := by
  cases t <;> simp [termNodes] <;> omega

theorem relationNodes_pos (r : Relation) : 0 < relationNodes r := by
  cases r <;> simp [relationNodes, termNodes] <;> omega

/-- Polynomial slack bound helper (Phase **3B**). Quadratic in 'n + 2'. -/
def nodeBound (n : Nat) : Nat :=
  (n + 2) ^ 2 * 1000000000000 + 1000000000000

theorem nodeBound_mono {m n : Nat} (h : m ≤ n) : nodeBound m ≤ nodeBound n :=
  by
    have hb : m + 2 ≤ n + 2 := Nat.add_le_add_right h 2
    have h2 := Nat.pow_le_pow_left hb 2
    simp [nodeBound]
    nlinarith

theorem formulaNodes_pos (f : Formula) : 0 < formulaNodes f := by
  cases f with
  | rel r => simp [formulaNodes, relationNodes_pos r]
  | not f => simp [formulaNodes, formulaNodes_pos f]
  | and f1 f2 => simp [formulaNodes, formulaNodes_pos f1, formulaNodes_pos f2]
  | or f1 f2 => simp [formulaNodes, formulaNodes_pos f1, formulaNodes_pos f2]
  | imp f1 f2 => simp [formulaNodes, formulaNodes_pos f1, formulaNodes_pos f2]
  | iff f1 f2 => simp [formulaNodes, formulaNodes_pos f1, formulaNodes_pos f2]

def termMass (t : Term) : Nat := termNodes t + maxVarTerm t

def relationMass (r : Relation) : Nat := relationNodes r + maxVarRelation r

def formulaMass (f : Formula) : Nat := formulaNodes f + maxVarFormula f

theorem termMass_pos (t : Term) : 0 < termMass t := by
  cases t <;> simp [termMass, termNodes, maxVarTerm] <;> omega

theorem relationMass_pos (r : Relation) : 0 < relationMass r := by
  cases r <;> simp [relationMass, relationNodes, maxVarRelation, termNodes,
    termMass] <;> omega

theorem formulaMass_pos (f : Formula) : 0 < formulaMass f := by
  cases f <;> simp [formulaMass, formulaNodes, maxVarFormula, relationMass,
    relationNodes] <;>

```

```

    try linarith [relationMass_pos _, formulaMass_pos _, formulaNodes_pos _,
relationNodes_pos _] <;>
    omega

theorem termMass_lt_combine_left (t1 t2 : Term) :
  termMass t1 + 1 < 1 + termNodes t1 + termNodes t2 + max (maxVarTerm t1) (
    maxVarTerm t2) := by
  simp [termMass, termNodes, maxVarTerm]
  have := termNodes_pos t2
  omega

theorem termMass_lt_combine_right (t1 t2 : Term) :
  termMass t2 + 1 < 1 + termNodes t1 + termNodes t2 + max (maxVarTerm t1) (
    maxVarTerm t2) := by
  simp [termMass, termNodes, maxVarTerm]
  have := termNodes_pos t1
  omega

theorem termMass_lt_union_left (t1 t2 : Term) : termMass t1 + 1 < termMass (.
  union t1 t2) := by
  simp [termMass, termNodes, maxVarTerm] using termMass_lt_combine_left t1 t2

theorem termMass_lt_union_right (t1 t2 : Term) : termMass t2 + 1 < termMass (.
  union t1 t2) := by
  simp [termMass, termNodes, maxVarTerm] using termMass_lt_combine_right t1 t2

theorem formulaMass_lt_combine_left (f1 f2 : Formula) :
  formulaMass f1 + 1 <
    1 + formulaNodes f1 + formulaNodes f2 + max (maxVarFormula f1) (
      maxVarFormula f2) := by
  simp [formulaMass, formulaNodes, maxVarFormula]
  have := formulaNodes_pos f2
  omega

theorem formulaMass_lt_combine_right (f1 f2 : Formula) :
  formulaMass f2 + 1 <
    1 + formulaNodes f1 + formulaNodes f2 + max (maxVarFormula f1) (
      maxVarFormula f2) := by
  simp [formulaMass, formulaNodes, maxVarFormula]
  have := formulaNodes_pos f1
  omega

theorem nodeBound_pow_gap (m : Nat) :
  1000000000000 * ((m + 3) ^ 2 - (m + 2) ^ 2) ≥ 3 := by
  have hsq : (m + 3) ^ 2 = (m + 2) ^ 2 + (2 * m + 5) := by ring_nf
  have : 3 ≤ 2 * m + 5 := by omega
  omega

theorem nodeBound_succ (m : Nat) : nodeBound m + 3 ≤ nodeBound (m + 1) := by
  simp [nodeBound]
  have hsq : (m + 3) ^ 2 = (m + 2) ^ 2 + (2 * m + 5) := by ring_nf
  nlinarith

theorem nodeBound_add3_le_succ (m : Nat) : 3 + nodeBound m ≤ nodeBound (m + 1)
  := by
  simp [Nat.add_comm] using nodeBound_succ m

```



```

/-- Sum of two child ['nodeBound']s fits at combined index 'a + b + 1' (Phase
**3B**). -/
theorem nodeBound_pair_sum_le {a b : Nat} (ha : 1 ≤ a) (hb : 1 ≤ b) :
  3 + nodeBound a + nodeBound b ≤ nodeBound (a + b + 1) := by
  simp [nodeBound]
  nlinarith [sq_nonneg (a : ), sq_nonneg (b : )]

theorem termMass_combine_le (t1 t2 : Term) :
  termMass (.union t1 t2) ≤ termMass t1 + termMass t2 + 1 := by
  simp [termMass, termNodes, maxVarTerm]
  omega

theorem termMass_inter_le (t1 t2 : Term) :
  termMass (.inter t1 t2) ≤ termMass t1 + termMass t2 + 1 := by
  simp [termMass, termNodes, maxVarTerm]
  omega

theorem termMass_diff_le (t1 t2 : Term) :
  termMass (.diff t1 t2) ≤ termMass t1 + termMass t2 + 1 := by
  simp [termMass, termNodes, maxVarTerm]
  omega

theorem formulaMass_combine_le (f1 f2 : Formula) :
  formulaMass (f1.and f2) ≤ formulaMass f1 + formulaMass f2 + 1 := by
  simp [formulaMass, formulaNodes, maxVarFormula]
  omega

theorem formulaMass_or_le (f1 f2 : Formula) :
  formulaMass (f1.or f2) ≤ formulaMass f1 + formulaMass f2 + 1 := by
  simp [formulaMass, formulaNodes, maxVarFormula]
  omega

theorem formulaMass_imp_le (f1 f2 : Formula) :
  formulaMass (f1.imp f2) ≤ formulaMass f1 + formulaMass f2 + 1 := by
  simp [formulaMass, formulaNodes, maxVarFormula]
  omega

theorem formulaMass_iff_le (f1 f2 : Formula) :
  formulaMass (f1.iff f2) ≤ formulaMass f1 + formulaMass f2 + 1 := by
  simp [formulaMass, formulaNodes, maxVarFormula]
  omega

private theorem sqMass_le_nodeBound (m : Nat) : (m + 30) ^ 2 ≤ nodeBound m :=
  by
  simp [nodeBound]
  nlinarith [sq_nonneg (m : )]

private def wireEndSlack : Nat := 603

private theorem wireSizeTerm_rec_bound (t : Term) :
  wireSizeTerm t ≤ (termNodes t + 1) * (maxVarTerm t + 4) + 3 * termNodes t
  := by
  induction t with
  | var n =>
    simp [wireSizeTerm, termNodes, maxVarTerm, wireSizeNat]

```

```

ring_nf
omega
| empty => simp [wireSizeTerm, termNodes, maxVarTerm]
| union t1 t2 ih1 ih2 | inter t1 t2 ih1 ih2 | diff t1 t2 ih1 ih2 =>
  have hM1 : maxVarTerm t1 ≤ max (maxVarTerm t1) (maxVarTerm t2) := Nat.
  le_max_left _ _
  have hM2 : maxVarTerm t2 ≤ max (maxVarTerm t1) (maxVarTerm t2) := Nat.
  le_max_right _ _
  have ih1' : wireSizeTerm t1 ≤
    (termNodes t1 + 1) * (max (maxVarTerm t1) (maxVarTerm t2) + 4) + 3 *
  termNodes t1 := by
    nlinarith [ih1, hM1]
  have ih2' : wireSizeTerm t2 ≤
    (termNodes t2 + 1) * (max (maxVarTerm t1) (maxVarTerm t2) + 4) + 3 *
  termNodes t2 := by
    nlinarith [ih2, hM2]
  simp [wireSizeTerm, termNodes, maxVarTerm]
  nlinarith [ih1', ih2', termNodes_pos t1, termNodes_pos t2]

private theorem wireSizeTerm_mul_bound (t : Term) :
  wireSizeTerm t ≤ (termNodes t + 1) * (maxVarTerm t + 4) + 3 * termNodes t +
  wireEndSlack := by
  nlinarith [wireSizeTerm_rec_bound t]

private theorem wireSizeTerm_mul_bound_le_nodeBound (t : Term) :
  (termNodes t + 1) * (maxVarTerm t + 4) + 3 * termNodes t + wireEndSlack ≤
  nodeBound (termMass t) := by
  have h1 : (termNodes t + 1) * (maxVarTerm t + 4) + 3 * termNodes t +
  wireEndSlack ≤
    (termMass t + 30) ^ 2 := by
    simp [termMass, wireEndSlack]
    nlinarith [sq_nonneg (termNodes t : ), sq_nonneg (maxVarTerm t : ),
    termNodes_pos t]
  exact Nat.le_trans h1 (sqMass_le_nodeBound (termMass t))

private theorem wireSizeTerm_le_nodeBound_aux (t : Term) : wireSizeTerm t ≤
  nodeBound (termMass t) :=
  Nat.le_trans (wireSizeTerm_mul_bound t) (wireSizeTerm_mul_bound_le_nodeBound t
  )

private theorem wireSizeRelation_rec_bound (r : Relation) :
  wireSizeRelation r ≤ (relationNodes r + 1) * (maxVarRelation r + 4) + 3 *
  relationNodes r := by
  cases r with
  | mem t1 t2 | not_mem t1 t2 | eq t1 t2 | neq t1 t2 =>
    have hM1 : maxVarTerm t1 ≤ max (maxVarTerm t1) (maxVarTerm t2) := Nat.
    le_max_left _ _
    have hM2 : maxVarTerm t2 ≤ max (maxVarTerm t1) (maxVarTerm t2) := Nat.
    le_max_right _ _
    have ih1' : wireSizeTerm t1 ≤
      (termNodes t1 + 1) * (max (maxVarTerm t1) (maxVarTerm t2) + 4) + 3 *
    termNodes t1 := by
      nlinarith [wireSizeTerm_rec_bound t1, hM1]
    have ih2' : wireSizeTerm t2 ≤
      (termNodes t2 + 1) * (max (maxVarTerm t1) (maxVarTerm t2) + 4) + 3 *
    termNodes t2 := by

```

```

    nlinarith [wireSizeTerm_rec_bound t2, hM2]
    simp [wireSizeRelation, relationNodes, maxVarRelation, relationMass,
    termNodes, maxVarTerm]
    nlinarith [ih1', ih2', termNodes_pos t1, termNodes_pos t2]

private theorem wireSizeRelation_mul_bound (r : Relation) :
  wireSizeRelation r ≤ (relationNodes r + 1) * (maxVarRelation r + 4) + 3 *
  relationNodes r +
  wireEndSlack + 3 := by
  nlinarith [wireSizeRelation_rec_bound r]

private theorem wireSizeRelation_mul_bound_le_nodeBound (r : Relation) :
  (relationNodes r + 1) * (maxVarRelation r + 4) + 3 * relationNodes r +
  wireEndSlack + 3 ≤
  nodeBound (relationMass r) := by
  have h1 : (relationNodes r + 1) * (maxVarRelation r + 4) + 3 * relationNodes r
  + wireEndSlack + 3 ≤
    (relationMass r + 30) ^ 2 := by
    simp [relationMass, wireEndSlack, relationNodes, maxVarRelation]
    nlinarith [sq_nonneg (relationNodes r : ), sq_nonneg (maxVarRelation r : ),
    relationMass_pos r]
  exact Nat.le_trans h1 (sqMass_le_nodeBound (relationMass r))

private theorem wireSizeRelation_le_nodeBound_aux (r : Relation) :
  wireSizeRelation r ≤ nodeBound (relationMass r) :=
  Nat.le_trans (wireSizeRelation_mul_bound r) (
  wireSizeRelation_mul_bound_le_nodeBound r)

private theorem wireSizeFormula_rec_bound (f : Formula) :
  wireSizeFormula f ≤ (formulaNodes f + 1) * (maxVarFormula f + 4) + 3 *
  formulaNodes f := by
  induction f with
  | rel r =>
    have hr := wireSizeRelation_rec_bound r
    simp [wireSizeFormula, formulaNodes, maxVarFormula]
    nlinarith [hr, relationNodes_pos r]
  | not f ih =>
    simp [wireSizeFormula, formulaNodes, maxVarFormula]
    nlinarith [ih, formulaNodes_pos f]
  | and f1 f2 ih1 ih2 | or f1 f2 ih1 ih2 | imp f1 f2 ih1 ih2 | iff f1 f2 ih1 ih2
  =>
    have hM1 : maxVarFormula f1 ≤ max (maxVarFormula f1) (maxVarFormula f2) :=
    Nat.le_max_left _ _
    have hM2 : maxVarFormula f2 ≤ max (maxVarFormula f1) (maxVarFormula f2) :=
    Nat.le_max_right _ _
    have ih1' : wireSizeFormula f1 ≤
      (formulaNodes f1 + 1) * (max (maxVarFormula f1) (maxVarFormula f2) + 4)
    + 3 * formulaNodes f1 := by
      nlinarith [ih1, hM1]
    have ih2' : wireSizeFormula f2 ≤
      (formulaNodes f2 + 1) * (max (maxVarFormula f1) (maxVarFormula f2) + 4)
    + 3 * formulaNodes f2 := by
      nlinarith [ih2, hM2]
    simp [wireSizeFormula, formulaNodes, maxVarFormula]
    nlinarith [ih1', ih2', formulaNodes_pos f1, formulaNodes_pos f2]

```

```

private theorem wireSizeFormula_mul_bound (f : Formula) :
  wireSizeFormula f ≤ (formulaNodes f + 1) * (maxVarFormula f + 4) + 3 *
  formulaNodes f +
  wireEndSlack + 3 := by
  nlinarith [wireSizeFormula_rec_bound f]

private theorem wireSizeFormula_mul_bound_le_nodeBound (f : Formula) :
  (formulaNodes f + 1) * (maxVarFormula f + 4) + 3 * formulaNodes f +
  wireEndSlack + 3 ≤
  nodeBound (formulaMass f) := by
  have h1 : (formulaNodes f + 1) * (maxVarFormula f + 4) + 3 * formulaNodes f +
  wireEndSlack + 3 ≤
  (formulaMass f + 30) ^ 2 := by
  simp [formulaMass, wireEndSlack, formulaNodes, maxVarFormula]
  nlinarith [sq_nonneg (formulaNodes f : ), sq_nonneg (maxVarFormula f : ),
  formulaMass_pos f]
  exact Nat.le_trans h1 (sqMass_le_nodeBound (formulaMass f))

private theorem wireSizeFormula_le_nodeBound_aux (f : Formula) :
  wireSizeFormula f ≤ nodeBound (formulaMass f) :=
  Nat.le_trans (wireSizeFormula_mul_bound f) (
  wireSizeFormula_mul_bound_le_nodeBound f)

theorem formulaMass_eq_astMass (f : Formula) : formulaMass f = formulaAstMass f
:= by
  rfl

theorem wireSizeTerm_le_nodeBound (t : Term) :
  wireSizeTerm t ≤ nodeBound (termMass t) :=
  wireSizeTerm_le_nodeBound_aux t

theorem wireSizeRelation_le_nodeBound (r : Relation) :
  wireSizeRelation r ≤ nodeBound (relationMass r) :=
  wireSizeRelation_le_nodeBound_aux r

theorem formulaSize_le_nodeBound (f : Formula) :
  formulaSize f ≤ nodeBound (formulaMass f) :=
  wireSizeFormula_le_nodeBound_aux f

theorem formulaNodes_le_astMass (f : Formula) : formulaNodes f ≤ formulaAstMass
f := by
  simp [formulaAstMass, Nat.le_add_right]

/-- Polynomial upper bound on encoded length from syntax mass. -/
def encodingBound (n : Nat) : Nat :=
  nodeBound n + 2

theorem encodingBound_mono {m n : Nat} (h : m ≤ n) :
  encodingBound m ≤ encodingBound n := by
  unfold encodingBound
  exact Nat.add_le_add_right (nodeBound_mono h) 2

theorem formulaSize_le_mass (f : Formula) :
  formulaSize f ≤ encodingBound (formulaAstMass f) := by
  have h := formulaSize_le_nodeBound f

```

```

simp [formulaMass_eq_astMass, encodingBound] using Nat.le_trans h (Nat.
  le_add_right _ 2)

theorem encodingBound_poly : IsPolynomial encodingBound := by
  refine ⟨6000000000000, 2, fun n => ?_⟩
  simp only [encodingBound, nodeBound]
  have h : (n + 2) ^ 2 ≤ 5 * n ^ 2 + 5 := by
    cases n with
    | zero => decide
    | succ n => nlinarith
  nlinarith

theorem formulaSize_le_encodingBound (f : Formula) :
  formulaSize f ≤ encodingBound (formulaAstMass f) :=
  formulaSize_le_mass f

theorem formulaSize_le_polyMass (f : Formula) (n : Nat) (h : formulaAstMass f ≤
  n) :
  formulaSize f ≤ encodingBound n :=
  Nat.le_trans (formulaSize_le_encodingBound f) (encodingBound_mono h)

/-! ### Phase 3A  deserialization (inverse of ['serializeFormula']) -/

def stripPrefix? (xs expected : Bitstring) : Option Bitstring :=
  if h : xs.length ≥ expected.length ∧ xs.take expected.length = expected then
    some (xs.drop expected.length)
  else
    none

def decodeNat? : Bitstring → Option (Nat × Bitstring)
| [] => none
| false :: rest => some (0, rest)
| true :: rest =>
  match decodeNat? rest with
  | none => none
  | some (n, rest') => some (n + 1, rest')

mutual
def decodeTermFuel (fuel : Nat) (bits : Bitstring) : Option (Term × Bitstring)
:=
  match fuel with
  | 0 => none
  | fuel + 1 =>
    if bits.length < 3 then
      none
    else if bits.take 3 = [false, false, false] then
      match decodeNat? (bits.drop 3) with
      | some (n, rest) => some (.var n, rest)
      | none => none
    else if bits.take 3 = [false, false, true] then
      some (.empty, bits.drop 3)
    else if bits.take 3 = [false, true, false] then
      match decodeTermFuel fuel (bits.drop 3) with
      | some (t1, rest) =>
        match decodeTermFuel fuel rest with
        | some (t2, rest') => some (.union t1 t2, rest')

```

```

    | none => none
  | none => none
else if bits.take 3 = [false, true, true] then
  match decodeTermFuel fuel (bits.drop 3) with
  | some (t1, rest) =>
    match decodeTermFuel fuel rest with
    | some (t2, rest') => some (.inter t1 t2, rest')
    | none => none
  | none => none
else if bits.take 3 = [true, false, false] then
  match decodeTermFuel fuel (bits.drop 3) with
  | some (t1, rest) =>
    match decodeTermFuel fuel rest with
    | some (t2, rest') => some (.diff t1 t2, rest')
    | none => none
  | none => none
else
  none

def decodeRelationFuel (fuel : Nat) (bits : Bitstring) : Option (Relation ×
  Bitstring) :=
  match fuel with
  | 0 => none
  | fuel + 1 =>
    if bits.length < 2 then
      none
    else if bits.take 2 = [false, false] then
      match decodeTermFuel fuel (bits.drop 2) with
      | some (t1, rest) =>
        match decodeTermFuel fuel rest with
        | some (t2, rest') => some (.mem t1 t2, rest')
        | none => none
      | none => none
    else if bits.take 2 = [false, true] then
      match decodeTermFuel fuel (bits.drop 2) with
      | some (t1, rest) =>
        match decodeTermFuel fuel rest with
        | some (t2, rest') => some (.not_mem t1 t2, rest')
        | none => none
      | none => none
    else if bits.take 2 = [true, false] then
      match decodeTermFuel fuel (bits.drop 2) with
      | some (t1, rest) =>
        match decodeTermFuel fuel rest with
        | some (t2, rest') => some (.eq t1 t2, rest')
        | none => none
      | none => none
    else if bits.take 2 = [true, true] then
      match decodeTermFuel fuel (bits.drop 2) with
      | some (t1, rest) =>
        match decodeTermFuel fuel rest with
        | some (t2, rest') => some (.neq t1 t2, rest')
        | none => none
      | none => none
    else
      none

```

```

def decodeFormulaFuel (fuel : Nat) (bits : Bitstring) : Option (Formula ×
  Bitstring) :=
  match fuel with
  | 0 => none
  | fuel + 1 =>
    if bits.length < 3 then
      none
    else if bits.take 3 = [false, false, false] then
      match decodeRelationFuel fuel (bits.drop 3) with
      | some (r, rest) => some (.rel r, rest)
      | none => none
    else if bits.take 3 = [false, false, true] then
      match decodeFormulaFuel fuel (bits.drop 3) with
      | some (f, rest) => some (.not f, rest)
      | none => none
    else if bits.take 3 = [false, true, false] then
      match decodeFormulaFuel fuel (bits.drop 3) with
      | some (f1, rest) =>
        match decodeFormulaFuel fuel rest with
        | some (f2, rest') => some (.and f1 f2, rest')
        | none => none
      | none => none
    else if bits.take 3 = [false, true, true] then
      match decodeFormulaFuel fuel (bits.drop 3) with
      | some (f1, rest) =>
        match decodeFormulaFuel fuel rest with
        | some (f2, rest') => some (.or f1 f2, rest')
        | none => none
      | none => none
    else if bits.take 3 = [true, false, false] then
      match decodeFormulaFuel fuel (bits.drop 3) with
      | some (f1, rest) =>
        match decodeFormulaFuel fuel rest with
        | some (f2, rest') => some (.imp f1 f2, rest')
        | none => none
      | none => none
    else if bits.take 3 = [true, false, true] then
      match decodeFormulaFuel fuel (bits.drop 3) with
      | some (f1, rest) =>
        match decodeFormulaFuel fuel rest with
        | some (f2, rest') => some (.iff f1 f2, rest')
        | none => none
      | none => none
    else
      none
  termination_by fuel
  decreasing_by
    all_goals simp_wf <;> split <;> omega

def decodeTerm? (bits : Bitstring) : Option (Term × Bitstring) :=
  decodeTermFuel bits.length bits

def decodeRelation? (bits : Bitstring) : Option (Relation × Bitstring) :=
  decodeRelationFuel bits.length bits

```

```

def decodeFormula? (bits : Bitstring) : Option (Formula × Bitstring) :=
  decodeFormulaFuel bits.length bits

end

theorem decodeNat?_encodeNat (n : Nat) : decodeNat? (encodeNat n) = some (n, [])
  := by
  induction n with
  | zero => simp [decodeNat?, encodeNat]
  | succ n ih => simp [decodeNat?, encodeNat, ih]

theorem decodeNat?_suffix (n : Nat) (rest : Bitstring) :
  decodeNat? (encodeNat n ++ rest) = some (n, rest) := by
  induction n with
  | zero => simp [decodeNat?, encodeNat]
  | succ n ih => simp [decodeNat?, encodeNat, ih, List.append_assoc]

private theorem take_prefix3 (p xs rest : Bitstring) (h : len p = 3) :
  List.take 3 (p ++ xs ++ rest) = p := by
  rw [len_eq] at h
  rw [List.append_assoc, List.take_append, h, Nat.sub_self, List.take_zero, List
    .append_nil]
  rw [← h, List.take_length]

private theorem take_prefix2 (p xs rest : Bitstring) (h : len p = 2) :
  List.take 2 (p ++ xs ++ rest) = p := by
  rw [len_eq] at h
  rw [List.append_assoc, List.take_append, h, Nat.sub_self, List.take_zero, List
    .append_nil]
  rw [← h, List.take_length]

@[simp] theorem take_termTag3 (t : Term) (mid rest : Bitstring) :
  List.take 3 (termTag t ++ mid ++ rest) = termTag t :=
  take_prefix3 (termTag t) mid rest (len_termTag t)

@[simp] theorem take_relationTag2 (r : Relation) (mid rest : Bitstring) :
  List.take 2 (relationTag r ++ mid ++ rest) = relationTag r :=
  take_prefix2 (relationTag r) mid rest (len_relationTag r)

@[simp] theorem take_formulaTag3 (f : Formula) (mid rest : Bitstring) :
  List.take 3 (formulaTag f ++ mid ++ rest) = formulaTag f :=
  take_prefix3 (formulaTag f) mid rest (len_formulaTag f)

private theorem take_serializeTerm_prefix3 (t : Term) (rest : Bitstring) :
  List.take 3 (serializeTerm t ++ rest) = termTag t := by
  cases t with
  | var n => simp [serializeTerm, termTag, take_prefix3, len_termTag, List
    .append_assoc]
  | empty => simp [serializeTerm, termTag, take_prefix3, len_termTag]
  | union t1 t2 => simp [serializeTerm, termTag, take_prefix3, len_termTag, List
    .append_assoc]
  | inter t1 t2 => simp [serializeTerm, termTag, take_prefix3, len_termTag, List
    .append_assoc]
  | diff t1 t2 => simp [serializeTerm, termTag, take_prefix3, len_termTag, List
    .append_assoc]

```



```

private theorem take_serializeRelation_prefix2 (r : Relation) (rest : Bitstring)
  :
  List.take 2 (serializeRelation r ++ rest) = relationTag r := by
cases r with
| mem t1 t2 | not_mem t1 t2 | eq t1 t2 | neq t1 t2 =>
  simp [serializeRelation, relationTag, take_prefix2, len_relationTag, List.
  append_assoc]

private theorem take_serializeFormula_prefix3 (f : Formula) (rest : Bitstring) :
  List.take 3 (serializeFormula f ++ rest) = formulaTag f := by
cases f with
| rel r => simp [serializeFormula, formulaTag, take_prefix3, len_formulaTag,
  List.append_assoc]
| not f => simp [serializeFormula, formulaTag, take_prefix3, len_formulaTag,
  List.append_assoc]
| and f1 f2 | or f1 f2 | imp f1 f2 | iff f1 f2 =>
  simp [serializeFormula, formulaTag, take_prefix3, len_formulaTag, List.
  append_assoc]

private theorem drop3_termTag_append (tag mid rest : Bitstring) (h : len tag =
  3) :
  (tag ++ mid ++ rest).drop 3 = mid ++ rest := by
rw [len_eq] at h
simp [List.drop_append, h, List.append_assoc]

private theorem drop3_serializeTerm (t : Term) (rest : Bitstring) :
  (serializeTerm t ++ rest).drop 3 =
  match t with
  | .var n => encodeNat n ++ rest
  | .empty => rest
  | .union t1 t2 => serializeTerm t1 ++ serializeTerm t2 ++ rest
  | .inter t1 t2 => serializeTerm t1 ++ serializeTerm t2 ++ rest
  | .diff t1 t2 => serializeTerm t1 ++ serializeTerm t2 ++ rest := by
cases t <|> simp [serializeTerm, termTag, drop3_termTag_append, len_termTag,
  List.append_assoc]

private theorem drop2_relationTag_append (tag mid rest : Bitstring) (h : len tag
  = 2) :
  (tag ++ mid ++ rest).drop 2 = mid ++ rest := by
rw [len_eq] at h
simp [List.drop_append, h, List.append_assoc]

private theorem drop2_serializeRelation (r : Relation) (rest : Bitstring) :
  (serializeRelation r ++ rest).drop 2 =
  match r with
  | .mem t1 t2 => serializeTerm t1 ++ serializeTerm t2 ++ rest
  | .not_mem t1 t2 => serializeTerm t1 ++ serializeTerm t2 ++ rest
  | .eq t1 t2 => serializeTerm t1 ++ serializeTerm t2 ++ rest
  | .neq t1 t2 => serializeTerm t1 ++ serializeTerm t2 ++ rest := by
cases r <|> simp [serializeRelation, relationTag, drop2_relationTag_append,
  len_relationTag,
  List.append_assoc]

private theorem drop3_formulaTag_append (tag mid rest : Bitstring) (h : len tag
  = 3) :
  (tag ++ mid ++ rest).drop 3 = mid ++ rest := by

```

```

rw [len_eq] at h
simp [List.drop_append, h, List.append_assoc]

private theorem drop3_serializeFormula (f : Formula) (rest : Bitstring) :
  (serializeFormula f ++ rest).drop 3 =
    match f with
    | .rel r => serializeRelation r ++ rest
    | .not f => serializeFormula f ++ rest
    | .and f1 f2 => serializeFormula f1 ++ serializeFormula f2 ++ rest
    | .or f1 f2 => serializeFormula f1 ++ serializeFormula f2 ++ rest
    | .imp f1 f2 => serializeFormula f1 ++ serializeFormula f2 ++ rest
    | .iff f1 f2 => serializeFormula f1 ++ serializeFormula f2 ++ rest := by
cases f <;> simp [serializeFormula, formulaTag, drop3_formulaTag_append,
  len_formulaTag,
  List.append_assoc]

theorem decodeTermFuel_suffix (fuel : Nat) (t : Term) (rest : Bitstring)
  (h : wireSizeTerm t + rest.length ≤ fuel) :
  decodeTermFuel fuel (serializeTerm t ++ rest) = some (t, rest) := by
induction t generalizing rest fuel with
| var n =>
  have hlen : 3 ≤ (serializeTerm (.var n) ++ rest).length := by
    simp [wireSizeTerm, length_serializeTerm, wireSizeNat, length_encodeNat]
  omega
  cases fuel with
  | zero => simp [wireSizeTerm] at h
  | succ fuel =>
    simp [decodeTermFuel, serializeTerm, termTag, drop3_serializeTerm, hlen]
    rw [decodeNat?_suffix n rest]
| empty =>
  have hlen : 3 ≤ (serializeTerm .empty ++ rest).length := by
    simp [wireSizeTerm, length_serializeTerm]
  cases fuel with
  | zero => simp [wireSizeTerm] at h
  | succ fuel =>
    simp [decodeTermFuel, serializeTerm, termTag, drop3_serializeTerm, hlen]
| union t1 t2 ih1 ih2 =>
  have hlen : 3 ≤ (serializeTerm (.union t1 t2) ++ rest).length := by
    simp [wireSizeTerm, length_serializeTerm]
  have := wireSizeTerm_pos t1
  omega
  cases fuel with
  | zero => simp [wireSizeTerm] at h
  | succ fuel =>
    have hf1 : wireSizeTerm t1 + (serializeTerm t2 ++ rest).length ≤ fuel :=
    by
      simp [wireSizeTerm, length_serializeTerm, List.length_append] at h ;
    omega
    have hf2 : wireSizeTerm t2 + rest.length ≤ fuel := by
      have h1 := wireSizeTerm_pos t1
      simp [wireSizeTerm, length_serializeTerm, List.length_append] at h ;
    omega
    simp [decodeTermFuel, serializeTerm, termTag, drop3_serializeTerm, hlen]
    rw [ih1 fuel (serializeTerm t2 ++ rest) hf1]
    simp
    rw [ih2 fuel rest hf2]

```

```

| inter t1 t2 ih1 ih2 =>
  have hlen : 3 ≤ (serializeTerm (.inter t1 t2) ++ rest).length := by
    simp [wireSizeTerm, length_serializeTerm]
    have := wireSizeTerm_pos t1
    omega
  cases fuel with
  | zero => simp [wireSizeTerm] at h
  | succ fuel =>
    have hf1 : wireSizeTerm t1 + (serializeTerm t2 ++ rest).length ≤ fuel :=
    by
      simp [wireSizeTerm, length_serializeTerm, List.length_append] at h ;
    omega
    have hf2 : wireSizeTerm t2 + rest.length ≤ fuel := by
      have h1 := wireSizeTerm_pos t1
      simp [wireSizeTerm, length_serializeTerm, List.length_append] at h ;
    omega
    simp [decodeTermFuel, serializeTerm, termTag, drop3_serializeTerm, hlen]
    rw [ih1 fuel (serializeTerm t2 ++ rest) hf1]
    simp
    rw [ih2 fuel rest hf2]
| diff t1 t2 ih1 ih2 =>
  have hlen : 3 ≤ (serializeTerm (.diff t1 t2) ++ rest).length := by
    simp [wireSizeTerm, length_serializeTerm]
    have := wireSizeTerm_pos t1
    omega
  cases fuel with
  | zero => simp [wireSizeTerm] at h
  | succ fuel =>
    have hf1 : wireSizeTerm t1 + (serializeTerm t2 ++ rest).length ≤ fuel :=
    by
      simp [wireSizeTerm, length_serializeTerm, List.length_append] at h ;
    omega
    have hf2 : wireSizeTerm t2 + rest.length ≤ fuel := by
      have h1 := wireSizeTerm_pos t1
      simp [wireSizeTerm, length_serializeTerm, List.length_append] at h ;
    omega
    simp [decodeTermFuel, serializeTerm, termTag, drop3_serializeTerm, hlen]
    rw [ih1 fuel (serializeTerm t2 ++ rest) hf1]
    simp
    rw [ih2 fuel rest hf2]

theorem decodeTerm?_suffix (t : Term) (rest : Bitstring) :
  decodeTerm? (serializeTerm t ++ rest) = some (t, rest) := by
  simp [decodeTerm?, length_serializeTerm]
  exact decodeTermFuel_suffix (wireSizeTerm t + rest.length) t rest le_refl

theorem decodeTerm?_serializeTerm (t : Term) :
  decodeTerm? (serializeTerm t) = some (t, []) := by
  simp [List.append_nil] using decodeTerm?_suffix t []

theorem decodeRelationFuel_suffix (fuel : Nat) (r : Relation) (rest : Bitstring)
  (h : wireSizeRelation r + rest.length ≤ fuel) :
  decodeRelationFuel fuel (serializeRelation r ++ rest) = some (r, rest) := by
  cases r with
  | mem t1 t2 =>
    have hlen : 2 ≤ (serializeRelation (.mem t1 t2) ++ rest).length := by

```

```

    simp [wireSizeRelation, length_serializeRelation, wireSizeTerm,
wireSizeNat]
    have := wireSizeTerm_pos t1; omega
cases fuel with
| zero => simp [wireSizeRelation] at h
| succ fuel =>
    have hf1 : wireSizeTerm t1 + (serializeTerm t2 ++ rest).length ≤ fuel :=
by
    simp [wireSizeRelation, length_serializeTerm, List.length_append] at h ;
    omega
    have hf2 : wireSizeTerm t2 + rest.length ≤ fuel := by
        have h1 := wireSizeTerm_pos t1
        simp [wireSizeRelation, length_serializeTerm, List.length_append] at h ;
        omega
    simp [decodeRelationFuel, serializeRelation, relationTag,
drop2_serializeRelation, hlen]
    rw [decodeTermFuel_suffix fuel t1 (serializeTerm t2 ++ rest) hf1]
    simp
    rw [decodeTermFuel_suffix fuel t2 rest hf2]
| not_mem t1 t2 =>
    have hlen : 2 ≤ (serializeRelation (.not_mem t1 t2) ++ rest).length := by
        simp [wireSizeRelation, length_serializeRelation, wireSizeTerm,
wireSizeNat]
        have := wireSizeTerm_pos t1; omega
cases fuel with
| zero => simp [wireSizeRelation] at h
| succ fuel =>
    have hf1 : wireSizeTerm t1 + (serializeTerm t2 ++ rest).length ≤ fuel :=
by
    simp [wireSizeRelation, length_serializeTerm, List.length_append] at h ;
    omega
    have hf2 : wireSizeTerm t2 + rest.length ≤ fuel := by
        have h1 := wireSizeTerm_pos t1
        simp [wireSizeRelation, length_serializeTerm, List.length_append] at h ;
        omega
    simp [decodeRelationFuel, serializeRelation, relationTag,
drop2_serializeRelation, hlen]
    rw [decodeTermFuel_suffix fuel t1 (serializeTerm t2 ++ rest) hf1]
    simp
    rw [decodeTermFuel_suffix fuel t2 rest hf2]
| eq t1 t2 =>
    have hlen : 2 ≤ (serializeRelation (.eq t1 t2) ++ rest).length := by
        simp [wireSizeRelation, length_serializeRelation, wireSizeTerm,
wireSizeNat]
        have := wireSizeTerm_pos t1; omega
cases fuel with
| zero => simp [wireSizeRelation] at h
| succ fuel =>
    have hf1 : wireSizeTerm t1 + (serializeTerm t2 ++ rest).length ≤ fuel :=
by
    simp [wireSizeRelation, length_serializeTerm, List.length_append] at h ;
    omega
    have hf2 : wireSizeTerm t2 + rest.length ≤ fuel := by
        have h1 := wireSizeTerm_pos t1
        simp [wireSizeRelation, length_serializeTerm, List.length_append] at h ;
        omega

```

```

    simp [decodeRelationFuel, serializeRelation, relationTag,
drop2_serializeRelation, hlen]
    rw [decodeTermFuel_suffix fuel t1 (serializeTerm t2 ++ rest) hf1]
    simp
    rw [decodeTermFuel_suffix fuel t2 rest hf2]
| neq t1 t2 =>
  have hlen :  $2 \leq (\text{serializeRelation } (.neq \ t1 \ t2) ++ \text{rest}).\text{length}$  := by
    simp [wireSizeRelation, length_serializeRelation, wireSizeTerm,
wireSizeNat]
    have := wireSizeTerm_pos t1; omega
  cases fuel with
| zero => simp [wireSizeRelation] at h
| succ fuel =>
  have hf1 :  $\text{wireSizeTerm } t1 + (\text{serializeTerm } t2 ++ \text{rest}).\text{length} \leq \text{fuel}$  :=
  by
    simp [wireSizeRelation, length_serializeTerm, List.length_append] at h ;
    omega
    have hf2 :  $\text{wireSizeTerm } t2 + \text{rest.length} \leq \text{fuel}$  := by
      have h1 := wireSizeTerm_pos t1
      simp [wireSizeRelation, length_serializeTerm, List.length_append] at h ;
      omega
    simp [decodeRelationFuel, serializeRelation, relationTag,
drop2_serializeRelation, hlen]
    rw [decodeTermFuel_suffix fuel t1 (serializeTerm t2 ++ rest) hf1]
    simp
    rw [decodeTermFuel_suffix fuel t2 rest hf2]

theorem decodeRelation?_suffix (r : Relation) (rest : Bitstring) :
  decodeRelation? (serializeRelation r ++ rest) = some (r, rest) := by
  simp [decodeRelation?, length_serializeRelation]
  exact decodeRelationFuel_suffix (wireSizeRelation r + rest.length) r rest
  le_rfl

theorem decodeRelation?_serializeRelation (r : Relation) :
  decodeRelation? (serializeRelation r) = some (r, []) := by
  simp [List.append_nil] using decodeRelation?_suffix r []

theorem decodeFormulaFuel_suffix (fuel : Nat) (f : Formula) (rest : Bitstring)
  (h :  $\text{wireSizeFormula } f + \text{rest.length} \leq \text{fuel}$ ) :
  decodeFormulaFuel fuel (serializeFormula f ++ rest) = some (f, rest) := by
  induction f generalizing rest fuel with
| rel r =>
  have hlen :  $3 \leq (\text{serializeFormula } (.rel \ r) ++ \text{rest}).\text{length}$  := by
    simp [wireSizeFormula, length_serializeFormula, wireSizeRelation]
  cases r with
| mem t1 _ | not_mem t1 _ | eq t1 _ | neq t1 _ =>
  have := wireSizeTerm_pos t1; omega
  cases fuel with
| zero => simp [wireSizeFormula, wireSizeRelation] at h
| succ fuel =>
  have hrel :  $\text{wireSizeRelation } r + \text{rest.length} \leq \text{fuel}$  := by
    simp [wireSizeFormula, wireSizeRelation] at h ; omega
  simp [decodeFormulaFuel, serializeFormula, formulaTag,
drop3_serializeFormula, hlen]
  rw [decodeRelationFuel_suffix fuel r rest hrel]
| not f ih =>

```

```

have hlen : 3 ≤ (serializeFormula (.not f) ++ rest).length := by
  simp [wireSizeFormula, length_serializeFormula]
  have := formulaSize_pos f; omega
cases fuel with
| zero => simp [wireSizeFormula] at h
| succ fuel =>
  simp [decodeFormulaFuel, serializeFormula, formulaTag,
drop3_serializeFormula, hlen]
  rw [ih fuel rest (by simp [wireSizeFormula] at h ; omega)]
| and f1 f2 ih1 ih2 =>
have hlen : 3 ≤ (serializeFormula (f1.and f2) ++ rest).length := by
  simp [wireSizeFormula, length_serializeFormula]
  have := formulaSize_pos f1; omega
cases fuel with
| zero => simp [wireSizeFormula] at h
| succ fuel =>
  have hf1 : wireSizeFormula f1 + (serializeFormula f2 ++ rest).length ≤
fuel := by
  simp [wireSizeFormula, length_serializeFormula, List.length_append] at h
  ; omega
  have hf2 : wireSizeFormula f2 + rest.length ≤ fuel := by
    have h1 := formulaSize_pos f1
    simp [wireSizeFormula, length_serializeFormula, List.length_append] at h
    ; omega
  simp [decodeFormulaFuel, serializeFormula, formulaTag,
drop3_serializeFormula, hlen]
  rw [ih1 fuel (serializeFormula f2 ++ rest) hf1]
  simp
  rw [ih2 fuel rest hf2]
| or f1 f2 ih1 ih2 =>
have hlen : 3 ≤ (serializeFormula (f1.or f2) ++ rest).length := by
  simp [wireSizeFormula, length_serializeFormula]
  have := formulaSize_pos f1; omega
cases fuel with
| zero => simp [wireSizeFormula] at h
| succ fuel =>
  have hf1 : wireSizeFormula f1 + (serializeFormula f2 ++ rest).length ≤
fuel := by
  simp [wireSizeFormula, length_serializeFormula, List.length_append] at h
  ; omega
  have hf2 : wireSizeFormula f2 + rest.length ≤ fuel := by
    have h1 := formulaSize_pos f1
    simp [wireSizeFormula, length_serializeFormula, List.length_append] at h
    ; omega
  simp [decodeFormulaFuel, serializeFormula, formulaTag,
drop3_serializeFormula, hlen]
  rw [ih1 fuel (serializeFormula f2 ++ rest) hf1]
  simp
  rw [ih2 fuel rest hf2]
| imp f1 f2 ih1 ih2 =>
have hlen : 3 ≤ (serializeFormula (f1.imp f2) ++ rest).length := by
  simp [wireSizeFormula, length_serializeFormula]
  have := formulaSize_pos f1; omega
cases fuel with
| zero => simp [wireSizeFormula] at h
| succ fuel =>

```

```

      have hf1 : wireSizeFormula f1 + (serializeFormula f2 ++ rest).length ≤
fuel := by
      simp [wireSizeFormula, length_serializeFormula, List.length_append] at h
    ; omega
      have hf2 : wireSizeFormula f2 + rest.length ≤ fuel := by
      have h1 := formulaSize_pos f1
      simp [wireSizeFormula, length_serializeFormula, List.length_append] at h
    ; omega
      simp [decodeFormulaFuel, serializeFormula, formulaTag,
drop3_serializeFormula, hlen]
      rw [ih1 fuel (serializeFormula f2 ++ rest) hf1]
      simp
      rw [ih2 fuel rest hf2]
| iff f1 f2 ih1 ih2 =>
have hlen : 3 ≤ (serializeFormula (f1.iff f2) ++ rest).length := by
  simp [wireSizeFormula, length_serializeFormula]
  have := formulaSize_pos f1; omega
cases fuel with
| zero => simp [wireSizeFormula] at h
| succ fuel =>
  have hf1 : wireSizeFormula f1 + (serializeFormula f2 ++ rest).length ≤
fuel := by
    simp [wireSizeFormula, length_serializeFormula, List.length_append] at h
  ; omega
    have hf2 : wireSizeFormula f2 + rest.length ≤ fuel := by
    have h1 := formulaSize_pos f1
    simp [wireSizeFormula, length_serializeFormula, List.length_append] at h
  ; omega
    simp [decodeFormulaFuel, serializeFormula, formulaTag,
drop3_serializeFormula, hlen]
    rw [ih1 fuel (serializeFormula f2 ++ rest) hf1]
    simp
    rw [ih2 fuel rest hf2]

theorem decodeFormula?_suffix (f : Formula) (rest : Bitstring) :
  decodeFormula? (serializeFormula f ++ rest) = some (f, rest) := by
  simp [decodeFormula?, length_serializeFormula]
  exact decodeFormulaFuel_suffix (wireSizeFormula f + rest.length) f rest le_rfl

theorem decodeFormula?_serializeFormula (f : Formula) :
  decodeFormula? (serializeFormula f) = some (f, []) := by
  simp [List.append_nil] using decodeFormula?_suffix f []

end MLS

```

0.8.3 AvgCaseMls/NPMembership.lean

AvgCaseMls/NPMembership.lean

Lean 4 Certificate

```

/-
Copyright (c) 2026 Catskills Research Company. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Lars Warren Ericson, Catskills Research Company

```

```

-/

import AvgCaseMls.AverageHardness
import AvgCaseMls.Serialization
import AvgCaseMls.DecideMLS

/-!
Phase **3A:** certificate-based NP membership for MLS satisfiability.

The semantic language ['SatMLS'] uses noncomputable ['evalFormula']. The NP-
verifiable
proxy ['SatMLSChecker'] decodes an input and runs ['decideMLSSat']; see
['DEFINITION_FORKS.md'] (../DEFINITION_FORKS.md).
-/

namespace MLS

open AvCom

/-- Poly-time verifiable MLS satisfiability on well-formed encodings. -/
def SatMLSChecker : Set Bitstring :=
  { s |
    match decodeFormula? s with
    | none => False
    | some (f, rest) => rest = [] ^ decideMLSSat f = true }

/--
NP verifier: decode the formula from 'x' and run ['decideMLSSat']. The
certificate is
unused (length bound '0'); see Phase **3A** fork in ['DEFINITION_FORKS.md'] (../
DEFINITION_FORKS.md).
-/
def verifySatMLS (x _cert : Bitstring) : Bool :=
  match decodeFormula? x with
  | none => false
  | some (f, rest) => decide (rest = [] && decideMLSSat f)

def satCertBound (_n : Nat) : Nat := 0

theorem satCertBound_poly : IsPolynomial satCertBound :=
  IsPolynomial.const 0

theorem verifySatMLS_true_iff (x : Bitstring) :
  verifySatMLS x [] = true ^ x ∈ SatMLSChecker := by
  simp [verifySatMLS, SatMLSChecker, Bool.and_eq_true, decide_eq_true_iff]
  split <;> simp [Bool.and_eq_true, decide_eq_true_iff]

theorem SatMLSChecker_in_NP : InNP SatMLSChecker :=
  InNP.intro satCertBound_poly fun x => by
  constructor
  û intro hx
  refine ⟨[], by simp [satCertBound], ?_⟩
  exact (verifySatMLS_true_iff x).mpr hx
  û intro ⟨cert, hlen, hver⟩
  have hc : cert = [] := by simpa [satCertBound] using hlen
  subst hc

```



```

    exact (verifySatMLS_true_iff x).mp hver

theorem SatMLSChecker_subset_SatMLS (s : Bitstring) (h : s ∈ SatMLSChecker)
  {f : Formula} (hf : serializeFormula f = s) (hfrag : InDecideSoundFormula f)
  :
  s ∈ SatMLS := by
  have hdec : decodeFormula? s = some (f, []) := by
    rw [← hf, decodeFormula?_serializeFormula]
  simp [SatMLSChecker, hdec] at h
  obtain ⟨env, he⟩ := decideMLSSat_sound f h hfrag
  exact ⟨f, hf, env, he⟩

end MLS

```

0.8.4 Phase 4 — NBH, reduction, and completeness

[AvgCaseMls/NBH.lean](#) formalizes bounded halting (NBH), the rankable distribution μ , and distNP membership. [AvgCaseMls/Reduction.lean](#) constructs the domination-preserving reduction into SatMLS. [AvgCaseMls/Completeness.lean](#) proves NP-average completeness of SatMLS.

0.8.5 AvgCaseMls/NBH.lean

[AvgCaseMls/NBH.lean](#)

Lean 4 Certificate

```

/-
Copyright (c) 2026 Catskills Research Company. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Lars Warren Ericson, Catskills Research Company
-/

import AvgCaseMls.AvCom
import Mathlib.Tactic

/-!
Phase **4A:** bounded halting for nondeterministic Turing machines (NBH) and a
simple
POL-rankable distribution  $\mu$  (TR1995-711 / Levin BH).

Literature:  $\mathrm{BH} = \{(M, x, 1^t) : M \text{ is an NTM accepting } x \text{ in } t \text{ steps}\}$ .

**Lean fork:** instances reference a canonical finite machine table by index;
step bound uses
['encodeNat'] rather than unary  $1^t$ . See ['DEFINITION_FORKS.md'](../
DEFINITION_FORKS.md).
-/

namespace NBH

open AvCom

/-! ### Nat codec -/

```

```

def encodeNat : Nat → Bitstring
| 0 => [false]
| n + 1 => true :: encodeNat n

def decodeNat? : Bitstring → Option (Nat × Bitstring)
| [] => none
| [false] => some (0, [])
| true :: rest =>
  match decodeNat? rest with
  | some (n, rest') => some (n + 1, rest')
  | none => none
| false :: _ :: _ => none

namespace decodeNat?

theorem encode (n : Nat) : decodeNat? (encodeNat n) = some (n, []) := by
  induction n with
  | zero => simp [encodeNat, decodeNat?]
  | succ n ih =>
    simp [encodeNat, decodeNat?]
    rw [ih]

end decodeNat?

/-! ### One-tape NTM scaffold -/

structure Trans where
  read : Bool
  write : Bool
  next : Nat
  right : Bool
  deriving Repr, DecidableEq

structure NTM where
  numStates : Nat
  hStates : 0 < numStates
  start : Nat
  accept : Nat
  hStart : start < numStates
  hAccept : accept < numStates
  trans : Nat → Bool → Option Trans
  hTrans : ∀ s b t, trans s b = some t → t.next < numStates

namespace NTM

def tapeGet (tape : Bitstring) (i : Nat) : Bool :=
  match tape[i]? with
  | some b => b
  | none => false

def tapeSet (tape : Bitstring) (i : Nat) (b : Bool) : Bitstring :=
  if hi : i < tape.length then
    tape.take i ++ [b] ++ tape.drop (i + 1)
  else
    tape ++ List.replicate (i - tape.length) false ++ [b]

```

```

def step (M : NTM) (state head : Nat) (tape : Bitstring) : Option (Nat × Nat ×
  Bitstring) :=
  match M.trans state (tapeGet tape head) with
  | none => none
  | some t =>
    if t.read = tapeGet tape head then
      let tape' := tapeSet tape head t.write
      let head' := if t.right then head + 1 else head - 1
      if t.next < M.numStates then
        some (t.next, head', tape')
      else
        none
    else
      none

end NTM

/ -! ### Canonical machines and instances -/

def trivialAcceptNTM : NTM where
  numStates := 1
  hStates := by decide
  start := 0
  accept := 0
  hStart := by decide
  hAccept := by decide
  trans := fun _ _ => none
  hTrans := by intro s b t ht; simp at ht

def canonicalMachines : List NTM := [trivialAcceptNTM]

def lookupMachine? (id : Nat) : Option NTM :=
  canonicalMachines[id]?

structure NBHInstance where
  machineId : Nat
  input : Bitstring
  bound : Nat

namespace NBHInstance

/-- Field delimiter '[false, false, true]'; never a substring of '[encodeNat]'
    outputs. -/
def delim : Bitstring := [false, false, true]

def delimFree (s : Bitstring) : Prop :=
  ¬ List.Sublist delim s

def WellFormed (inst : NBHInstance) : Prop :=
  delimFree inst.input

def splitDelim.go (pref rest : Bitstring) : Option (Bitstring × Bitstring) :=
  match rest with
  | [] => none
  | false :: false :: true :: suffix => some (pref, suffix)
  | b :: suffix => splitDelim.go (pref ++ [b]) suffix

```

```

def splitDelim (s : Bitstring) : Option (Bitstring × Bitstring) :=
  splitDelim.go [] s

namespace splitDelim

theorem go_delim_suffix (pref suffix : Bitstring) :
  splitDelim.go pref (delim ++ suffix) = some (pref, suffix) := by
  induction pref generalizing suffix with
  | nil => simp [go, delim]
  | cons b pref' _ => simp [go, delim, List.cons_append, List.nil_append]

theorem delimFree_cons_append_eq_delim (b : Bool) (pref' s t : Bitstring)
  (h : delimFree (b :: pref'))
  (heq : b :: pref' ++ (delim ++ s) = delim ++ t) : False := by
  rw [delim] at heq
  set p := b :: pref'
  by_cases hlt : p.length < 3
  û have htake : List.take 3 (p ++ false :: false :: true :: s) = [false, false,
    true] :=
    congrArg (List.take 3) heq
  revert h
  rcases b with rfl | rfl
  û cases pref' with
  | nil =>
    have hleft : List.take 3 (p ++ false :: false :: true :: s) = [false,
    false, false] := by
      simp [p, List.take, List.cons_append]
      rw [hleft] at htake; exact fun _ => by cases htake
  | cons head tail =>
    cases head with
    | false =>
      cases tail with
      | nil =>
        have hleft : List.take 3 (p ++ false :: false :: true :: s) = [false
        , false, false] := by
          simp [p, List.take, List.cons_append]
          rw [hleft] at htake; exact fun _ => by cases htake
        | cons _ tl =>
          have hlen : 3 ≤ p.length := by simp [p]
          exact fun _ => absurd hlt (Nat.not_lt.mpr hlen)
      | true =>
        cases tail with
        | nil =>
          have hleft : List.take 3 (p ++ false :: false :: true :: s) = [false
          , true, false] := by
            simp [p, List.take, List.cons_append]
            rw [hleft] at htake; exact fun _ => by cases htake
          | cons _ tl =>
            have hlen : 3 ≤ p.length := by simp [p]
            exact fun _ => absurd hlt (Nat.not_lt.mpr hlen)
        û cases pref' with
        | nil =>
          have hleft : List.take 3 (p ++ false :: false :: true :: s) = [true,
          false, false] := by
            simp [p, List.take, List.cons_append]

```

```

    rw [hleft] at htake; exact fun _ => by cases htake
  | cons head tail =>
    cases tail with
    | nil =>
      have hleft : List.take 3 (p ++ false :: false :: true :: s) = [true,
head, false] := by
        simp [p, List.take, List.cons_append]
      rw [hleft] at htake; exact fun _ => by cases htake
    | cons _ tl =>
      have hlen : 3 ≤ p.length := by simp [p]
      exact fun _ => absurd hlt (Nat.not_lt.mpr hlen)
  | have hp : 3 ≤ p.length := Nat.le_of_not_gt hlt
  have htake : p.take 3 = delim := by
    have eq := congrArg (List.take 3) heq
    simp [List.take_append, hp, delim] at eq
    exact eq
  apply h
  exact List.IsPrefix.sublist ⟨p.drop 3, by rw [← htake, List.
take_append_drop 3 p]⟩

theorem go_cons_not_delim (acc : Bitstring) (headBit : Bool) (rest : Bitstring)
  (hdel : ¬ ∃ s, headBit :: rest = false :: false :: true :: s) :
  go acc (headBit :: rest) = go (acc ++ [headBit]) rest := by
  cases rest with
  | nil => simp [go]
  | cons bf rest1 =>
    cases bf with
    | true => simp [go]
    | false =>
      cases rest1 with
      | nil => simp [go]
      | cons bf' rest2 =>
        cases bf' with
        | true =>
          by_cases hh : headBit = false
          | hh => exact absurd ⟨rest2, by simp [hh]⟩ hdel
          | hh => cases headBit <;> simp_all [go]
        | false => simp [go]

theorem go_acc_append (acc pref suffix : Bitstring) (h : delimFree pref) :
  splitDelim.go acc (pref ++ (delim ++ suffix)) = some (acc ++ pref, suffix)
:= by
  induction pref generalizing acc suffix with
  | nil => simp [go_delim_suffix, List.nil_append]
  | cons b pref' ih =>
    rw [List.cons_append]
    by_cases hdel : ∃ s, b :: (pref' ++ (delim ++ suffix)) = false :: false ::
true :: s
    | hdel => obtain ⟨s, heq⟩ := hdel
      exfalso
      apply delimFree_cons_append_eq_delim b pref' suffix s h
      rw [delim] at heq
      exact heq
    | hdel => rw [go_cons_not_delim acc b (pref' ++ (delim ++ suffix)) hdel]
      simp [List.cons_append] using ih (acc ++ [b]) suffix fun hsub => h (List.
Sublist.cons b hsub)

```

```

theorem append (pref suffix : Bitstring) (h : delimFree pref) :
  splitDelim (pref ++ (delim ++ suffix)) = some (pref, suffix) := by
  rw [splitDelim, go_acc_append [] pref suffix h, List.nil_append]

end splitDelim

def encode (inst : NBHInstance) : Bitstring :=
  inst.input ++ (delim ++ (encodeNat inst.machineId ++ (delim ++ encodeNat inst.
    bound)))

def decode? (s : Bitstring) : Option (NBHInstance × Bitstring) :=
  match splitDelim s with
  | none => none
  | some (input, rest1) =>
    match splitDelim rest1 with
    | none => none
    | some (mid, rest2) =>
      match decodeNat? mid with
      | none => none
      | some (machineId, _) =>
        match decodeNat? rest2 with
        | none => none
        | some (bound, rest3) => some ({ machineId, input, bound }, rest3)

def ntm? (inst : NBHInstance) : Option NTM :=
  lookupMachine? inst.machineId

end NBHInstance

namespace encodeNat

theorem length (n : Nat) : (encodeNat n).length = n + 1 := by
  induction n with
  | zero => rfl
  | succ n ih => simp [encodeNat, ih]

theorem count_false (n : Nat) : (encodeNat n).count false = 1 := by
  induction n with
  | zero => simp [encodeNat, List.count]
  | succ n ih => simp [encodeNat, List.count_cons, ih, Bool.false_eq_true]

theorem not_sublist_delim (n : Nat) : ¬ List.Sublist NBHInstance.delim (
  encodeNat n) := by
  intro hsub
  have hle := List.Sublist.count_le (a := false) hsub
  have hdelim : NBHInstance.delim.count false = 2 := by decide
  simp [count_false, hdelim] using hle

theorem delimFree (n : Nat) : NBHInstance.delimFree (encodeNat n) :=
  not_sublist_delim n

end encodeNat

namespace NBHInstance

```

```

theorem decode_encode (inst : NBHInstance) (h : WellFormed inst) :
  decode? (encode inst) = some (inst, []) := by
  simp only [decode?, encode]
  rw [splitDelim.append inst.input (encodeNat inst.machineId ++ (delim ++
    encodeNat inst.bound)) h]
  simp only [splitDelim.append (encodeNat inst.machineId) (encodeNat inst.bound)
    (encodeNat.delimFree inst.machineId), decodeNat?.encode]

theorem decode_encode_trivial (inst : NBHInstance) (h : inst.input = []) :
  decode? (encode inst) = some (inst, []) :=
  decode_encode inst (by
    intro hsub
    rw [h] at hsub
    exact (by simp [delim] : delim ≠ []) (List.eq_nil_of_sublist_nil hsub))

end NBHInstance

/#!/ ### Run certificates -/

structure Config where
  state : Nat
  head : Nat
  tape : Bitstring
  deriving Repr, DecidableEq

namespace Config

def delimFree (tape : Bitstring) : Prop :=
  NBHInstance.delimFree tape

def WellFormed (c : Config) : Prop :=
  delimFree c.tape

def initial (M : NTM) (input : Bitstring) : Config :=
  { state := M.start, head := 0, tape := input }

def step (M : NTM) (c : Config) : Option Config :=
  match NTM.step M c.state c.head c.tape with
  | none => none
  | some (s, h, t) => some { state := s, head := h, tape := t }

def encode (c : Config) : Bitstring :=
  encodeNat c.state ++ (NBHInstance.delim ++
    (encodeNat c.head ++ (NBHInstance.delim ++
      encodeNat c.tape.length ++ (NBHInstance.delim ++ c.tape))))

def decode? (s : Bitstring) : Option (Config × Bitstring) :=
  match NBHInstance.splitDelim s with
  | none => none
  | some (stateBits, rest1) =>
    match decodeNat? stateBits with
    | none => none
    | some (state, _) =>
      match NBHInstance.splitDelim rest1 with
      | none => none
      | some (headBits, rest2) =>

```

```

    match decodeNat? headBits with
    | none => none
    | some (head, _) =>
      match NBHInstance.splitDelim rest2 with
      | none => none
      | some (tapeLenBits, rest3) =>
        match decodeNat? tapeLenBits with
        | none => none
        | some (tapeLen, _) =>
          if tapeLen ≤ rest3.length then
            some ({ state, head, tape := rest3.take tapeLen }, rest3.drop
tapeLen)
          else
            none

theorem decode_encode (c : Config) : decode? (encode c) = some (c, []) := by
  simp only [decode?, encode]
  rw [NBHInstance.splitDelim.append (encodeNat c.state)
    (encodeNat c.head ++ (NBHInstance.delim ++ encodeNat c.tape.length ++ (
    NBHInstance.delim ++ c.tape)))
    (encodeNat.delimFree c.state)]
  simp [decodeNat?.encode]
  rw [NBHInstance.splitDelim.append (encodeNat c.head)
    (encodeNat c.tape.length ++ (NBHInstance.delim ++ c.tape)) (encodeNat.
    delimFree c.head)]
  simp [decodeNat?.encode]
  rw [NBHInstance.splitDelim.append (encodeNat c.tape.length) c.tape (encodeNat.
    delimFree c.tape.length)]
  simp [decodeNat?.encode, encodeNat.length, if_pos (Nat.le_refl c.tape.length),
    List.take_length, List.drop_length]

theorem decode?_append (c : Config) (rest : Bitstring) :
  decode? (encode c ++ rest) = some (c, rest) := by
  have henc :
    encode c ++ rest =
      encodeNat c.state ++
        (NBHInstance.delim ++
          (encodeNat c.head ++ (NBHInstance.delim ++
            encodeNat c.tape.length ++ (NBHInstance.delim ++ (c.tape ++ rest))
          ))) := by
    simp [encode, List.append_assoc]
  simp only [decode?]
  rw [henc]
  rw [NBHInstance.splitDelim.append (encodeNat c.state)
    (encodeNat c.head ++ (NBHInstance.delim ++ encodeNat c.tape.length ++ (
    NBHInstance.delim ++ (c.tape ++ rest))))
    (encodeNat.delimFree c.state)]
  simp [decodeNat?.encode]
  rw [NBHInstance.splitDelim.append (encodeNat c.head)
    (encodeNat c.tape.length ++ (NBHInstance.delim ++ (c.tape ++ rest))) (
    encodeNat.delimFree c.head)]
  simp [decodeNat?.encode]
  rw [NBHInstance.splitDelim.append (encodeNat c.tape.length) (c.tape ++ rest)
    (encodeNat.delimFree c.tape.length)]
  have hle : c.tape.length ≤ (c.tape ++ rest).length := by
    rw [List.length_append]

```



```

    exact Nat.le_add_right _ _
    simp [decodeNat?.encode, encodeNat.length, if_pos hle, List.length_take,
    List.take_append, List.drop_append]

theorem encode_length_pos (c : Config) : 0 < (encode c).length := by
  simp [encode, NBHInstance.delim, encodeNat.length]

end Config

def encodeRun (run : List Config) : Bitstring :=
  run.foldl1 (fun acc c => acc ++ Config.encode c) []

theorem encodeRun_cons (c : Config) (cs : List Config) :
  encodeRun (c :: cs) = Config.encode c ++ encodeRun cs := by
  simp [encodeRun, List.foldl1_cons, List.foldl1_nil, List.append_nil]

def decodeRun?Fuel : Nat → Bitstring → Option (List Config × Bitstring)
| 0, _ => none
| fuel + 1, [] => some ([], [])
| fuel + 1, s =>
  match Config.decode? s with
  | none => none
  | some (c, rest) =>
    match decodeRun?Fuel fuel rest with
    | none => none
    | some (run, rest') => some (c :: run, rest')

def decodeRun? (s : Bitstring) : Option (List Config × Bitstring) :=
  decodeRun?Fuel (s.length + 1) s

theorem decodeRun?Fuel_ge (fuel : Nat) (run : List Config)
  (h : (encodeRun run).length + 1 ≤ fuel) :
  decodeRun?Fuel fuel (encodeRun run) = some (run, []) := by
  revert fuel
  induction run with
  | nil =>
    intro fuel h
    cases fuel with
    | zero => omega
    | succ fuel => simp [encodeRun, decodeRun?Fuel]
  | cons c cs ih =>
    intro fuel h
    rw [encodeRun_cons] at h
    rw [List.length_append] at h
    have hpos := Config.encode_length_pos c
    cases fuel with
    | zero => omega
    | succ fuel =>
      have hcs := ih fuel (by omega)
      have hlen : 0 < (c.encode ++ encodeRun cs).length := by
        rw [List.length_append]
        exact Nat.lt_of_lt_of_le hpos (Nat.le_add_right _ _)
      have hne : c.encode ++ encodeRun cs ≠ [] := List.ne_nil_of_length_pos
        hlen
      simp [encodeRun_cons, decodeRun?Fuel, Config.decode?_append, hcs, hne]

```

```

theorem decodeRun?_encodeRun (run : List Config) :
  decodeRun? (encodeRun run) = some (run, []) := by
  simp [decodeRun?]
  exact decodeRun?Fuel_ge _ _ (Nat.le_refl _)

def runSteps (run : List Config) : Nat :=
  run.length.pred

def runValid (M : NTM) (input : Bitstring) (run : List Config) : Bool :=
  match run with
  | [] => false
  | c0 :: cs =>
    let rec check (prev : Config) (rest : List Config) : Bool :=
      match rest with
      | [] => true
      | c :: rest' =>
        match Config.step M prev with
        | some next => decide (next == c) && check c rest'
        | none => false
    decide (c0 == Config.initial M input) && check c0 cs

def runAccepts (M : NTM) (input : Bitstring) (bound : Nat) (run : List Config) :
  Bool :=
  runValid M input run &&
  decide (runSteps run ≤ bound) &&
  match run.getLast? with
  | none => false
  | some c => decide (c.state == M.accept)

def verifyRun (inst : NBHInstance) (cert : Bitstring) : Bool :=
  match inst.ntm?, decodeRun? cert with
  | some M, some (run, rest) =>
    decide (rest = [] && runAccepts M inst.input inst.bound run)
  | _, _ => false

/-! ### NBH languages -/

def NBH (inst : NBHInstance) : Prop :=
  match inst.ntm? with
  | none => False
  | some M => ∃ run : List Config, runAccepts M inst.input inst.bound run

def NBHSemantic : Set Bitstring :=
  { s | ∃ inst, NBHInstance.encode inst = s ∧ NBH inst }

def nbhCertBound (n : Nat) : Nat := (n + 1) ^ 2 * 4 + 1

theorem nbhCertBound_poly : IsPolynomial nbhCertBound := by
  refine ⟨100, 2, fun n => ?_⟩
  simp only [nbhCertBound]
  have h : (n + 1) ^ 2 * 4 + 1 ≤ 100 * n ^ 2 + 100 := by
    cases n with
    | zero => decide
    | succ n => nlinarith
  exact h

```

```

def verifyNBH (x cert : Bitstring) : Bool :=
  match NBHInstance.decode? x with
  | none => false
  | some (inst, rest) =>
    decide (rest = [] ∧ len cert ≤ nbhCertBound (len x) ∧ verifyRun inst cert =
      true)

theorem verifyNBH_true_iff (x cert : Bitstring) :
  verifyNBH x cert = true
  match NBHInstance.decode? x with
  | none => False
  | some (inst, rest) =>
    rest = [] ∧ len cert ≤ nbhCertBound (len x) ∧ verifyRun inst cert =
      true := by
    simp [verifyNBH, decide_eq_true_iff]
    split <;> simp [decide_eq_true_iff, and_assoc]

def NBHChecker : Set Bitstring :=
  { s | ∃ cert, verifyNBH s cert = true }

theorem NBHChecker_in_NP : InNP NBHChecker :=
  InNP.intro nbhCertBound_poly fun x => by
    constructor
    ũ intro ⟨cert, hver⟩
    cases dec : NBHInstance.decode? x with
    | none =>
      have hf : verifyNBH x cert = false := by simp [verifyNBH, dec]
      rw [hf] at hver
      cases hver
    | some p =>
      obtain ⟨inst, rest⟩ := p
      have hb := (verifyNBH_true_iff x cert).mp hver
      simp [dec] at hb
      exact ⟨cert, hb.2.1, hver⟩
    ũ intro ⟨cert, _, hver⟩
    exact ⟨cert, hver⟩

/-! ### POL-rankable  $\mu_0$  -/

def trivialInstance : NBHInstance :=
  { machineId := 0, input := [], bound := 0 }

def  $\mu$ Support : Finset Bitstring :=
  {NBHInstance.encode trivialInstance}

theorem  $\mu$ Support_nonempty :  $\mu$ Support.Nonempty :=
  ⟨NBHInstance.encode trivialInstance, by simp [ $\mu$ Support]⟩

noncomputable def  $\mu$  : Distribution :=
  uniformOn  $\mu$ Support  $\mu$ Support_nonempty

theorem  $\mu$ _polRankable : IsPolRankable  $\mu$  :=
  IsPolRankable.uniformOn_polRankable  $\mu$ Support  $\mu$ Support_nonempty

noncomputable def nbhProb : DistributionalProblem :=
  { L := NBHChecker,  $\mu$  :=  $\mu$  }

```

```

theorem nbhProb_in_DistNP : InDistNP nbhProb :=
  InDistNP.intro NBHChecker_in_NP  $\mu$ _polRankable

def trivialCert : Bitstring :=
  encodeRun [Config.initial trivialAcceptNTM []]

theorem trivialInstance_in_NBHChecker :
  NBHInstance.encode trivialInstance  $\in$  NBHChecker := by
  refine ⟨trivialCert, ?_⟩
  native_decide

theorem  $\mu$ _mass_on_trivial :
   $\mu$ .prob (NBHInstance.encode trivialInstance) = 1 := by
  simp [ $\mu$ , uniformOn, uniformProb,  $\mu$ Support]

end NBH

```

0.8.6 AvgCaseMls/Reduction.lean

AvgCaseMls/Reduction.lean

Lean 4 Certificate

```

/-
Copyright (c) 2026 Catskills Research Company. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Lars Warren Ericson, Catskills Research Company
-/

import AvgCaseMls.NBH
import AvgCaseMls.NPMembership
import AvgCaseMls.EMLS

/-!
Phase **4B:** distributional reduction from NBH (Phase **4A**) into MLS
satisfiability.

Literature: TR1995-711 §3.2 reduction with domination. The general TM→MLS
translation for
arbitrary MLS formulas in paper scope is axiomatized as ['nbhToMlsMap']; see
['DEFINITION_FORKS.md'] (../DEFINITION_FORKS.md).
-/

namespace Reduction

open MLS NBH AvCom EMLS

/-!
**Lean fork (general case):** ['nbhToMlsMap'] stands in for the full TR1995-711
compiler from
NBH instances to serialized MLS formulas (any expression in the paper's MLS
fragment). The
step-function ['reduceNBHToSatMLSStep'] remains as an explicit domination
scaffold on  $\mu$ .

```

```

-/

axiom nbhToMlsMap : Bitstring → Bitstring

axiom nbhToMlsMap_correct :
  ∀ x, x ∈ NBHChecker  nbhToMlsMap x ∈ SatMLSChecker

axiom nbhToMlsMap_lenBound :
  ∃ k0 k1 : Nat, ∀ x, lenBot (nbhToMlsMap x) ≤ k0 * (lenBot x) ^ k1

axiom nbhToMlsMap_domination :
  ∃ c0 c1 : Nat, 0 < c0 ∧ 0 < c1 ∧
    ∀ x, rank μ (nbhToMlsMap x) ≤ c0 * (lenBot x) ^ c1 * rank μ x

/-!
**Physical cost model (CookLevin style, TR1995-711 §3.2).**

For an NBH instance with state-space '|Q|', input length 'n', and step bound 't', the
constructed MLS formula has bit-length bounded by 'O(|Q| * t * (n + t))': each
of the
'O(t)' configuration snapshots contributes 'O(|Q| + n + t)' bits under the NBH/
MLS
encodings, and the formula size is linear in the certificate length.

This growth is **quadratic in the instance description** ('lenBot x' is
polynomial in
'|Q|', 'n', and 't'), which supports ['nbhToMlsMap_lenBound'] and the polynomial
length/domination constraints in ['DistributionalReduction'].
-/

def satTargetFormula : Formula :=
  Formula.rel (Relation.eq Term.empty Term.empty)

def unsatTargetFormula : Formula :=
  Formula.rel (Relation.neq Term.empty Term.empty)

def satTargetEnc : Bitstring :=
  serializeFormula satTargetFormula

def unsatTargetEnc : Bitstring :=
  serializeFormula unsatTargetFormula

theorem satTargetEnc_ne_unsatTargetEnc : satTargetEnc ≠ unsatTargetEnc := by
  native_decide

theorem satTargetEnc_in_checker : satTargetEnc ∈ SatMLSChecker := by
  rw [← verifySatMLS_true_iff]
  native_decide

theorem satTargetEnc_in_SatMLS : satTargetEnc ∈ SatMLS := by
  refine ⟨satTargetFormula, rfl, ?_⟩
  refine ⟨fun _ => ZFSet.empty, ?_⟩
  simp [evalFormula, evalTerm, satTargetFormula, Relation.eq]

theorem unsatTargetEnc_not_in_checker : unsatTargetEnc ∉ SatMLSChecker := by

```

```

intro h
have hver : verifySatMLS unsatTargetEnc [] = true := (verifySatMLS_true_iff
  unsatTargetEnc).mpr h
have hf : verifySatMLS unsatTargetEnc [] = false := by
  simp [verifySatMLS, unsatTargetEnc, unsatTargetFormula, serializeFormula,
    decodeFormula?, decideMLSSat, formulaToConjunct?, decideConjunct,
    relationToLiteral?, literalToFormula]
  native_decide
rw [hf] at hver
exact nomatch hver

/-! ### Target distributional problem -/

def  $\mu$ Support : Finset Bitstring :=
  {satTargetEnc}

theorem  $\mu$ Support_nonempty :  $\mu$ Support.Nonempty :=
  ⟨satTargetEnc, by simp [ $\mu$ Support]⟩

noncomputable def  $\mu$  : Distribution :=
  uniformOn  $\mu$ Support  $\mu$ Support_nonempty

theorem  $\mu$ _polRankable : IsPolRankable  $\mu$  :=
  IsPolRankable.uniformOn_polRankable  $\mu$ Support  $\mu$ Support_nonempty

noncomputable def satMLSProb : DistributionalProblem :=
  { L := SatMLSChecker,  $\mu$  :=  $\mu$  }

theorem satMLSProb_in_DistNP : InDistNP satMLSProb :=
  InDistNP.intro SatMLSChecker_in_NP  $\mu$ _polRankable

/-! ### Reduction map -/

/--
Step-function scaffold on  $\mu$  support (domination witness only; not globally
correct).
-/
def reduceNBHToSatMLSStep (x : Bitstring) : Bitstring :=
  if x ∈  $\mu$ Support then satTargetEnc else unsatTargetEnc

/--
Distributional reduction map used in ['nbhToSatMLS_red']: axiomatized general TM
→MLS translation.
-/
noncomputable def reduceNBHToSatMLS : Bitstring → Bitstring := nbhToMlsMap

namespace reduceNBHToSatMLSStep

theorem on_ $\mu$ Support (x : Bitstring) (hx : x ∈  $\mu$ Support) :
  reduceNBHToSatMLSStep x = satTargetEnc := by
  simp [reduceNBHToSatMLSStep, hx]

theorem off_ $\mu$ Support (x : Bitstring) (hx : x ∉  $\mu$ Support) :
  reduceNBHToSatMLSStep x = unsatTargetEnc := by
  simp [reduceNBHToSatMLSStep, hx]

```

```

end reduceNBHToSatsMLStep

/ -! ### Rank helpers for singleton uniform distributions -/

namespace Distribution

theorem mem_support_of_prob_pos ( $\mu$  : Distribution) (x : Bitstring) (h : 0 <  $\mu$ .
  prob x) :
  x  $\in$   $\mu$ .support := by
  by_contra hx
  exact not_lt.mpr (by simp [ $\mu$ .prob_zero_outside x hx]) h

theorem uniformOn_prob_pos {S : Finset Bitstring} (h : S.Nonempty) {x :
  Bitstring} (hx : x  $\in$  S) :
  0 < (uniformOn S h).prob x := by
  have hcard : 0 < (S.card : Real) := Nat.cast_pos.mpr (Finset.card_pos.mpr h)
  simp only [uniformOn, uniformProb, hx]
  exact div_pos zero_lt_one hcard

theorem uniformOn_prob_zero {S : Finset Bitstring} (h : S.Nonempty) {x :
  Bitstring} (hx : x  $\notin$  S) :
  (uniformOn S h).prob x = 0 := by
  simp [uniformOn, uniformProb, hx]

end Distribution

theorem rank_pos_of_prob_pos ( $\mu$  : Distribution) (x : Bitstring) (h : 0 <  $\mu$ .prob
  x) :
  0 < rank  $\mu$  x := by
  unfold rank
  split_ifs with h0
   $\mathbb{U}$  rw [h0] at h
  norm_num at h
   $\mathbb{U}$  have hx : x  $\in$   $\mu$ .support.filter (fun z =>  $\mu$ .prob x  $\leq$   $\mu$ .prob z) := by
    simp [Finset.mem_filter, Distribution.mem_support_of_prob_pos  $\mu$  x h,
    le_refl]
  exact Finset.card_pos.mpr  $\langle$ x, hx $\rangle$ 

theorem  $\mu$ _rank_on_support (x : Bitstring) (hx : x  $\in$   $\mu$ Support) :
  rank  $\mu$  x = 1 := by
  have hle : rank  $\mu$  x  $\leq$  1 := by
    have := rank.le_support_card  $\mu$  x
    simp [ $\mu$ , uniformOn,  $\mu$ Support, hx] at this
    exact this
  have hge : 1  $\leq$  rank  $\mu$  x := by
    have hprob : 0 <  $\mu$ .prob x := Distribution.uniformOn_prob_pos  $\mu$ 
    Support_nonempty hx
    have : 0 < rank  $\mu$  x := rank_pos_of_prob_pos  $\mu$  x hprob
    omega
  omega

theorem  $\mu$ _rank_off_support (x : Bitstring) (hx : x  $\notin$   $\mu$ Support) :
  rank  $\mu$  x = 0 :=
  rank.zero  $\mu$  x (Distribution.uniformOn_prob_zero  $\mu$ Support_nonempty hx)

theorem  $\mu$ _rank_on_target : rank  $\mu$  satTargetEnc = 1 := by

```

```

have hle : rank  $\mu$  satTargetEnc  $\leq 1$  := by
  have := rank.le_support_card  $\mu$  satTargetEnc
  simp [ $\mu$ , uniformOn,  $\mu$ Support] at this
  exact this
have hge :  $1 \leq$  rank  $\mu$  satTargetEnc := by
  have hprob :  $0 < \mu$ .prob satTargetEnc :=
    Distribution.uniformOn_prob_pos  $\mu$ Support_nonempty (by simp [ $\mu$ Support])
  have :  $0 < \text{rank } \mu \text{ satTargetEnc} := \text{rank\_pos\_of\_prob\_pos } \mu \text{ satTargetEnc hprob}$ 
  omega
omega

theorem  $\mu$ _rank_off_target (x : Bitstring) (hx : x  $\in \mu$ Support) :
  rank  $\mu$  x = 0 :=
  rank.zero  $\mu$  x (Distribution.uniformOn_prob_zero  $\mu$ Support_nonempty hx)

/-! ### Domination -/

theorem reduce_domination (x : Bitstring) :
  rank  $\mu$  (reduceNBHToSatsMLSSStep x)  $\leq 1 * (\text{lenBot } x) ^ 1 * \text{rank } \mu \text{ x} :=$  by
  by_cases hx : x  $\in \mu$ Support
  û rw [reduceNBHToSatsMLSSStep.on_ $\mu$ Support x hx,  $\mu$ _rank_on_target,  $\mu$ 
    _rank_on_support x hx]
  simp only [one_mul, pow_one]
  exact Nat.le_mul_of_pos_left 1 (lenBot_ne_zero x)
  û rw [reduceNBHToSatsMLSSStep.off_ $\mu$ Support x hx,  $\mu$ _rank_off_support x hx]
  have hunsat : unsatTargetEnc  $\mu$ Support := by
    intro hmem
    simp [ $\mu$ Support] at hmem
    exact satTargetEnc_ne_unsatTargetEnc hmem.symm
  rw [ $\mu$ _rank_off_target unsatTargetEnc hunsat]
  simp

/-! ### Correctness (scaffold) -/

theorem reduce_correct_on_ $\mu$ Support (x : Bitstring) (hx : x  $\in \mu$ Support) :
  x  $\in$  NBHChecker reduceNBHToSatsMLSSStep x  $\in$  SatsMLSChecker := by
  have heq : x = NBHInstance.encode trivialInstance := by
    simp [ $\mu$ Support] using hx
  subst heq
  constructor
  û intro _
    simp [reduceNBHToSatsMLSSStep.on_ $\mu$ Support _ hx, satTargetEnc_in_checker]
  û intro _
    exact trivialInstance_in_NBHChecker

theorem reduce_correct (x : Bitstring) :
  x  $\in$  NBHChecker reduceNBHToSatsMLS x  $\in$  SatsMLSChecker :=
  nbhToMlsMap_correct x

/-! ### Distributional reduction -/

theorem nbhToSatsMLS_red : DistributionalReduction nbhProb satsMLSProb := by
  refine (reduceNBHToSatsMLS, reduce_correct, ?_, ?_)
  û exact nbhToMlsMap_lenBound
  û exact nbhToMlsMap_domination

```



```

theorem nbhToSatMLS_red_on_μ (x : Bitstring) (hx : x ∈ μSupport) :
  x ∈ NBHChecker reduceNBHToSatMLSStep x ∈ SatMLSChecker :=
  reduce_correct_on_μSupport x hx

end Reduction

```

0.8.7 AvgCaseMls/Completeness.lean

AvgCaseMls/Completeness.lean

Lean 4 Certificate

```

/-
Copyright (c) 2026 Catskills Research Company. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Lars Warren Ericson, Catskills Research Company
-/

import AvgCaseMls.Reduction

/-!
Phase **4C:** NP-average completeness of MLS satisfiability (TR1995-711
Corollary 5.1).

Literature: every distNP problem reduces to bounded halting (NBH); Phase **4B**
reduces NBH
into ['satMLSProb']. Universal reduction into NBH and reduction transitivity
remain scaffold
gaps see ['DEFINITION_FORKS.md'](../DEFINITION_FORKS.md).
-/

namespace Completeness

open Reduction AvCom NBH

/--
Levin universal reduction: every distNP problem reduces to bounded halting (NBH)
.

Literature: TR1995-711 / Levin; full constructive proof deferred.
-/
axiom distNP_reduces_to_nbh (source : DistributionalProblem) (h : InDistNP
source) :
  DistributionalReduction source nbhProb

theorem nbhProb_NPAverageComplete : IsNPAverageComplete nbhProb :=
  IsNPAverageComplete.intro nbhProb_in_DistNP distNP_reduces_to_nbh

/--
Corollary 5.1 (adapted): ['satMLSProb'] is NP-average complete, via NBH
completeness and
['nbhToSatMLS_red'].
-/
theorem satMLSProb_NPAverageComplete : IsNPAverageComplete satMLSProb :=

```

```

    IsNPAverageComplete.of_reductor satMLSProb_in_DistNP nbhProb_NPAverageComplete
      nbhToSatMLS_red

end Completeness

```

0.8.8 Phase 5 — Hardness and non-AvP consequences

The 1995 paper proves that the satisfiability of MLS formulas is **NP-average complete**. Under the defined AvCom classes, this implies that MLS cannot belong to AvP under certain rankable distributions unless the nondeterministic and deterministic exponential-time hierarchies collapse.

We represent this structurally in Lean 4:

0.8.9 AvgCaseMls/ComplexityAxioms.lean

AvgCaseMls/ComplexityAxioms.lean

Lean 4 Certificate

```

/-
Copyright (c) 2026 Catskills Research Company. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Lars Warren Ericson, Catskills Research Company
-/

import AvgCaseMls.AvCom

open AvCom

/-!
Minimal complexity collapse hypothesis for conditional average-case hardness (
  Phase **5**).

Literature: TR1995-711 Corollary 5.1 consequence  NP-average complete targets
  are not in AvP
unless  $\text{NEXP} = \text{EXP}$ . Mathlib does not yet host this implication;
  we axiomatize
only the collapse hypothesis, not the full proof.
-/

/-- Nondeterministic exponential time is strictly larger than deterministic
  exponential time. -/
axiom NEXP_neq_EXP : Prop

def NEXP_eq_EXP : Prop := ¬ NEXP_neq_EXP

/--
Levin / TR1995-711 collapse equivalence: distNP is average-case tractable iff
  NEXP = EXP.

Literature: decades of structural complexity; full proof is out of scope for
  this project.
-/
axiom distNP_subseteq_AvP_iff_NEXP_eq_EXP :

```

```

(∀ p, InDistNP p → AvP p) NEXP_eq_EXP

/--
AvP pulls back along distributional reductions from a complete distNP target.

Literature: compose a poly-time decider for the target with the reduction map;
deferred until
['DistTime'] is linked to concrete deciders.
-/
axiom AvP_pullback {source target : DistributionalProblem}
  (hAvP : AvP target) (hRed : DistributionalReduction source target) :
  AvP source

```

0.8.10 AvgCaseMls/AverageHardness.lean

AvgCaseMls/AverageHardness.lean

Lean 4 Certificate

```

/-
Copyright (c) 2026 Catskills Research Company. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Lars Warren Ericson, Catskills Research Company
-/

import AvgCaseMls.Serialization
import AvgCaseMls.AvCom

/-!
Semantic language of satisfiable MLS formulas (Phase **2D** / §8).

Average-case hardness corollaries live in [AvgCaseMls/NonAvP.lean] (#avgcasemls-
nonavp-lean) (Phase **5**).
-/

open MLS AvCom

def SatMLS : Set Bitstring :=
  { s | ∃ (f : Formula), serializeFormula f = s ∧ ∃ (env : Env), evalFormula env
    f }

```

0.8.11 AvgCaseMls/NonAvP.lean

AvgCaseMls/NonAvP.lean

Lean 4 Certificate

```

/-
Copyright (c) 2026 Catskills Research Company. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Lars Warren Ericson, Catskills Research Company
-/

```

```

import AvgCaseMls.ComplexityAxioms
import AvgCaseMls.Completeness
import AvgCaseMls.AverageHardness

/-!
Phase **5A:** conditional non-AvP from NP-average completeness (TR1995-711 §3.2
/ Corollary 5.1).

Literature: if an NP-average complete problem were in AvP, bounded halting (NBH)
would be in AvP,
collapsing NEXP to EXP. See ['DEFINITION_FORKS.md'](../DEFINITION_FORKS.md).
-/

namespace NonAvP

open Completeness Reduction AvCom NBH MLS

theorem AvP_of_distNP_of_complete_target {target : DistributionalProblem}
(hComplete : IsNPAverageComplete target) (hAvP : AvP target) :
  ∀ source, InDistNP source → AvP source := by
  intro source hdist
  exact AvP_pullback hAvP (hComplete.2 source hdist)

theorem all_distNP_in_AvP_of_complete_target {target : DistributionalProblem}
(hComplete : IsNPAverageComplete target) (hAvP : AvP target) :
  ∀ p, InDistNP p → AvP p :=
  AvP_of_distNP_of_complete_target hComplete hAvP

theorem NEXP_eq_EXP_of_AvP_complete {target : DistributionalProblem}
(hComplete : IsNPAverageComplete target) (hAvP : AvP target) :
  NEXP_eq_EXP :=
  (distNP_subseteq_AvP_iff_NEXP_eq_EXP).mp (all_distNP_in_AvP_of_complete_target
    hComplete hAvP)

theorem not_AvP_of_NPAverageComplete {target : DistributionalProblem}
(hComplete : IsNPAverageComplete target) (h : NEXP_neq_EXP) :
  ¬ AvP target :=
  fun hAvP => (NEXP_eq_EXP_of_AvP_complete hComplete hAvP) h

theorem nbhProb_not_AvP (h : NEXP_neq_EXP) : ¬ AvP nbhProb :=
  not_AvP_of_NPAverageComplete nbhProb_NPAverageComplete h

theorem satMLSProb_not_AvP (h : NEXP_neq_EXP) : ¬ AvP satMLSProb :=
  not_AvP_of_NPAverageComplete satMLSProb_NPAverageComplete h

theorem nbhProb_not_AvP_via_complete (h : NEXP_neq_EXP) : ¬ AvP nbhProb :=
  not_AvP_of_NPAverageComplete nbhProb_NPAverageComplete h

/-- Simple POL-rankable distribution from Phase **4B** (uniform on ['
satTargetEnc']). -/
noncomputable def simpleSat $\mu$  : Distribution :=  $\mu$ 

theorem simpleSat $\mu$ _polRankable : IsPolRankable simpleSat $\mu$  :=  $\mu$ _polRankable

theorem simpleSat $\mu$ _prob_satTarget :
  simpleSat $\mu$ .prob satTargetEnc = 1 := by

```

```

simp [simpleSat $\mu$ ,  $\mu$ , uniformOn, uniformProb,  $\mu$ Support]

theorem exists_simple_rankable_checker_not_AvP (h : NEXP_neq_EXP) :
   $\exists \mu$ , IsPolRankable  $\mu \wedge \neg \text{AvP } \langle \text{SatMLSChecker}, \mu \rangle :=$ 
   $\langle \text{simpleSat}\mu, \text{simpleSat}\mu_{\text{polRankable}}, \text{fun hAvP} \Rightarrow$ 
    satMLSProb_not_AvP h (by simpa [satMLSProb] using hAvP)  $\rangle$ 

/-! ### Phase 5B  MLS average-case hardness corollaries -/

/--
Corollary 5.1 consequence (checker + Phase **4B** distribution): ['satMLSProb']
  is not in AvP
assuming NEXP  $\neq$  EXP.
-/
theorem SatMLS_average_hard (h : NEXP_neq_EXP) :  $\neg \text{AvP satMLSProb} :=$ 
  satMLSProb_not_AvP h

/--
Existential form: a simple POL-rankable distribution on MLS checker encodings is
  not AvP-tractable.
-/
theorem exists_simple_rankable_not_AvP (h : NEXP_neq_EXP) :
   $\exists \mu$ , IsPolRankable  $\mu \wedge \neg \text{AvP } \langle \text{SatMLSChecker}, \mu \rangle :=$ 
  exists_simple_rankable_checker_not_AvP h

/--
Semantic ['SatMLS'] on the same simple distribution ['AvP'] depends only on ['
  simpleSat $\mu$ ']
(see ['AvP.same_ $\mu$ ']), so checker hardness transfers directly.
-/
theorem SatMLS_semantic_not_AvP (h : NEXP_neq_EXP) :  $\neg \text{AvP } \langle \text{SatMLS}, \text{simpleSat}\mu \rangle$ 
  := by
  intro hAvP
  have hchecker : AvP satMLSProb := by
    simpa [satMLSProb, simpleSat $\mu$ ] using (AvP.same_ $\mu$  (L := SatMLS) (L' :=
      SatMLSChecker)).mp hAvP
  exact SatMLS_average_hard h hchecker

/-! ### Axiom audit (peer-review transparency) -/

#print axioms SatMLS_average_hard
#print axioms SatMLS_semantic_not_AvP

end NonAvP

```

0.9 9. Results

§9 is the **report card** for the proof program. Each row is a **subphase** from §1. **Outcome** is **TBD** while work is in progress, or one of the four accepted outcomes (*Proofs check*; *Lean is not expressive enough (yet)*; *Paper proofs are wrong*; *Field definitions are not solid*). Phase 0 (infrastructure) is complete and not graded here.

Phase	Phase goal	Outcome
1A	Bitstring, len, lenBot, Distribution, DistributionalProblem, IsPolynomial (ğ5); finite-support fork in DEFINITION_FORKS.md	Proofs check
1B	rank, T_inv without sorry; finite-support rank + partial T_inv in DEFINITION_FORKS.md	Proofs check
1C	IsAvTime, rankLe, DistTime, AvDTime, IsTRankable; forks in DEFINITION_FORKS.md	Proofs check
1D	AvP, InDistNP, DistributionalReduction, IsNPAverageComplete; forks in DEFINITION_FORKS.md	Proofs check
2A	MLS syntax + axiomatic semantics (ğ6)	Proofs check
2B	Literal, literalToFormula, conjunctToFormula, Literal.holds (ğ6)	Proofs check
2C	decideMLSSat, FOS80 Steps 2–4; sound + partial completeness on sound fragment (ğ7)	Proofs check (decideMLSSat_complete sorry)
2D	serializeFormula, SatMLS, stepsMLS (ğ8)	Proofs check
3A	SatMLSChecker_in_NP, decodeFormula?_serializeFormula; checker vs semantic SatMLS fork (ğ8)	Proofs check
3B	encodingBound, formulaSize_le_encodingBound, encodingBound_poly (ğ8)	Proofs check
4A	NBHChecker_in_NP, $\mu_{\text{polRankable}}$, nbhProb_in_DistNP, codec round-trip (ğ8)	Proofs check
4B	nbhToSatMLS_red, reduce_domination, reduce_correct (ğ8)	Proofs check (modulo nbhToMlsMap_* axioms)
4C	satMLSProb_NPAverageComplete, IsNPAverageComplete.of_reductor, DistributionalReduction.trans (ğ8)	Proofs check (modulo distNP_reduces_to_nbh axiom)
5A	not_AvP_of_NPAverageComplete, NEXP_eq_EXP_of_AvP_complete, nbhProb_not_AvP (ğ8)	Proofs check (modulo collapse axioms)
5B	SatMLS_average_hard, SatMLS_semantic_not_AvP, exists_simple_rankable_not_AvP (ğ8)	Proofs check

*Last updated: Phases 1A–1D, 2A–2D, 3A, 3B, 4A–4C, 5A–5B graded **Proofs check***

where noted (modulo named axioms in `DEFINITION_FORKS.md`).

0.10 10. Suggestions for Future Work

Building on this integration of automated theorem proving and structural complexity, several avenues for future work emerge:

1. **Formalizing Smoothed Analysis in Lean 4:** While average-case complexity under fixed distributions can be overly pessimistic, formalizing Spielman-Teng smoothed analysis would allow researchers to verify the typical-case tractability of modern SAT/SMT algorithms under random perturbations.
 2. **Verified SMT Solvers with Monadic Cost Models:** One could implement an executable SMT solver in Lean 4 (using a monadic state to track recursive steps) and formally prove that it runs in polynomial time on structured, non-random formula distributions.
 3. **Extending Mathlib’s Complexity Library:** The current complexity theory developments in Mathlib4 are focused on worst-case bounds. Standardizing Levin’s structural average-case reductions, the domination condition, DistTime, AvDTime, and AvP in Mathlib would provide a robust framework for certifying post-quantum security and for revisiting TR1995-711-style applied completeness proofs.
 4. **Step-counting the model-graph procedure:** Instrument `decideMLS` (or the full model-graph search) with a monadic step counter and prove $(\text{stepsMLS}, \mu) \in \text{Av}(\mathbf{T})$ for the rankable distributions used in the report—closing the loop between §5 complexity classes and §6 decision procedures.
-

0.11 Acknowledgements

The human authors retain sole responsibility for the mathematical content, definition forks, axioms, and every statement graded in §9. Following standard publisher practice (e.g., COPE guidance on authorship and AI tools [COPE24]), **no large language model is listed as a co-author**—authorship implies accountability that automated systems cannot bear.

We gratefully acknowledge assistance from the following tools:

Cursor ([Cur25]): agent-assisted editing in the Cursor IDE, including models routed through Cursor’s **Auto** agent mode (which may invoke Composer-family and other backend models depending on task). These agents helped draft and refactor Lean 4 modules, suggest proof and refactoring strategies, debug `lake` / type-class errors, maintain `./run_lean_check.sh` and smoke tests, and build the `arxiv_with_includes.md` pipeline. Generated Lean was treated as provisional until it compiled under CI and matched our forks in `DEFINITION_FORKS.md`.

Google Gemini 3.5 Flash ([Gem25]): independent technical briefs on Phases 4 and 5 (NBH codec invertibility, distributional-reduction transitivity, reduction correctness, and complexity-collapse axiomatization). Those briefs informed subsequent human-directed revisions; we did not adopt every recommendation verbatim (for example, we kept full `NBChecker` scope via an axiomatized general $\text{TM} \rightarrow \text{MLS}$ map rather than restricting to a singleton language).

All definitions, axiom choices, remaining `sorry` obligations, and final prose were reviewed and owned by the human authors. Intellectual property in the Lean codebase and this note rests with the authors under the project’s stated license.

0.12 References

- [Ajt96] Ajtai, M. (1996). Generating hard instances of lattice problems. *STOC*.
- [BDCGL89] Ben-David, S., Chor, B., Goldreich, O., & Luby, M. (1989). On the theory of average case complexity. *STOC*.
- [COPE24] Committee on Publication Ethics (COPE). (2024). Authorship and AI tools: COPE position statement. <https://publicationethics.org/guidance/cope-position/authorship-and-ai-tools>
- [Cur25] Anysphere, Inc. Cursor: AI-native code editor and agent environment. <https://cursor.com> (accessed 2025).
- [deM08] de Moura, L., & Bjørner, N. (2008). Z3: An efficient SMT solver. *TACAS*.
- [CEM95] Cox, J., Ericson, L., & Mishra, B. (1995). The average case complexity of multilevel syllogistic. *NYU Courant Institute Technical Report TR1995-711*.
- [DS77] Davis, M., & Schwartz, J. T. (1977). Metamathematical extensibility for theorem verifiers. *NYU Technical Report*.
- [FOS80] Ferro, A., Omodeo, E. G., & Schwartz, J. T. (1980). Decision procedures for elementary sublanguages of set theory. *CPAM*.
- [Gol79] Goldberg, A. T. (1979). On the complexity of the satisfiability problem. *NYU PhD Thesis*.
- [Gem25] Google DeepMind. (2025). Gemini model family (including Flash). Technical documentation and model cards. <https://ai.google.dev/gemini-api/docs/models>
- [Gur91] Gurevich, Y. (1991). Average case completeness. *Journal of Computer and System Sciences*.
- [Lev86] Levin, L. (1986). Average case complete problems. *SIAM Journal on Computing*.
- [Reg05] Regev, O. (2005). On lattices, learning with errors, and cryptography. *STOC*.
- [RS93] Reischuk, R., & Schindelhauer, C. (1993). Precise average case complexity. *STOC*.
- [SY92] Schnorr, C. P., & Yoshida, T. (1992). Average-case complexity of NP-complete problems. *STOC*.
- [Sny90a] Snyder, W. K. (1990). The SETL2 programming language. *NYU Technical Report*.
- [ST01] Spielman, D. A., & Teng, S. H. (2001). Smoothed analysis of algorithms. *STOC*.
- [VR92] Venkatesan, R., & Rajagopalan, S. (1992). Average case intractability of matrix and Diophantine problems. *STOC*.